

论文标题（待定）

摘要

本文以生鲜商超蔬菜补货与定价决策为研究对象，基于 2020 年 7 月至 2023 年 6 月的销售、批发与损耗数据，针对销售规律分析、品类级补货定价、单品级选品组合优化及数据改进建议四个问题，建立了融合统计分析与数学规划的优化决策框架，并对关键参数进行了灵敏度分析与模型检验。

针对问题一：构建了分布拟合、季节分解、协同网络与时序聚类四个子模型。品类日销量由伽马或对数正态分布拟合，单品日销量采用“售出概率 + 条件正销量”的两阶段混合分布；STL 分解显示季节强度介于 0.1729 至 0.2616 之间，趋势强度介于 0.5135 至 0.6268 之间；去日历效应后的 Spearman 检验在 132 个单品中识别出 1062 条主边（含 154 条稳健边）；六维时序特征的 Ward 层次聚类将单品分为五类需求形态。阈值扫描与抽样聚类（ARI=0.4567）验证了结构稳健性。

针对问题二：建立了 Log-Log 弹性回归、最优加价率求解、去价格预测与报童补货四模块。六品类价格弹性估计为花叶类-0.45、花菜类-0.50、水生根茎类-0.07、茄类-0.26、辣椒类-0.13、食用菌-0.31；因弹性绝对值均小于 1，最优加价率在历史区间内取数值边界解，且与批发价无关、可提前确定；报童模型给出未来一周各品类日补货量与定价策略，7 天总补货 2607.89 kg、总期望利润 5305.19 元。批发价 $\pm 10\%$ 扰动下利润变化小于 1%，报童较确定性基线多盈利 754.74 元。

针对问题三：建立了单品级选品、定价与补货的 0-1 混合整数规划（MIP）模型。以 2023 年 6 月 24—30 日可售品种为候选（45 个单品），采用经验贝叶斯收缩将单品弹性向品类弹性靠拢，收缩后弹性区间为 $[-2.52, -0.03]$ ；将连续售价离散为 11 档价格网格实现线性化；在选品数 27—33 个、最小陈列量 2.5 kg 约束下，求得 7 月 1 日最优方案：上架 33 个单品、6 品类全覆盖，总订购 296.83 kg、总利润 682.08 元、满足率 1.00。批发价为最敏感输入。

针对问题四：从需求侧、市场侧、运营侧、供给侧与外部环境五个维度提出五类数据采集方案：会员 ID 与购物篮数据基于 Lift 提升度识别互补/替代关系；缺货登记修正销量截断低估（ $D_{i,t}^* = Q_{i,t} + \alpha_i m_{i,t}$ ）；分时段库存与打折流水支撑日内动态定价；分环节损耗率升级为时变衰减函数；天气与节假日作为外生变量提升预测精度。五类数据协同构成商超数据体系升级方案。

关键词：报童模型；混合整数规划；价格弹性；Shrinkage 估计；组合优化；生鲜零售

一、引言

1.1 问题背景

生鲜商超是城市居民日常消费的重要渠道，其中蔬菜类商品因消费频率高、需求刚性强的特点，在商超经营中占据关键地位。然而，蔬菜类商品保鲜期短、品相变化快，商超需按日进行补货。并且，由于蔬菜品种与产地繁多，且进货交易时间集中在凌晨 3:00 至 4:00，商超往往在信息不完全的情况下做出当日补货决策。同时，蔬菜类商品在需求侧呈现出销售量与时间关联关系，在供给侧中则表现出季节性，以及与销售空间限制的紧密联系。在此背景下，如何依据历史销售数据、批发价格和损耗率等信息，科学制定蔬菜类商品的补货、定价、销售组合策略，已成为生鲜商超经营管理的核心问题之一。

1.2 问题重述

本题基于某商超经销的 6 个蔬菜品类的商品信息（附件 1）、2020 年 7 月 1 日至 2023 年 6 月 30 日的销售流水明细数据（附件 2）与批发价格数据（附件 3），以及各商品近期的损耗率数据（附件 4），解决以下四个问题：

问题一：要求基于附件 1、2 等数据来源，分析各蔬菜品类及单品在销售量上的分布规律，如时间分布特征（季节性、周期性等），并揭示相关关系，如不同品类之间、不同单品之间可能存在的互补或替代关系，为后续补货与定价决策提供依据。

问题二：要求基于附件 1、2、3、4 等数据来源，在品类层面上建立销售总量与成本加成定价之间的关系模型，并在此基础上确定未来一周（2023 年 7 月 1 日至 7 日）日补货总量和定价策略，以最大化商超的总收益。

问题三：要求基于附件 2 等数据来源，在可售单品总数控制在 27 至 33 个，且各单品订购量满足最小陈列量 2.5 千克的基础下，根据 2023 年 6 月 24 日至 30 日的可售品种，制定 7 月 1 日的单品补货量和定价策略，在尽量满足市场需求前提下，使得商超收益最大。

问题四：要求从优化制定蔬菜类商品补货和定价决策的角度出发，识别当前数据体系的不足，提出补充采集数据的意见和理由，并论述这些数据如何帮助解决上述问题。

1.3 问题分析

问题一：属于统计规律刻画与关联结构识别问题。以附件 1（6 品类、251 单品）与附件 2（2020-07-01 至 2023-06-30 销售流水， $T = 1095$ 天）为输入，输出品类与单品的销量分布形态、时间特征及协同关联。三个统计事实决定建模路径：(1) 品类日销量为

连续正值、右偏长尾，用对数正态/伽马分布拟合，AIC/BIC 选型；(2) 单品日销量存在大量无销售日，采用“售出概率 + 正销量条件分布”的两阶段混合模型；(3) 原始日销量相关混入共同日历与季节成分会产生伪相关 [1]，须先剔除时间效应再构建关联网络；单品销售形态差异悬殊，以时序特征聚类分组。据此拆解为分布拟合、STL 季节分解、去季节关联网络、时序特征聚类四个子模型。

问题二：属于品类层面的关系建模与随机优化决策问题。要求刻画销售总量与成本加成定价的关系（Log-Log 弹性回归），并给出未来一周各品类的日补货总量与定价策略使收益最大。三个难点决定模型结构：(1) 价格与销量的内生性：须先界定品类售价口径；(2) 时序约束：最优加价率 m_c^* 只依赖品类弹性与损耗率、与批发价无关；(3) 需求随机与损耗：按报童临界比 $\kappa = (P^* - c')/P^*$ 在缺货与损耗间权衡。按“弹性回归 → 最优定价 → 去价格需求预测 → 报童补货”四步建模，批发价以最近一周均价估计。

问题三：属于单品级联合组合优化问题。以 2023-06-24 至 30 可售品种为候选，同时决策单品是否上架、选哪档价、订多少货，满足可售总数 27–33、单品最小陈列量 2.5kg 的耦合约束。与问题二相比：(1) 决策粒度由品类下放到单品；(2) 跨单品共享约束使问题无法按品类独立优化 [2, 3]。三个对策：(1) 单品弹性借用品类弹性先验做 Shrinkage 估计 [4]；(2) 售价离散为 $K = 11$ 档价格网格，转化为线性 0-1 MIP 保证全局最优 [5]；(3) 以缺货惩罚软约束量化“满足需求”，配合灵敏度给出权衡 [6]。模型复用问题一的聚类分组与问题二的弹性、损耗参数。

问题四：属于数据体系诊断与采集方案建议问题。从优化补货定价决策出发，识别现有数据不足，提出补充采集建议。基于建模过程定位五类缺失：(1) 日销量聚合口径无法区分共同趋势与真实关联，购物篮关系不可识别；(2) 售价口径混合正常销售与打折价，弹性向零衰减；(3) 缺货日真实需求不可观测；(4) 损耗率为静态均值、缺乏动态结构；(5) 数据粒度为日级，缺少日内信息与外部竞品参照。据此从需求侧、市场侧、运营侧、供给侧与外部环境五个维度提出采集方案，每条按“补什么数据—改进哪个模型—改善哪个指标”的链条与前文挂钩。

二、模型假设

为简化问题，本文做出以下假设：

- **假设 1：单周期零库存假设。**每日凌晨到货的蔬菜品质一致，当日未售罄部分全额损耗，不结转次日。
- **假设 2：需求价格独立假设。**短期内定价调整不影响顾客对商超的长期忠诚度，即单日需求仅由当期价格决定。
- **假设 3：损耗率稳定假设。**附件 4 提供的近期损耗率在预测期（2023 年 7 月 1-7 日）内保持恒定。

- **假设 4：批发价可观测假设。**未来一周批发价采用最近可得均价（2023-06-24 至 06-30）作为已知常数，并以 $\pm 10\%$ 扰动做灵敏度检验。
- **假设 5：需求独立与无后向转移假设。**各日期的市场需求相互独立，当日因缺货未被满足的消费者需求不会顺延至次日。
- **假设 6：无日内清仓假设。**不考虑日内打折清仓，未售罄部分残值为零。

三、符号说明

表 1 符号说明

符号说明			符号说明		
符号	含义	单位	符号	含义	单位
i, c, t	品类/单品/日期索引	—	$Q_{c,t}, Q_{i,t}$	品类/单品日净销量	kg
$P_{c,t}, W_{c,t}$	品类加权售价/批发价	元/kg	L_i, L_c	单品/品类损耗率	—
β_c	品类价格弹性	—	$\hat{\beta}_i$	Shrinkage 修正后单品弹性	—
E_c	$ \beta_c $ ，定价用	—	$R_{c,t}$	品类日补货总量	kg
q_i	单品订购量	kg	y_i	0-1 上架决策变量	—
S	候选单品集	—	$p_{i,k}$	第 k 档候选价	元/kg
$x_{i,k}$	是否选第 k 档价	—	s_i	单品实际销量	kg
u_i	单品缺货量	kg	$z_{i,k}$	线性化辅助变量	kg
$D_{i,k}$	第 k 档需求点估计	kg	d_i	基准日需求	kg
p_i^0	参考价	元/kg	γ_i	单品缺货惩罚	元/kg
γ	缺货惩罚系数	元/kg	ρ	惩罚调节系数	—
M_i	订购量上界	kg	C_c	品类 c 单品集合	—
FR	需求满足率	—	$\Phi(\cdot)$	标准正态 CDF	—

四、问题一的模型建立与求解

4.1 模型准备

本问数据为附件 1 与附件 2：附件 1 提供 6 个蔬菜品类、251 个单品（246 个有售）的标识信息；附件 2 为 2020-07-01 至 2023-06-30 的销售流水，共 878,503 行，其中退货 461 行按负销量并入当日净销量口径。按“日期 $\times$ 单品”将流水聚合为单品日销量表（46,599 行），再按品类汇总为品类日销量表（6,474 行）；以完整日历（ $T = 1095$ 天）为基准补零对齐各序列，供季节分解、残差回归与特征计算使用。数据预处理代码见附录 2.1。

4.2 模型建立

4.2.1 品类与单品销量分布模型

品类日销量为连续正值、右偏长尾，选用对数正态分布（lognormal）与伽马分布（gamma）两类连续分布拟合，并以 AIC[7]/BIC[8] 选型。两类分布均固定位置参数 $\text{loc} = 0$ ，使支撑为 $(0, \infty)$ ，与销量下界为 0 相符；固定位置参数亦避免无意义的平移外推。两类分布自由参数个数相同（均为 2），AIC/BIC 可直接比较。

对数正态分布：若 $\ln X \sim N(\mu, s^2)$ ，则 $X \sim \text{LogNormal}(s, 0, \text{scale})$ ，其密度函数为

$$f(x) = \frac{1}{x s \sqrt{2\pi}} \exp\left(-\frac{(\ln x - \mu)^2}{2s^2}\right), \quad x > 0 \quad (1)$$

其中 s 为对数标准差， $\text{scale} = e^\mu$ 为尺度参数，且 $E[X] = \text{scale} \cdot e^{s^2/2}$ ， $\text{Var}(X) = \text{scale}^2(e^{s^2} - 1)e^{s^2}$ 。

伽马分布：固定 $\text{loc} = 0$ ，记为 $X \sim \text{Gamma}(a, 0, \text{scale})$ ，其密度函数为

$$f(x) = \frac{x^{a-1} e^{-x/\text{scale}}}{\Gamma(a) \text{scale}^a}, \quad x > 0 \quad (2)$$

其中 a 为形状参数， scale 为尺度参数，且 $E[X] = a \cdot \text{scale}$ ， $\text{Var}(X) = a \cdot \text{scale}^2$ 。当 $a > 1$ 时分布单峰右偏， a 越大分布越集中。

设某品类正销量样本 $x_1, \dots, x_n$ ，密度函数为 $f(x; \theta)$ ，参数 θ 由最大似然估计得到：

$$L(\theta) = \prod_{i=1}^n f(x_i; \theta), \quad \text{LLF} = \ln L(\hat{\theta}) = \sum_{i=1}^n \ln f(x_i; \hat{\theta}) \quad (3)$$

$$\text{AIC} = 2k - 2\text{LLF}, \quad \text{BIC} = k \ln n - 2\text{LLF} \quad (4)$$

其中 k 为自由参数个数， n 为正销量天数；AIC/BIC 取值越小表示模型越优，本文以 AIC 选型、BIC 交叉核对。拟合质量以 QQ 图诊断，以理论分位数与样本分位数的贴近程度检验分布假设。

单品日销量存在大量无销售日，采用两阶段模型 [9]：令 $Y_t = \mathbf{1}\{q_t > 0\}$ 表示第 t 日是否销售，则 $Y_t \sim \text{Bernoulli}(p_i)$ ，售出概率取经验频率 $p_i = n_i/T$ ；正销量条件分布 $q_t | Y_t = 1 \sim f(q; \theta_i)$ 按 AIC 在对数正态与伽马之间选优。单品日销量的整体分布为混合分布：

$$f(q) = (1 - p_i) \delta_0(q) + p_i f(q; \theta_i), \quad q \geq 0 \quad (5)$$

其中 δ_0 为 0 点的点质量。对 $n_i < 60$ 的稀疏单品不做参数估计，仅报告经验分位数 $Q_\tau = \inf\{x : F(x) \geq \tau\}$ （ $\tau = 0.5, 0.9, 0.99$ 对应 P50、P90、P99），避免小样本下参数估计与分布外推不可靠。

4.2.2 季节规律模型

品类日销量受周内与年内规律共同驱动。周内规律体现为工作日与周末的需求差异，对应 7 天周期；年内规律受供给与需求季节性影响，但观测期仅 3 年、365 天周期循环数不足，直接估计年周期不稳定。本文采用 STL[10, 11] 对品类日销量做加性分解以刻画周季节，并对年季节改用月度聚合：

$$y_t = T_t + S_t + R_t \quad (6)$$

其中 T_t 为趋势项， S_t 为季节项， R_t 为残差项。STL 通过内层与外层循环迭代执行 loess 平滑估计各分量，对离群值具有鲁棒性。本文取季节周期 $\text{period} = 7$ （周内季节）并启用 $\text{robust} = \text{True}$ ，以降低极端促销日对分量估计的影响。

以方差分解度量季节与趋势成分的相对强度 [12]：

$$F_s = \max\left(0, 1 - \frac{\text{Var}(R_t)}{\text{Var}(S_t + R_t)}\right), \quad F_t = \max\left(0, 1 - \frac{\text{Var}(R_t)}{\text{Var}(T_t + R_t)}\right) \quad (7)$$

F_s 、 $F_t \in [0, 1]$ ，越接近 1 表示该成分对总波动的贡献越大。

年季节按月份聚合平均日销量 $\bar{y}_{c,m}$ ，定义旺季为 4-10 月、淡季为 11-3 月，旺季比值

$$\rho_c = \frac{\bar{y}_{c,\text{peak}}}{\bar{y}_{c,\text{off}}} \quad (8)$$

$\rho_c > 1$ 表示品类 c 旺季平均日销量高于淡季。该指标用于检验题面“4-10 月供应丰富”的供给侧信息是否对应需求侧销量高峰。

4.2.3 去季节协同变动网络模型

单品日销量同时受日历效应与自身需求驱动，直接计算相关系数会产生由共同季节因素导致的伪相关 [1, 13]，故先剔除日历效应。对每个单品 i 拟合线性模型 [14]

$$q_{it} = \alpha_i + \sum_{k=1}^6 \delta_k D_{kt}^{\text{wk}} + \sum_{m=1}^{11} \eta_m D_{mt}^{\text{mo}} + \varepsilon_{it} \quad (9)$$

其中 D_{kt}^{wk} 、 D_{mt}^{mo} 分别为星期与月份虚拟变量，参数由最小二乘估计；残差 $e_{it} = q_{it} - \hat{q}_{it}$ 为去日历效应后的销量序列。

对残差序列按日秩变换后取 Pearson 相关，得到 Spearman 秩相关系数 [15]

$$r_{ij} = \text{corr}(\text{rank}(e_i), \text{rank}(e_j)) \quad (10)$$

秩相关对极端销量与分布形态不敏感，比 Pearson 相关更稳健。在原假设 $r_{ij} = 0$ 下，检验统计量 $t = \frac{r_{ij}\sqrt{n-2}}{\sqrt{1-r_{ij}^2}} \sim t_{n-2}$ ，双侧 p 值为 $p = 2(1 - F_t(|t|; n-2))$ 。对全部 m 个单品对的 p 值做 BH-FDR 校正 [16]：

$$q_{(i)} = \min_{j \geq i} \left\{ \frac{m}{j} p_{(j)} \right\} \quad (11)$$

其中 $p_{(1)} \leq \dots \leq p_{(m)}$ 为排序后的 p 值。建边条件为 $|r_{ij}| \geq 0.4$ 且 $q_{ij} < 0.05$ 。随后将样本按时间分为前半（2020-07-01 至 2021-12-31）与后半（2022-01-01 至 2023-06-30），分别执行相同的残差化与秩相关检验，若某条边在两半中均满足 $p < 0.05$ 且符号与全样本一致，则记为稳健边。前后半检验保证网络结构不依赖单一时段，避免将偶然相关误判为稳定关系。

以单品为节点、稳健边为边构建无向图 [17, 18]，边权为相关系数，按符号区分正负相关；节点中心性采用度中心度与介数中心度 [19, 20]，用于识别销量协同变动中的关键单品。

4.2.4 单品时序特征聚类模型

不同单品在销售频率、销量水平、波动幅度与日历响应上差异悬殊，直接以原始序列聚类易受噪声与量纲干扰。本文对每个单品 i 提取六维特征刻画其需求形态 [12]：售出天数占比 p_i 、有售日均销量 μ_i 、有售日变异系数 CV_i 、周末偏置 w_i 、节假日脉冲 h_i 与总销量占比 s_i ，分别反映“多久卖一次、一次卖多少、波动多大、何时集中、节假日响应与体量贡献”：

$$p_i = \frac{n_i}{T}, \quad \mu_i = \frac{1}{n_i} \sum_{q_{it} > 0} q_{it}, \quad CV_i = \frac{1}{\mu_i} \sqrt{\frac{1}{n_i} \sum_{q_{it} > 0} (q_{it} - \mu_i)^2} \quad (12)$$

$$w_i = \frac{\bar{q}_{i, \text{weekend}}}{\bar{q}_{i, \text{weekday}}} - 1, \quad h_i = \frac{\bar{q}_{i, \text{holiday}}}{\bar{q}_{i, \text{other}}} - 1, \quad s_i = \frac{\sum_t q_{it}}{\sum_j \sum_t q_{jt}} \quad (13)$$

其中均值均基于补零后的完整日序列计算，变异系数仅基于正销量样本以避免零值膨胀失真；节假日集合取中国法定节假日（含调休）。对六维特征做 **z-score** 标准化，消除量纲差异后计算欧式距离，采用 **Ward** 最小方差法层次聚类 [21–23]，使簇内离差平方和最小、簇间差异最大；对 $k \in \{3, \dots, 10\}$ 分别计算 **silhouette** 系数 [24]，取 **silhouette** 最大且不小于 0.15 的 k 为最优簇数。稳定性评估采用 80% 抽样，重复聚类 50 次，以调整兰德指数（ARI）[25] 度量抽样结果与全量聚类的一致性，取平均报告。

4.3 模型求解与结果分析

4.3.1 分布拟合结果

各品类正销量天数分别为 1085、1084、1085、1050、1085、1085，合计 6,474。AIC 选型结果：花叶类、花菜类、水生根茎类、茄类为伽马分布，辣椒类、食用菌为对数正态分布；AIC 与 BIC 选型完全一致，参数估计结果见表 2。

伽马分布的形状参数揭示品类内部的集中程度：花叶类 $a = 5.2599$ ，分布相对集中，说明日销量围绕均值波动较小；水生根茎类 $a = 1.6376$ ，右偏更明显，存在更多高销量

表 2 品类日销量的最优分布与 AIC/BIC

品类	最优分布	形状/对数标准差	尺度参数	AIC	BIC
花叶类	gamma	5.2599	34.8338	12444.912	12454.891
花菜类	gamma	2.9925	12.8900	9548.503	9558.480
水生根茎类	gamma	1.6376	22.8694	9898.119	9908.098
茄类	gamma	2.8055	7.6219	8059.611	8069.525
辣椒类	lognorm	0.5365	72.8523	11037.887	11047.866
食用菌	lognorm	0.6123	58.3107	10841.520	10851.499

日。对数正态的形状参数 s 刻画尾部厚度：辣椒类 $s = 0.5365$ 尾部相对较薄，食用菌 $s = 0.6123$ 略厚，说明食用菌出现极端高峰日的概率更高。

246 个有售单品中，132 个进入参数拟合（伽马 81 个、对数正态 51 个），114 个稀疏单品仅报告经验分位数，另有 5 个单品无销售记录。

图 1 的 QQ 图诊断显示各品类中部拟合良好、两端略有偏离，分布主体可由所选分布近似 [26]。

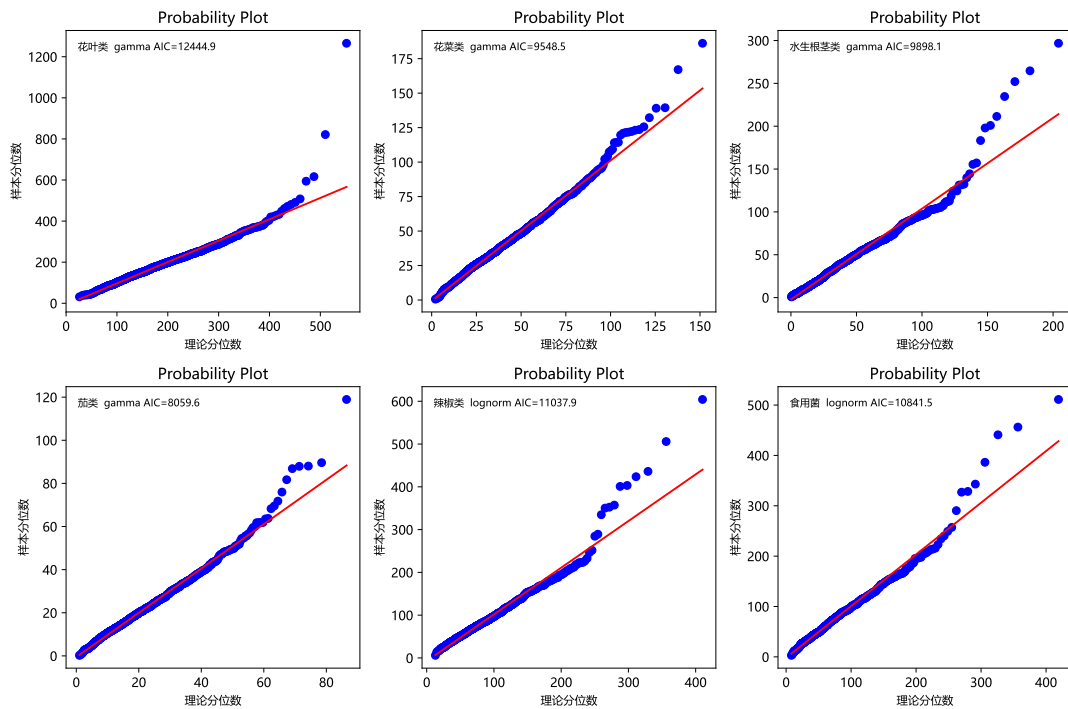


图 1 品类日销量最优分布 QQ 图

4.3.2 季节规律结果

各品类季节强度介于 0.1729 至 0.2616，趋势强度介于 0.5135 至 0.6268，品类日销量以趋势成分为主、周季节成分中等偏弱；旺季检验结果见表 3。

表 3 品类季节强度、趋势强度与旺季检验结果

品类	季节强度	趋势强度	旺季平均日销量	淡季平均日销量	旺季比值
花叶类	0.2114	0.5518	188.110	171.871	1.0945
花菜类	0.2182	0.5135	38.311	37.975	1.0089
水生根茎类	0.1729	0.5389	30.206	46.748	0.6462
茄类	0.2385	0.6268	24.015	15.582	1.5412
辣椒类	0.2001	0.5174	78.951	90.792	0.8696
食用菌	0.2616	0.5729	56.150	88.527	0.6343

旺季比值超过 1 的品类为花叶类、花菜类与茄类；水生根茎类、辣椒类与食用菌淡季日销量反而更高。可见 4-10 月供给丰富并不必然对应销量高峰，问题二的定价与补货决策不能简单套用“旺季多备货”的经验规则。月度形态上，花叶类、花菜类、茄类在 7-8 月达年内高点，辣椒类、食用菌、水生根茎类在 1-2 月达年内高点。

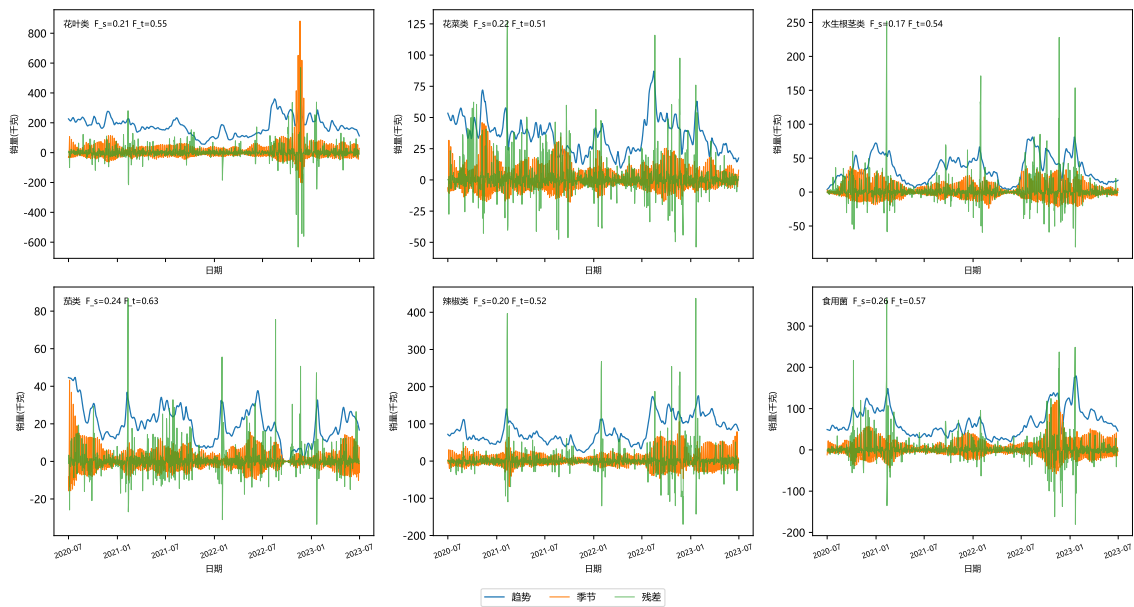


图 2 品类日销量 STL 分解图

图 2 显示季节分量呈明显周内起伏、残差围绕零波动。

4.3.3 协同变动网络结果

进入分析的 132 个单品共 8,646 个单品对。满足 $|r| \geq 0.4$ 且 $q < 0.05$ 的主边共 1,062 条，其中正相关 820 条、负相关 242 条；通过前后半样本检验的稳健边 154 条，构成包含 51 个节点的协同变动网络。相关系数阈值敏感性见表 4。

表 4 相关系数阈值敏感性分析

阈值	边数统计	正边数	负边数	稳健边数
0.3	1848	1289	559	249
0.4	1062	820	242	154
0.5	523	456	67	81

阈值从 0.3 提高到 0.5 时边数由 1848 降至 523，正边始终占多数，说明品类与单品销量以正向同步为主，负向关系仅存在于少数组合。品类级去季节残差的 Spearman 相关中，花叶类与食用菌、花叶类与辣椒类、辣椒类与食用菌相关较强 ($|r|$ 约 0.64)，茄类与其余品类相关最弱 ($|r| < 0.14$)，在销量上相对独立。单品级网络中，度中心度最高的单品为红椒 (1)、小米椒、青线椒等 (中心度 0.3 以上)，以正相关边为主。辣椒类、食用菌与花叶类单品之间呈现明显的正向同步变动，可作为销量同步变动的候选组合；是否构成需求互补，需结合问题二的交叉价格弹性进一步判定 [27, 28]。负相关边占比约 23% (242/1062)，可作为潜在替代关系候选。

图 3 展示了按度中心度选取的 15 个代表性单品构成的协同变动网络 (红色实线为正相关、蓝色虚线为负相关)。

4.3.4 单品聚类结果

132 个单品的最优簇数为 $k = 5$ ，对应 silhouette 系数 0.2583；50 次 80% 抽样聚类的平均 ARI 为 0.4567，聚类结构基本稳定。各簇画像见表 5。

表 5 单品聚类画像

簇	单品数	售出天数占比	有售日均销量	有售日变异系数	周末偏置	节假日脉冲	总销量占比	建议命名
1	13	0.3236	27.4374	0.6927	0.3466	0.3544	0.0207	高量周末脉冲型
2	7	0.8384	20.7024	0.7215	0.4876	0.5192	0.0414	高频平稳型
3	55	0.2053	4.7746	0.7616	0.2129	0.2126	0.0025	低频稀疏型
4	34	0.1908	6.1442	0.7681	0.5531	0.5963	0.0028	低频脉冲型
5	23	0.5631	7.3427	1.0807	0.3617	0.4005	0.0091	周末脉冲型

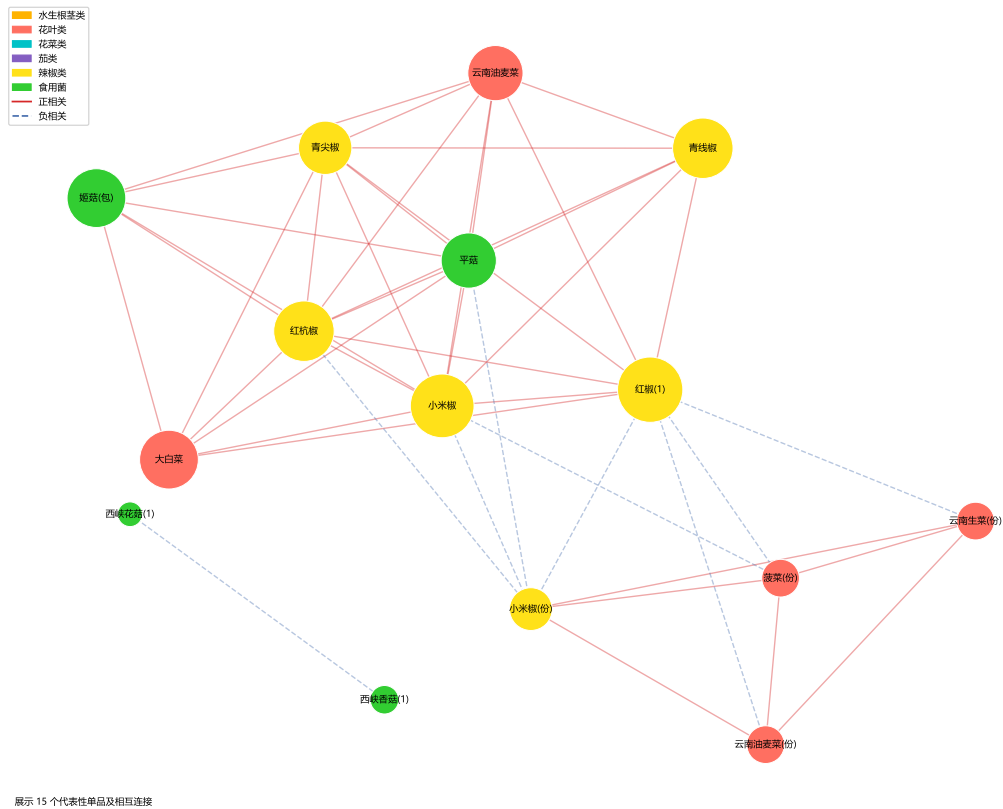


图 3 单品级协同变动网络

各簇特征差异明显：簇 1（高量周末脉冲型）呈“高量低频”特征，补货需预留较高单日峰值；簇 2（高频平稳型）适合作稳定的主销单品；簇 4（低频脉冲型）周末偏置与节假日脉冲最高，适合节假日与周末备货；簇 5（周末脉冲型）波动较大。低频稀疏类单品在选品与定价时需谨慎处理。聚类标签可直接为问题三提供候选单品集合与差异化定价依据。

其他图像（代表性单品分布图，月度柱状图，品类热图，silhouette 曲线，簇画像曲线分布结果和聚类结果）见附录 15。

4.4 小结

问题一以分布拟合、STL 季节分解、去季节协同网络与时序特征聚类四个子模型刻画销量规律：分布与季节结论为问题二提供需求分布依据与补货节奏约束；网络与聚类结论为问题三提供选品组合候选与差异化定价框架。四个子模型的可靠性已由第八章检验复核，可直接支撑后续建模。

五、问题二的模型建立与求解

5.1 模型准备

本问使用附件 1-4：品类日销量 $Q_{c,t}$ 、品类加权售价 $P_{c,t}$ 与批发价 $W_{c,t}$ 在附件 2、3 基础上沿用问题一的清洗与聚合口径构建（退货以负销量并入净销量），损耗率 L_c 由附件 4 单品损耗率按固定销量份额加权聚合。关键量的统计口径如表 6 所示。

表 6 问题二统计口径

量	口径	来源
$Q_{c,t}$ 品类日净销量	附件 2 按单品—品类映射聚合，退货以负销量并入当日净销量	附件 2
$W_{c,t}$ 品类批发价	附件 3 单品批发价按固定基期销量份额加权	附件 3
L_c 品类损耗率	附件 4 单品损耗率按固定销量份额加权： $L_c = \sum_i w_i L_i$ （百分数，代入公式前除以 100）	附件 4
$P_{c,t}$ 品类售价	单品售价按 固定基期销量权重 加权（Laspeyres 型价格指数）	附件 2

5.2 模型建立

5.2.1 需求—价格关系：品类 Log-Log 弹性回归

对品类 c 建立双对数（常弹性）回归：

$$\ln Q_{c,t} = \alpha_c + \beta_c \ln P_{c,t} + \delta_c t + \sum_{k=1}^6 \delta_{c,k} \mathbb{I}_{\{\text{weekday}=k\}} + \sum_{m=1}^{11} \eta_{c,m} \mathbb{I}_{\{\text{month}=m\}} + \varepsilon_{c,t}, \quad (14)$$

其中 β_c 为价格弹性（预期为负）， t 为线性趋势项（防止伪回归），星期虚拟变量（基准周一）与月份虚拟变量（基准 1 月）刻画周期结构， $\delta_{c,k}$ 、 $\eta_{c,m}$ 分别为星期与月份虚拟变量系数，记号与问题一残差回归（ δ_k 、 η_m ）一致。双对数形式下弹性为常数，便于直接外推“销量—价格”反事实，是销量—价格实证研究的标准形式 [29, 30]。

每个品类独立做 OLS 估计，报告弹性 $\hat{\beta}_c$ 、 t 值、 R^2 与样本量，并做残差平稳性（DW/ADF）检验防止伪回归 [14]。价格与当期需求同时决定可能带来内生性偏误，本文以滞后一期价格 $\ln P_{c,t-1}$ 的稳健性估计作对照 [14]；缺货日销量截断对弹性估计的影响作为局限写入小结。

价格口径与加价率的衔接：售价 $P = (1 + k)W$ ，故控制批发价 $\ln W$ 后 $\ln(1 + k)$ 的系数即 β_c ，即“批发价不变时，加价率弹性等于价格弹性”， β_c 同时刻画“销售总量对成本加成加价率”的敏感程度；反事实示例需注明“批发价不变”。

5.2.2 最优定价：成本加成加价率解析解

有效进货成本。损耗率 L_c 表示进货量中最终损耗的比例，售出 1 kg 需进货 $1/(1 - L_c)$

kg，故单位售出成本为 [31–34]

$$c'_{c,t} = \frac{W_{c,t}}{1 - L_c}. \quad (15)$$

注意附件 4 的损耗率为百分数（如 12.83 表示 12.83%），代入上式前需除以 100，否则有效成本为负、临界比越界。

常弹性最优加价率。在参考点 $(\bar{Q}_c, \bar{P}_c)$ 处把需求近似为常弹性函数 $Q(P) = \bar{Q}_c(P/\bar{P}_c)^{\beta_c}$ ，单日利润为 [35]

$$\pi(P) = (P - c'_{c,t}) \bar{Q}_c \left(\frac{P}{\bar{P}_c} \right)^{\beta_c}. \quad (16)$$

对 P 求导并令导数为零（记 $E_c = |\beta_c| > 1$ 时为极大值）：

$$\frac{d\pi}{dP} = \bar{Q}_c \left(\frac{P}{\bar{P}_c} \right)^{\beta_c} \left[1 + \beta_c \frac{P - c'_{c,t}}{P} \right] = 0 \implies P_{c,t}^* = \frac{E_c}{E_c - 1} c'_{c,t} = \frac{E_c}{E_c - 1} \cdot \frac{W_{c,t}}{1 - L_c}. \quad (17)$$

对应批发价加价率（即“成本加成”倍率）为

$$m_c^* = \frac{P_{c,t}^*}{W_{c,t}} - 1 = \frac{E_c}{(E_c - 1)(1 - L_c)} - 1. \quad (18)$$

该式等价于垄断定价的逆弹性规则（Lerner 条件） $(P^* - c')/P^* = 1/E_c$ [28, 36]，且解析最优价处报童临界比恰为 $\kappa^* = (P^* - c')/P^* = 1/E_c$ ，与 5.2.4 节衔接。

需说明： $\kappa^* = 1/E_c$ 仅在解析最优解处成立；本问各品类弹性均小于 1、落入数值求解分支（5.3 节），临界比一律按定义 $\kappa = (P^* - c')/P^*$ 直接计算（表 8）。

关键性质与决策规则： m_c^* 只依赖弹性与损耗率、与批发价 $W_{c,t}$ 无关。因此商超可在凌晨未知当日批发价时预先确定品类加价率策略，到货后按实际批发价确定售价 $P_{c,t}^* = (1 + m_c^*)W_{c,t}$ ——既对应题面“成本加成定价”，也化解了“凌晨 3:00–4:00 不知具体进货价”的时序约束。

边界处理与数值求解：

1. 若历史估计 $E_c \leq 1$ （无弹性或弱弹性），解析解不存在或无最大值，改在历史加价率百分位区间 $[m_{5\%}, m_{95\%}]$ 内以 201 点等距网格数值搜索最优加价率 [29]；
2. 网格搜索中利润一律按**截断后的最终售价** $P = \min(\max((1 + k)W, P_5), P_{95})$ 评估，需求仍以最终售价为自变量，避免“评估价与最终售价脱节”；解析解分支亦将最优价截断在历史价格区间 $[P_5, P_{95}]$ 内，防止常弹性外推越界。

5.2.3 基准需求预测：去价格序列 + 趋势—周期外推

去价格变换。先用第 5.2 节估计的弹性 $\hat{\beta}_c$ 把历史销量折算到统一参考价 $\bar{P}_c$ （取历史加权均价）下的“无价格效应需求”：

$$Q_{c,t}^0 = Q_{c,t} \left(\frac{\bar{P}_c}{P_{c,t}} \right)^{\hat{\beta}_c}. \quad (19)$$

因 $\hat{\beta}_c < 0$ ，历史价高于参考价的日子需求被上调、低于参考价的日子被下调，序列 Q^0 保留趋势/星期/季节结构但剔除价格影响，避免价格效应被重复计入 [11, 37]。

外推预测。对去价格序列 $Q_{c,t}^0$ 建模并外推未来一周参考价下的基准需求 $\bar{Q}_{c,t}$ 与残差标准差 $\sigma_{c,t}$ （模型设计以 Prophet 为基准，含周与年季节项；代码实现为等价的含线性趋势、星期与月份虚拟变量的回归外推，并对数均值修正，即 $\bar{Q} = \exp(\widehat{\ln Q} + \text{MSE}/2)$ 、 $\sigma = \bar{Q}\sqrt{\exp(\text{MSE}) - 1}$ ，两种实现等价 [11, 37]）。

最优定价下的需求分布（乘性缩放）。在最优售价 P^* 下，需求按弹性整体缩放，该乘性需求假设已在模型假设中说明（价格只改变需求水平、不改变变异系数）[35]：

$$\mu_{c,t}(P^*) = \bar{Q}_{c,t} \left(\frac{P_{c,t}^*}{\bar{P}_c} \right)^{\hat{\beta}_c}, \quad \sigma_{c,t}(P^*) = \sigma_{c,t} \left(\frac{P_{c,t}^*}{\bar{P}_c} \right)^{\hat{\beta}_c}. \quad (20)$$

5.2.4 报童补货模型

设进货可售量 $y = (1 - L_c)R$ ，需求分布为 F （均值为 μ 、标准差为 σ 、价格为 P^* ），单位售出成本 $c' = W/(1 - L_c)$ ，残值为 0。报童模型的最优条件对任意需求分布成立 [6, 35, 38, 39]：

$$E\pi = P^* \cdot E[\min(D, y)] - c' \cdot y. \quad (21)$$

最优可售量满足分位数条件 $F(y^*) = \kappa_{c,t}$ ，临界比为

$$\kappa_{c,t} = \frac{P_{c,t}^* - c'_{c,t}}{P_{c,t}^*} = \frac{P_{c,t}^* - W_{c,t}/(1 - L_c)}{P_{c,t}^*}. \quad (22)$$

分布口径与问题一衔接：问题一按 AIC 为各品类从伽马、对数正态两类右偏连续分布中选择拟合（花叶类、花菜类、水生根茎类、茄类为伽马，辣椒类、食用菌为对数正态）。为与代码实现一致，本问统一采用对数正态矩匹配分位数作为实现口径：由预测期均值 μ 与标准差 σ 矩匹配对数参数（ $\mu_{\ln} = \ln(\mu^2/\sqrt{\mu^2 + \sigma^2})$ 、 $\sigma_{\ln} = \sqrt{\ln(1 + \sigma^2/\mu^2)}$ ）后计算 $y^* = \exp(\mu_{\ln} + \sigma_{\ln} \Phi^{-1}(\kappa))$ ，伽马与对数正态分位数之差并入近似误差。

$$y_{c,t}^* = \mu_{c,t}(P^*) + \sigma_{c,t}(P^*) z_c(\kappa_{c,t}), \quad (23)$$

其中 $z_c(\kappa) = F_c^{-1}(\kappa)$ 为品类 c 拟合分布标准化后的 κ 分位数；不依赖参数分布时，用历史残差的经验分位数 $\hat{F}^{-1}(\kappa)$ 替代 [40]。每日品类补货总量为

$$R_{c,t} = \frac{y_{c,t}^*}{1 - L_c} = \frac{\mu_{c,t}(P^*) + \sigma_{c,t}(P^*) z_c(\kappa_{c,t})}{1 - L_c}. \quad (24)$$

其中除以 $(1 - L_c)$ 是把可售量折算回进货量，补偿损耗 [31, 32]。正态分布仅是 $z_c(\kappa) = \Phi^{-1}(\kappa)$ 的特例；右偏分布下同一 κ 的分位数与正态明显不同（如 $\kappa = 0.5$ 时正态取均值、对数正态取中位数）[40]。

$z_c(\kappa) < 0$ 表示“负安全库存”，即可售量 y^* 低于需求均值 $\mu(P^*)$ 。对右偏分布均值高于中位数， $z_c(\kappa) < 0$ 的临界 κ 大于 0.5；本问 6 品类临界比均小于 0.5，均处于“负安

全库存”区域——需求波动大且超量损耗成本高时，以适度缺货风险换取更低的超量损耗成本，属报童最优条件的正常结果。

确定性对照：忽略需求波动（ $z_c = 0$ ）时的补货量为 $R_{c,t}^0 = \mu_{c,t}(P^*)/(1 - L_c)$ ，作为“无安全库存”基准。

5.2.5 品类独立优化的合理性：相关性只进入方差、不进期望

问题一识别出品类销量存在协同变动，本节论证逐品类独立定价与补货不损失期望利润。记品类 c 的期望利润 $f_c(R_c) = E_{D_c}[\pi_c(R_c; D_c)]$ ，其只依赖品类 c 自身的边际需求分布 F_c （ π_c 中不含其他品类的销量或价格）。

命题（期望可加与可分最大化）：期望的线性性给出 $E[\sum_c \pi_c] = \sum_c E[\pi_c]$ ；又因每个 f_c 只含自己的决策 R_c 且可行集可分，和的极大化可逐项分解；该分解性是多品无约束报童问题的标准结论，即无容量约束时多品报童退化为相互独立的单品问题 [38, 41]：

$$\max_{R \in X} \sum_{c=1}^6 f_c(R_c) = \sum_{c=1}^6 \max_{R_c \in X_c} f_c(R_c). \quad (25)$$

推论（最优订货只依赖边际分布）：报童最优条件 $F_c(y_c^*) = \kappa_c$ 只含品类 c 的边际 CDF；品类间相关性刻画的是联合分布结构，不改变任一边际分布 F_c ，故不改变任一品类的最优订货量 y_c^* 与补货量 R_c^* （单周期报童模型的基本性质 [6, 35, 39]）。因此在风险中性假设下（题面“收益最大”即指最大化期望利润），逐品类独立优化与整体联合优化的最优决策相同：协方差矩阵既不进入临界比条件（5.2.4 节），也不进入最优价公式（5.2 节），最优补货量与最优售价均与品类间相关性无关；相关性只进入总利润的方差分解：

$$\text{Var}\left(\sum_c \pi_c\right) = \sum_c \text{Var}(\pi_c) + 2 \sum_{c < c'} \text{Cov}(\pi_c, \pi_{c'}). \quad (26)$$

正相关通过协方差项放大总利润波动，但不改变期望利润（组合理论的基本结论 [42]）。

结论成立的条件与失效情形：上述结论依赖三项条件——目标可加（各品类利润函数互不包含，需求函数无交叉价格项）、决策集可分（无共享资金、货架或订货容量约束；问题二成立，问题三不成立）、边际分布外生（品类 c 的需求边际分布不受其他品类定价决策的影响）。若商超风险厌恶或存在联合风险约束（资金预算、亏损概率上限），利润联动会改变最优决策 [43, 44]；存在共享容量约束时需联合优化 [41, 45]；需求存在替代/互补（涨价导致跨品类转移）时，独立定价亦非真实意义上的全局最优。其中边际分布外生最贴近本题现实：问题一的相关性分析只能识别“销量同步变动”、不能识别因果替代结构，交叉价格弹性建模留待问题四的数据扩展；本文在 8.2.2 节以交叉价格弹性联合定价模拟作为稳健性对照。

5.3 模型求解与结果分析

5.3.1 弹性估计结果

对 6 个品类分别估计 Log-Log 回归，结果如表 7 所示（样本期 1095 天， n 为各品类正销量日样本量；参考价为历史加权均价，参考需求为正销量日均值）。

表 7 品类价格弹性与定价参数

品类	价格弹性 $\hat{\beta}_c$	t 值	R^2	样本量 n	参考价 $\bar{P}_c$ (元/kg)	参考需求 $\bar{Q}_c$ (kg)	有效成本 c' (元/kg)	加价率
花叶类	-0.45	-6.79	0.40	1085	5.89	183.22	4.11	0.4
花菜类	-0.50	-5.45	0.31	1084	9.95	38.57	9.69	0.2
水生根茎类	-0.07	-0.70	0.61	1085	9.22	37.45	12.93	0.2
茄类	-0.26	-3.16	0.52	1050	8.80	21.38	4.41	0.3
辣椒类	-0.13	-3.03	0.45	1085	8.63	84.52	3.49	0.2
食用菌	-0.31	-5.28	0.41	1085	11.09	70.21	2.76	0.3

解读如下：

- 全部品类价格弹性均为负，花叶类（-0.45）、花菜类（-0.50）需求价格敏感度最高，水生根茎类（-0.07）几乎无价格弹性；除水生根茎类（ $t = -0.70$ ，不显著）外，其余品类弹性均在 1% 水平显著；
- 由于“批发价不变时加价率弹性等于价格弹性”（5.2 节），表 7 的 $\hat{\beta}_c$ 即销售总量对成本加成加价率的敏感度：花叶类加价率每提高 1%，销量约下降 0.45%；
- 所有品类 $E_c = |\beta_c| < 1$ ，均不满足解析解条件**，表 7 的最优加价率均为“历史加价率区间 $[m_{5\%}, m_{95\%}]$ 网格 201 点的数值最优”，即历史区间约束下的最优，而非无条件解析最优；水生根茎类弹性不显著，其定价结论的证据强度相应降低、加价率主要由历史区间约束决定，需谨慎解读。

图 4 中，水生根茎类拟合线接近水平、散点分散，直观反映其弹性不显著；其余品类散点整体呈负向倾斜，面板标注的 β 、 t 、 R^2 与表 7 一一对应。

5.3.2 最优定价策略

基于表 7 的弹性与加价率，代入最近一周批发价后得到未来一周固定的定价策略，如表 8 所示（批发价取 2023-06-24 至 06-30 最近均价；参考价 $\bar{P}_c$ 为历史加权均价；加价率与临界比在周内保持不变）。

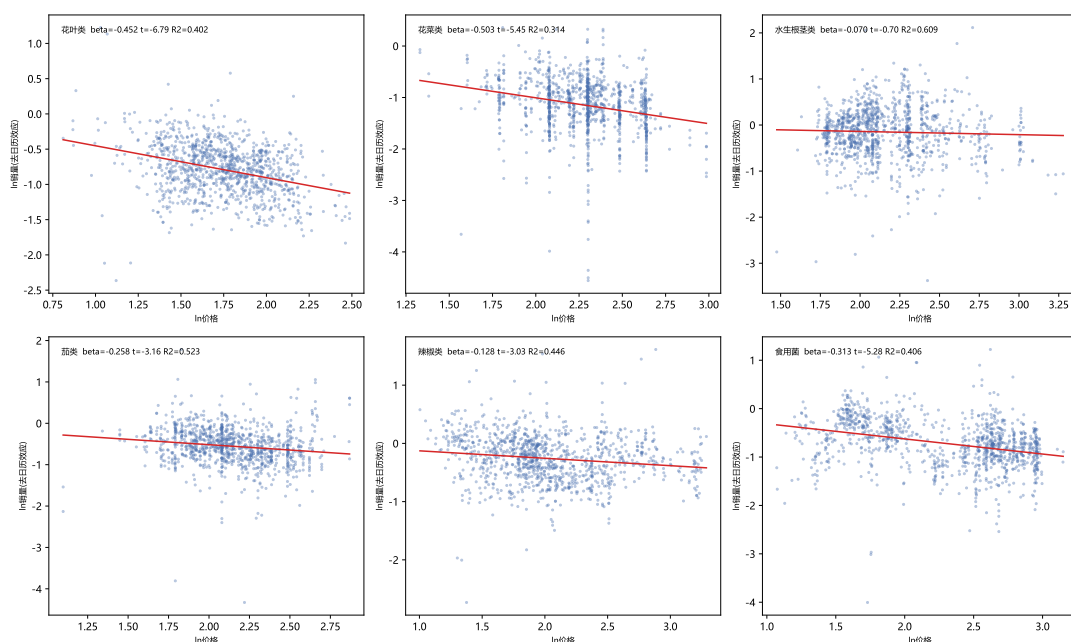


图 4 品类需求—价格弹性拟合（散点为去日历效应后的 $\ln$ 销量与 $\ln$ 价格，红色直线为弹性拟合线，标注 β 、 t 与 R^2 ）

表 8 品类最优定价策略（未来一周）

品类	批发价 W （元/kg）	参考价 $\bar{P}_c$ （元/kg）	最优售价 P^* （元/kg）	加价率 m^*	临界比 κ
花叶类	3.59	5.89	7.22	1.01	0.43
花菜类	8.19	9.95	14.00	0.71	0.31
水生根茎类	11.16	9.22	16.07	0.44	0.20
茄类	4.11	8.80	7.81	0.90	0.44
辣椒类	3.17	8.63	6.79	1.14	0.49
食用菌	2.50	11.09	4.53	0.82	0.39

由 $\frac{d\pi}{dP} = \bar{Q}_c (P/\bar{P}_c)^{\beta_c} (1 - E_c + E_c c'/P)$ 可见，当 $E_c = |\beta_c| < 1$ （需求缺乏价格弹性）时该导数对一切可行售价恒为正（因 $1 - E_c > 0$ 且 $E_c c'/P > 0$ ），价格上调的收入增量超过销量缩减造成的损失，利润函数在可行域内随价格单调上升，最优加价率落在可行域边界：花叶类、茄类、辣椒类、食用菌的加价率取到历史加价率上界 $m_{95\%}$ ；花菜类与水生根茎类的最优售价被约束在历史价格区间上界 P_{95} （分别为 14.00 元/kg 与 16.07 元/kg）。因此该策略的稳健解读是“在历史交易区间内使期望利润最大化的加价率”，而非无条件全局最优；区间边界敏感性见 8.1 节。

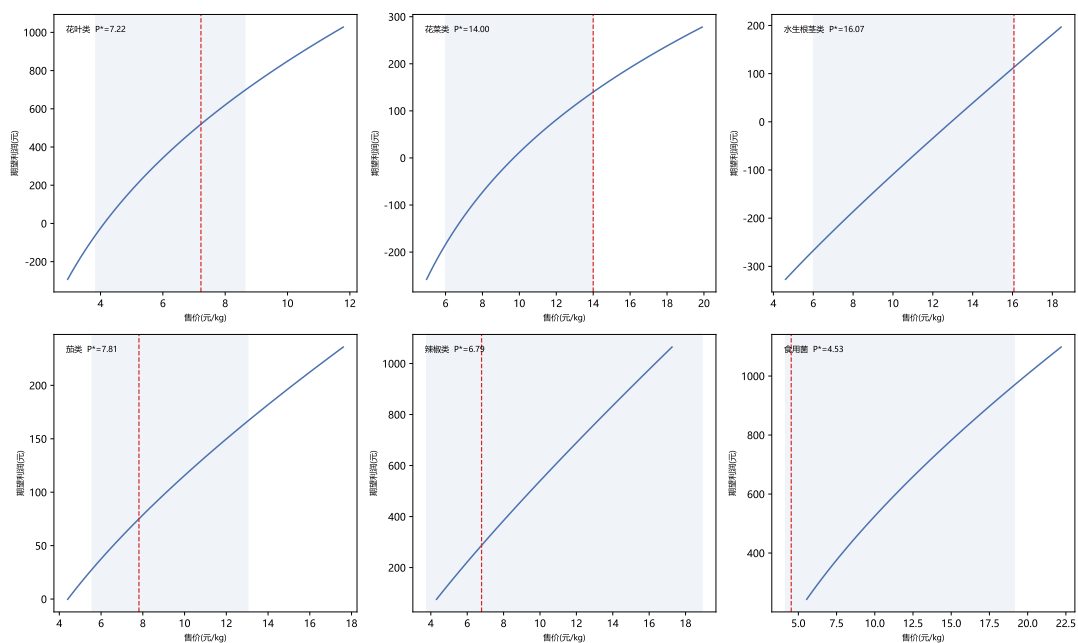


图 5 期望利润—售价曲线（蓝色阴影为历史价格 P_5 – P_{95} 区间，红色虚线为最优售价 P^* ）

图 5 展示了各品类的期望利润—售价曲线：利润随售价上升并在区间边界达到峰值，红色虚线标出表 8 的最优售价，与数值搜索结果一致。需注意，常弹性函数形式仅在参考点附近可信：脱离历史价格区间向外推时，需求在远高价区可能转为强弹性而出现新的峰值，故本文不宣称无条件全局最优。

5.3.3 需求预测与报童补货

先按 5.2 节在参考价下外推未来 7 天基准需求，再按最优售价做乘性缩放并代入报童模型，得到每日补货量与期望利润。

表 9 未来 7 天参考价下基准需求（中间输入，均值 $\pm$ 标准差，kg）

品类	07-01	07-02	07-03	07-04	07-05	07-06	07-07
花叶类	232.74 $\pm$ 84.74	223.95 $\pm$ 81.54	164.15 $\pm$ 59.76	159.99 $\pm$ 58.25	163.99 $\pm$ 59.71	160.14 $\pm$ 58.31	174.50 $\pm$ 63.5
花菜类	37.96 $\pm$ 21.84	35.94 $\pm$ 20.68	24.57 $\pm$ 14.14	25.18 $\pm$ 14.49	25.03 $\pm$ 14.40	24.54 $\pm$ 14.12	28.27 $\pm$ 16.2
水生根茎类	59.19 $\pm$ 35.83	54.36 $\pm$ 32.90	35.45 $\pm$ 21.46	35.90 $\pm$ 21.73	34.86 $\pm$ 21.10	34.61 $\pm$ 20.95	43.64 $\pm$ 26.4
茄类	30.93 $\pm$ 15.06	29.78 $\pm$ 14.50	22.47 $\pm$ 10.94	20.76 $\pm$ 10.11	20.82 $\pm$ 10.14	21.33 $\pm$ 10.39	24.22 $\pm$ 11.8
辣椒类	97.44 $\pm$ 41.05	92.44 $\pm$ 38.94	66.66 $\pm$ 28.08	64.74 $\pm$ 27.27	65.61 $\pm$ 27.64	65.30 $\pm$ 27.51	72.01 $\pm$ 30.3
食用菌	69.58 $\pm$ 35.20	65.99 $\pm$ 33.38	43.93 $\pm$ 22.22	42.21 $\pm$ 21.35	46.54 $\pm$ 23.54	44.90 $\pm$ 22.71	53.05 $\pm$ 26.8

表 10 未来 7 天每日补货总量 $R_{c,t}$ (kg)

品类	07-01	07-02	07-03	07-04	07-05	07-06	07-07	7 天合计
花叶类	215.13	207.00	151.72	147.88	151.58	148.02	161.29	1182.61
花菜类	25.07	23.74	16.23	16.63	16.53	16.20	18.67	133.07
水生根茎类	34.94	32.09	20.93	21.19	20.57	20.43	25.76	175.91
茄类	28.51	27.45	20.71	19.14	19.20	19.66	22.33	157.01
辣椒类	100.50	95.34	68.75	66.77	67.66	67.35	74.27	540.64
食用菌	79.55	75.44	50.22	48.26	53.21	51.33	60.65	418.66
合计	483.70	461.05	328.56	319.87	328.75	322.99	362.97	2607.89

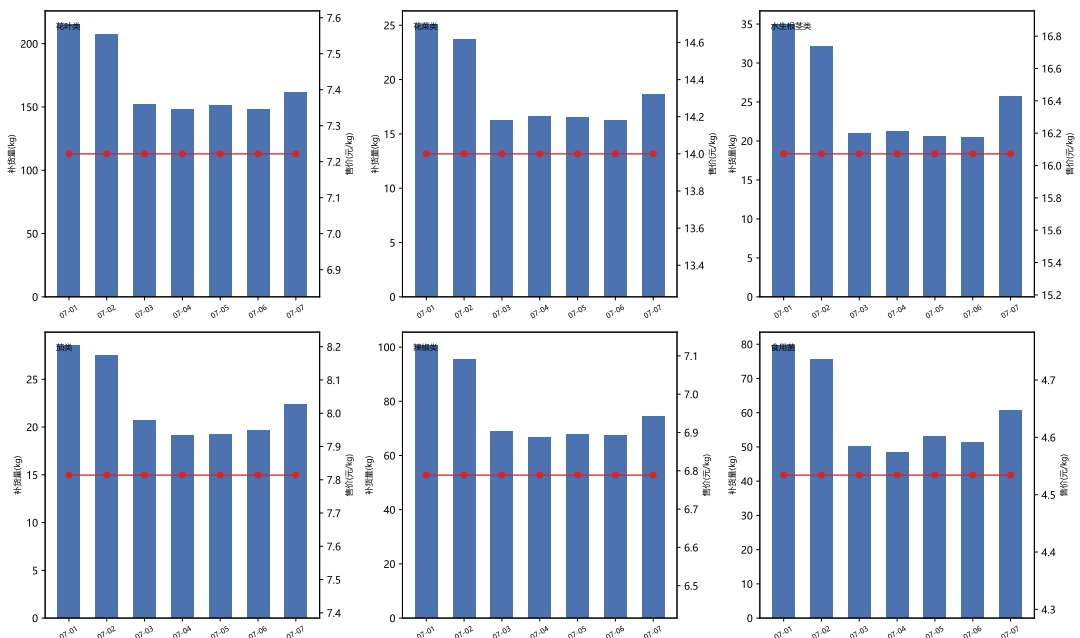


图 6 未来 7 天每日补货量（柱状图，左轴）与最优售价水平线（红色折线，右轴）

图 6 直观展示各品类补货量随周内需求节奏波动，同时售价在整个预测期内保持恒定（加价率策略周内固定）。主要结果解读如下：

- **总量与节奏：**未来一周总补货量 2607.89 kg、总期望利润 5305.19 元；补货量呈周末集中（7 月 1 日与 7 月 2 日合计 944.75 kg，占全周 36.2%）、周中回落、周五小幅回升的周内结构，与表 9 的“周末偏置”需求结构一致；
- **花叶类**为最大补货品类（7 天合计 1182.61 kg，占总量 45.3%），加价率 1.01、售价 7.22 元/kg，弹性 -0.45 意味着加价 1% 仅损失约 0.45% 销量，定价上调空间最大；
- **水生根茎类**补货量（175.91 kg）明显低于确定性基准（332.03 kg）：其临界比 $\kappa = 0.20$

为各品类最小、安全库存为负 ($z_c(\kappa) < 0$)，以适度缺货风险换取更低的超量损耗成本；

- **利润贡献：**花叶类（2517.33 元）与辣椒类（1209.04 元）合计占 7 天总期望利润的 70.2%，构成利润的主要来源。

5.3.4 决策时序与操作含义

策略执行分三步：（1）商超在 2023-06-30 依据表 7 的弹性与损耗率确定 6 个品类的加价率 m_c^* ，周内保持不变（对应“加价率与批发价无关”的性质，见 5.2 节）；（2）每日凌晨批发价到货后按 $P^* = (1 + m^*)W$ 确定实际售价（对应“凌晨未知当日进货价”的时序约束）；（3）按表 10 的每日补货量执行采购。批发价 $\pm 10\%$ 的扰动风险由 8.1 节灵敏度覆盖。

5.4 小结

问题二以“Log-Log 弹性回归 $\rightarrow$ 最优加价率 $\rightarrow$ 去价格预测 $\rightarrow$ 报童补货”四步完成品类级定价与补货决策。弹性回归表明，6 个品类需求价格弹性均为负，花叶类（-0.45）、花菜类（-0.50）弹性最强，水生根茎类（-0.07）弹性不显著；批发价不变时，价格弹性即销售总量对加价率的弹性。由于各品类 $|\beta_c| < 1$ ，最优加价率为历史加价率区间约束下的数值解——花叶类加价率 1.01（售价 7.22 元/kg）、花菜类 0.71（14.00 元/kg）、水生根茎类 0.44（16.07 元/kg）、茄类 0.90（7.81 元/kg）、辣椒类 1.14（6.79 元/kg）、食用菌 0.82（4.53 元/kg），且与批发价无关、可提前确定。未来一周总补货量 2607.89 kg、总期望利润 5305.19 元，补货节奏与问题一周内周期一致，水生根茎类因需求波动大、临界比最小而明显减少补货量。独立性论证表明，风险中性下逐品类独立优化即可达到期望利润最优，相关性只进入总利润方差；后 28 天滚动回测 MAPE 20.6%-37.2%、差分进化多种子结果一致、联合定价模拟显示交叉价格效应仅对最优价格水平产生边际影响，主结论稳健。

局限：水生根茎类弹性不显著，其定价结论证据不足；缺货日销量截断使补货量可能系统性偏低；固定权重价格指数因品类单品进出而不连续；模型不含日内清仓与跨期需求转移，风险中性假设下未考虑资金预算等联合风险约束。

六、问题三的模型建立与求解

6.1 模型准备

6.1.1 数据来源与整合

本问使用附件 1–4：单品—品类映射与单品名称来自附件 1；单品日净销量 $Q_{i,t}$ 与日售价 $p_{i,t}$ 由附件 2 按单品聚合（退货以负销量并入当日净销量）；单品批发价 W_i 来自附件 3（按单品 fill，仍缺失者用单品历史均值补齐）；损耗率 L_i 取自附件 4 单品损耗率（百分数，代入公式前除以 100），分类级损耗率用于交叉校验。

6.1.2 单品级需求模型：幂律需求与 Shrinkage 弹性

模型设定上，延续问题二的常弹性（幂律）需求结构 [29, 30]，单品 i 在售价 p 下的日需求为

$$D_i(p) = d_i \left(\frac{p}{p_i^0} \right)^{\hat{\beta}_i}, \quad (27)$$

其中 d_i 为参考价下的基准日需求（kg/日）， p_i^0 为参考价（元/kg）， $\hat{\beta}_i$ 为 Shrinkage 修正后的单品价格弹性（预期为负）。该式把问题二的品类弹性结构下放到单品，保证两问的需求函数口径一致。

单品弹性估计与 Shrinkage 收缩。对价格有变异的单品独立回归：

$$\ln Q_{i,t} = \alpha_i + \beta_{\text{raw},i} \ln p_{i,t} + \delta_i t + \sum_{k=1}^6 \delta_{i,k} \mathbb{1}_{\{\text{weekday}=k\}} + \sum_{m=1}^{11} \eta_{i,m} \mathbb{1}_{\{\text{month}=m\}} + \varepsilon_{i,t}, \quad (28)$$

其中 $\beta_{\text{raw},i}$ 为原始弹性。采用经验贝叶斯收缩 [4, 46, 47]：

$$\hat{\beta}_i = w_i \beta_{\text{raw},i} + (1 - w_i) \beta_{c(i)}, \quad w_i = \frac{N_i}{N_i + M_0}, \quad (29)$$

其中 N_i 为有效回归样本量（价格发生变动的观测天数）， $M_0 = 30$ 为先验权重常数。

弹性兜底：若 $N_i < N_{\min} = 14$ 或价格无变异，令 $\hat{\beta}_i = \beta_{c(i)}$ ；若收缩后 $\hat{\beta}_i > 0$ ，同样令 $\hat{\beta}_i = \beta_{c(i)}$ 。不设人工弹性下界，弱弹性单品最优价落在网格上界属正常边界解，与问题二口径一致。实测 45 个候选中 12 个借用品类弹性；选中 33 个单品中 10 个借用品类弹性，收缩弹性区间为 $[-2.52, -0.03]$ ，均值 -0.44。

6.1.3 基准需求与参考价口径

- d_i ：6/24–30 日均净销量；销售天数不足 3 天的单品用近 4 周日均可替代，并标注“低样本”；
- p_i^0 ：6/24–30 销量加权均价；
- W_i ：6/24–30 批发价均值；
- 价格百分位区间 $[P_5, P_{95}]$ 以近 4 周（2023-06-03 06-30）为基准。

6.2 模型建立

6.2.1 离散价格网格

把每个单品的价格区间取历史售价百分位区间 $[\min(P_5, p_i^0), \max(P_{95}, p_i^0)]$ ，离散为 $K = 11$ 档候选价 $p_{i,k}$ ($k = 1, \dots, K$)，并把参考价 p_i^0 显式并入网格（去重后取 K 档）。网格生成规则默认等距，灵敏度分析（详见第九章）对比 $K = 7$ 与分位数取点两种口径。若 $|\hat{\beta}_i| > 1$ ，内点最优价 $p_i^* = \frac{E_i}{E_i - 1} \cdot \frac{W_i}{1 - L_i}$ ($E_i = |\hat{\beta}_i|$) 可作为网格内插参考。

6.2.2 目标函数

$$\max \sum_{i \in S} \left(\sum_{k=1}^K p_{i,k} z_{i,k} - W_i q_i - \gamma_i u_i \right). \quad (30)$$

其中第一项为销售收入，第二项为进货成本（口径统一为“ q_i = 订购量”），第三项为缺货惩罚。单位缺货惩罚取机会损失毛利并截尾：

$$\gamma_i = \rho \cdot \max \left(0, p_i^0 - \frac{W_i}{1 - L_i} \right), \quad \rho \in \{0.5, 1.0, 1.5\}, \quad (31)$$

ρ 基准取 1.0，灵敏度分析见第九章。

6.2.3 约束条件

$$27 \leq \sum_{i \in S} y_i \leq 33, \quad (1)$$

$$2.5 y_i \leq q_i \leq M_i y_i, \quad \forall i \in S, \quad (2)$$

$$\sum_{k=1}^K x_{i,k} = y_i, \quad \forall i \in S, \quad (3)$$

$$s_i = \sum_{k=1}^K z_{i,k}, \quad s_i \leq q_i(1 - L_i), \quad z_{i,k} \leq D_{i,k} x_{i,k}, \quad z_{i,k} \geq 0, \quad (4)$$

$$u_i = \sum_{k=1}^K D_{i,k} x_{i,k} - s_i, \quad \forall i \in S, \quad (5)$$

$$\sum_{i \in C_c} y_i \geq 1, \quad \forall c. \quad (6)$$

需求满足率输出指标为

$$\text{FR} = \frac{\sum_i s_i}{\sum_i \sum_k D_{i,k} x_{i,k}}, \quad \text{FR}_c = \frac{\sum_{i \in C_c} s_i}{\sum_{i \in C_c} \sum_k D_{i,k} x_{i,k}}. \quad (32)$$

6.2.4 大 M 取值与求解算法

订购量上界取

$$M_i = \max \left(3 \cdot \max_t Q_{i,t}^{\text{hist}}, 2.5, 1.05 D_{i,1} \right), \tag{33}$$

其中 $D_{i,1}$ 为最低档价格下的需求。用 HiGHS (`scipy.optimize.milp`) 求解 0-1 MILP [48], 时间上限 60 秒, 报告求解状态、最优性 gap 与节点数。基准 case 求解状态为 optimal (status=0)、gap=0、节点数 1、无降级。

6.3 模型求解与结果分析

6.3.1 候选集与单品弹性估计结果

候选集构成与品类弹性见表 11。单品弹性诊断：45 个候选的有效样本量 N_i 为 28 559；13 个单品原始弹性为正，其中 12 个借用品类弹性；选中单品收缩弹性区间 $[-2.52, -0.03]$ ，均值 -0.44。

表 11 候选集构成与品类弹性

品类	候选单品数	品类弹性 β_c	弹性绝对值 $ \beta_c $
花叶类	16	-0.45	0.45
花菜类	2	-0.50	0.50
水生根茎类	6	-0.07	0.07
茄类	4	-0.26	0.26
辣椒类	9	-0.13	0.13
食用菌	8	-0.31	0.31
合计	45	—	—

6.3.2 组合优化求解与决策结果

MIP 一次求解即达最优，约束校验全部通过。问题三总体求解结果见表 12。

表 12 问题三总体求解结果

指标		指标	
候选单品数	45	总收入	1915.69 元
选中单品数	33	进货总成本	1233.61 元
总订购量	296.83 kg	缺货惩罚	0.00 元
损耗后可售量	271.19 kg	总利润	682.08 元
总销量/总需求	270.14/270.14 kg	需求满足率 FR	1.00
总缺货量	0.00 kg	陈列刚性超订量	1.16 kg (10.53 元)

6.3.3 品类结构与收益解读

候选与选中单品分品类构成见图 7

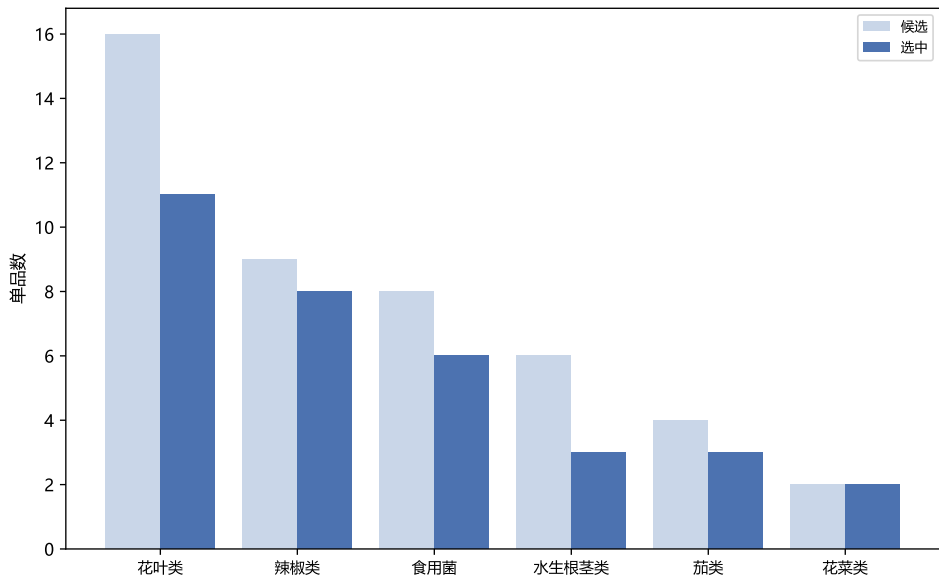


图 7 候选与选中单品的分品类构成（柱状图，浅色为候选、深色为选中）

分品类汇总见表 13。

表 13 分品类决策汇总

品类	选中数	订购量合计 (kg)	可售量合计 (kg)	销量 (kg)	需求 (kg)	毛利合计 (元)	毛利占比
花叶类	11	124.03	111.20	111.20	111.20	201.99	29.6%
辣椒类	8	81.35	74.18	73.76	73.76	220.56	32.3%
食用菌	6	43.89	42.60	42.60	42.60	91.30	13.4%
花菜类	2	15.90	14.42	14.42	14.42	80.11	11.7%
茄类	3	17.82	16.74	16.74	16.74	58.56	8.6%
水生根茎类	3	13.85	12.06	11.43	11.43	29.55	4.3%
合计	33	296.83	271.19	270.14	270.14	682.08	100.0%

主要结果解读：最优解取到选品数上界 33 个，6 个品类全部覆盖；总体与各品类满足率均为 1.00；利润来源方面，辣椒类（220.56 元）与花叶类（201.99 元）合计占 62%，小米椒 (份) 单品毛利 109.49 元；定价行为上，弱弹性单品售价落在网格上界附近，弹性充分单品（西峡花菇，-2.52）取参考档 24.00 元/kg。

8 展示售价—订购量散点。

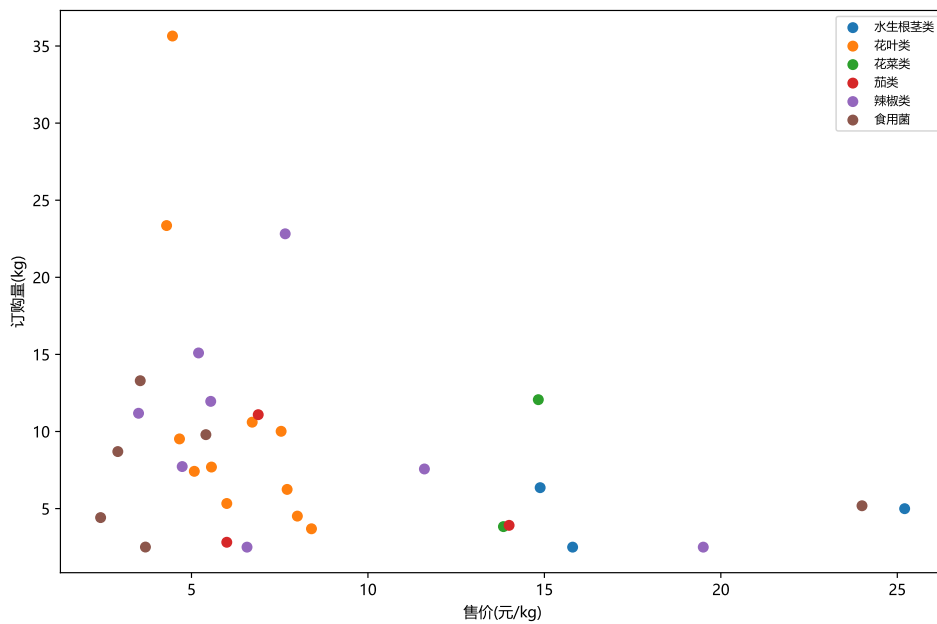


图 8 选中单品售价—订购量散点（按品类着色）

6.3.4 小结

问题三建立了单品级选品、定价与补货联合优化框架。以 45 个候选单品，在选品数 27–33、最小陈列量 2.5 kg 约束下，最优方案上架 33 个单品，总利润 682.08 元，满

足率 1.00。灵敏度分析详见第九章，结论稳健；与问题二交叉核对通过。操作层面应优先保障小米椒 (份)、西兰花等高毛利单品备货。

局限与展望：单品需求独立假设忽略了替代效应 [3, 49]；确定性需求可升级为单级报童 [35, 50]；净销量低估真实需求 [40]；未按周末偏置上调基准需求，结果偏保守；满足率为软惩罚，可改为硬约束或鲁棒选品 [3, 51]；未考虑动态折扣 [52] 与品类角色差异 [53]。

七、问题四的模型建立与求解

7.1 五类数据缺失

基于前三问的建模与求解过程，现有数据（附件 1 至附件 4）存在以下五个方面的缺失，制约了各模型的进一步优化改进：

1. 日销量聚合口径无法区分共同趋势与真实关联，购物篮层面的互补/替代关系不可识别；
2. 售价口径混入清仓打折价，价格弹性 $\hat{\beta}_c$ 向零衰减；
3. 缺货日真实需求不可观测，单品基准需求 d_i 系统性低估；
4. 损耗率为静态平均值，缺乏随批次品质与储存时长变化的动态结构；
5. 数据粒度为日级，缺日内信息与外部参照（竞品价、天气），无法支撑日内定价与去混淆估计。

7.2 数据采集建议

7.2.1 需求侧：会员 ID 与缺货登记（成本：中/低）

采集：POS 脱敏会员 ID（或家庭户标识）；收银台/小程序缺货登记次数。改进：以真实购物篮替代销量相关性做关联分析（问题一）；以缺货登记次数修正单品基准需求 d_i 与预测均值 $\mu_{c,t}$ ，缓解缺货截断导致的低估（问题三）；按会员分层（RFM+ 价格敏感度）实现分群促销（问题二）。风险：登记重复/误报、会员数据稀疏——按“会员 + 商品 + 时段”去重并设最小登记量阈值，稀疏会员仅聚合到分群。

7.2.2 市场侧：陈列拓扑与竞品实时均价（成本：中/高）

采集：单品货架位置坐标（空间距离）；周边可比商圈（如 3km 内）农贸市场与生鲜电商同品类零售均价（日采样）。改进：将问题三的独立需求扩展为含交叉价格弹性的联合需求（交叉弹性按货架距离加权），量化替代效应、避免选品“同类相食”；以竞品价为定价设置外部参照上限（可接受的相对溢价率约束）。风险：采样本底不一致、价格口径差异——只对同口径价格建模，缺失用品类均值填充并打缺失标记。

7.2.3 运营侧：分时段库存与日内打折流水（成本：高/低）

采集：RFID/电子秤分时段库存余量与鲜度评级；19:00 后清仓时段的具体折扣率与销量（附件 2 已有打折标记，只需补折扣率与时段字段）。改进：在日级报童补货决策之上叠加日内动态定价层（分时段折扣率决策），并把预期日内折价损失回传报童临界比 κ ，避免晚间积压报废。风险：时段库存测量噪声、鲜度评级主观——统一打分标准，折扣单单独标记。

7.2.4 供给侧：分环节损耗率与供应商履约指数（成本：中低/中）

采集：运输 → 入库 → 上架 → 清仓四环节分段损耗；供应商到货提前期与批次合格率。改进：将恒定损耗率 L_i 替换为随批次品质与储存时长变化的动态损耗，改进补货量折算；提前期不确定性引入安全库存，指导供应商选择与订货分配。风险：环节归属不清、提前期样本少——交接单逐节点签字确认责任，样本不少于 10 批再启用安全库存。

7.2.5 外部环境：天气、节假日与促销日历（成本：极低）

采集：日/小时级天气（温度、降水、极端天气哑变量）；节假日、节气与大型活动日历；自身促销计划（品类、力度、起止时间）。改进：作为外生变量进入需求—价格回归与预测，控制（去混淆）促销与天气的影响，避免弹性估计被外生波动混淆，并改善预测均值—方差结构；预测期遇极端天气自动上调补货量并放宽价格调整区间。风险：天气粒度不一致、促销与需求互为因果——用门店最近气象站数据，弹性估计以非促销样本为主。

7.3 实施优先级与 ROI 评估

按“决策增益—采集成本”排序：

1. **外部环境**：API 接入，极低成本，立即见效；
2. **需求侧**：缺货登记 POS 加字段，低改造成本；
3. **运营侧**：打折流水补字段，低成本增量；
4. **供给侧**：分环节损耗验收流程改造，中低成本；
5. **市场侧**：货架测绘 + 竞品采集，成本较高，建议中期落地。

会员 ID 属中期改造项，RFID 分时段库存属长期试点项。

7.4 小结

五维方案分别对应前三问的关键局限：需求侧解决关联识别与需求低估；市场侧弥补独立需求假设；运营侧修正混合口径偏误并升级日内定价；供给侧动态化损耗结构；外部环境提升预测精度并去混淆弹性估计。

八、模型的检验与评价

为检验结论的可信度及其成立条件，本章从灵敏度分析与模型检验两个层面展开验证：灵敏度分析通过对由数据估计或人为设定的关键参数施加合理扰动，考察最优决策与目标值是否发生量级性变化 [54]；模型检验则从分布诊断、样本外回测、基线对比、随机模拟与求解器状态校验等方面，检验模型假设、估计精度与求解可靠性。判定阈值均在运行前声明，检验契约（claim-perturbation-metric-threshold-observed-status）固化于 `q2_robustness_summary.json`，数值可追溯至对应 CSV 与脚本，随机模拟固定种子 `SEED=7`、每组样本量 $N = 20000$ ；量级稳定判据为：利润与补货量扰动分别不超过 30% 与 20%，回测 MAPE 不超过 40%，相关模拟期望利润偏差不超过 1%。

8.1 灵敏度分析

8.1.1 问题一：网络阈值与聚类稳定性

相关系数阈值取 0.3、0.4、0.5 时，主边数分别为 1848、1062、523 条，稳健边数分别为 249、154、81 条（表 26，见附录），边数随阈值单调下降、正负边占比方向稳定，表明“品类/单品间以正相关为主、少数负相关”的结构性结论不依赖于阈值选择；主口径 0.4 下 154 条稳健边构成 51 节点协同变动网络。聚类簇数扫描中，`silhouette` 系数在 $k = 5$ 处取得最大值 0.2583，超过 0.15 的选取下限（表 27，见附录）；对 132 个单品做 80% 抽样并重复聚类 50 次 [55]，平均调整兰德指数 $ARI=0.4567$ ，聚类结构中等稳定，且聚类标签与全样本独立重算的 Ward 聚类完全一致（特征最大偏差约 10^{-14} ）。

8.1.2 问题二：定价与补货参数灵敏度

对批发价、价格弹性、价格口径与损耗率四类关键输入施加扰动，统一以未来一周总补货量与总期望利润为口径（表 28、图 24，见附录）：

- **批发价** $\pm 10\%$ ：总补货量变化约 $\pm 5\%$ ，总利润变化 $+0.3\%/-0.7\%$ ，均小于 1%，策略稳健（PASS）；
- **弹性** $\pm 20\%$ ：扰动后各品类弹性绝对值最大约 0.60，最优加价率与售价不变，总利润变化 $-0.4\%/+0.5\%$ （PASS）；

- **价格口径切换：**当期销售额加权口径下弹性符号不变、绝对值增大，利润+26.7%，机制与主口径一致；中位价口径下水生根茎类、食用菌弹性符号翻转、利润虚高+72.8%，该口径不可采用（CONDITIONAL）；
- **损耗率替换：**以 2022-07 至 2023-06 单品销量份额加权损耗率替代固定基期份额，利润+7.5%、补货量-1.0%，量级不变（PASS）。

8.1.3 问题三：组合优化灵敏度

对缺货惩罚、选品数、弹性、批发价、损耗率、价格网格与大 M 七类设定逐一重解 MIP（表 29，见附录），全部情景求解状态 optimal、最优性 gap=0。缺货惩罚 ρ （0.5–1.5）与大 M 放大 50% 情景的最优解完全不变：前者因基准方案无缺货、惩罚项不进入目标值，后者结合订购量上界余量检查（最小余量 12.58 kg）确认可行域未被截断。选品数 27/30/33 对应利润 657.54/673.60/682.08 元；弹性 $\pm 20\%$ 使利润变化 $-1.6\%/+1.7\%$ ，方向与问题二一致；批发价 $\pm 10\%$ 为最敏感输入（利润 $-17.6\%/+18.1\%$ ），且 $\times 1.1$ 时 1 个低毛利单品被挤出；损耗率取分类级时利润-3.6%；价格网格 $K = 7$ 与分位数取点分别使利润 $-3.7\%/+4.5\%$ ，离散化影响控制在 $\pm 5\%$ 以内。

8.2 模型检验与误差分析

8.2.1 问题一：分布、季节与关联结构检验

六品类日销量按 AIC 选型为伽马（4 个）与对数正态（2 个），BIC 交叉核对一致，QQ 图无系统性偏离（图 1）；K-S 检验在参数估计后偏保守，仅作图形诊断。STL 分解的季节强度 0.1729–0.2616、趋势强度 0.5135–0.6268，与独立重算最大偏差 5×10^{-5} 。协同网络主边 1062 条中 154 条通过前后半样本检验成为稳健边，抽样 200 条边独立重算的相关系数最大偏差 5×10^{-5} ，FDR q 值与标准 BH 校正一致；聚类抽样稳定性 ARI=0.4567。

8.2.2 问题二：需求—价格模型、预测与报童检验

回归诊断表明，六品类价格弹性 t 值分别为 -6.79、-5.45、-0.70、-3.16、-3.03、-5.28，除水生根茎类（ $t = -0.70$ ）外均在 1% 水平显著，该品类定价结论相应降级（详见表 7）； R^2 介于 0.31–0.61， $\ln P$ 相关矩阵条件数为 7.9，不存在严重共线性（图 4）。

样本外回测以 MAPE 度量，求和仅对实际销量 $Q_t > 0$ 的日期进行：

$$\text{MAPE} = \frac{100\%}{n} \sum_t \frac{|\hat{Q}_t - Q_t|}{Q_t}. \quad (34)$$

后 28 天滚动回测 MAPE 为 20.6%–37.2%；2022-07 同期回测为 23.9%–66.4%，除辣椒类外各品类误差均高于后 28 天口径，最大 66.4%（茄类）超过 40% 阈值（CONDITIONAL），

表明跨年外推稳定性有限，2023-07 预测区间应保守解读（表 30，见附录；数据文件：q2_backtest.csv、q2_backtest_2022.csv）。

基线对比中，报童最优分位数订货相对确定性补货 $R_{c,t}^0 = \mu_{c,t}(P^*)/(1 - L_c)$ 的总期望利润为 5305.19 vs 4550.45 元（增益 +754.74 元），总补货量 2607.89 vs 3190.54 kg（低 18.3%），日均缺货率 0.51–0.80 vs 0.39–0.43，体现“少订少损耗换更高期望利润”的最优取舍；水生根茎类在确定性口径下利润为负（-112.79 元）、报童口径转正（360.99 元），与其临界比最小（ $\kappa = 0.20$ ）一致（表 31，见附录；数据文件：q2_nv_det.csv）[56]。

品类相关联合模拟以残差 Pearson 相关矩阵（非对角元 0.24–0.74）构造高斯 copula [57]，配合对数正态边际模拟每日 2 万组（SEED=7）：相关口径期望利润 5311.26 vs 独立口径 5305.19 元（+0.11%，在 1% 阈值内，PASS），方差比 2.47，证实“相关性只进入方差、不进期望”（数据文件：q2_joint_sim.csv）。

8.2.3 问题三：求解可靠性与可行性检验

基准及全部灵敏度情景求解状态 status=0、最优性 gap=0，五项约束逐条校验通过（选品数 $33 \in [27,33]$ 、最小陈列量 2.5 kg、可售量 $\leq$ 订购量 $\times (1 - L_i)$ 、价格在网格内、品类全覆盖）（数据文件：q3_diagnostics.csv）；3 个低需求单品因最小陈列约束超订 1.16 kg（成本 10.53 元）已计入成本。大 M 取值 $\max(3 \max_t Q_{i,t}^{\text{hist}}, 2.5, 1.05 D_{i,1})$ 的最小余量 12.58 kg，放大 50% 后最优解不变，双重确认可行域未被截断（数据文件：q3_mbound.csv）。跨问题交叉核对中，问题三各品类可售量加总 $\sum_{i \in C_c} q_i(1 - L_i)$ 与问题二品类期望需求 $\mu_{c,t}(P^*)$ 之比为 0.21–0.74，量级同阶、结构一致，差异源于计划层级与约束口径，属预期差异而非矛盾（数据文件：q3_vs_q2.csv）。

8.2.4 检验结论汇总

七项量化检验的契约见表 14：5 项 PASS、2 项 CONDITIONAL（中位价口径不可采用、2022-07 跨年回测误差更高），相应边界已在正文标注；主结论在参数扰动、口径替换、基线对比与随机模拟下保持稳健。

8.2.5 模型局限（三问共享）

缺货日销量截断使需求与弹性估计轻微低估；水生根茎类弹性不显著（ $t = -0.70$ ），其定价结论证据不足；跨年（2022-07）预测误差高于后 28 天口径，外推至 2023-07 应保守。

8.3 模型评价

优点：

表 14 模型检验契约汇总（阈值运行前声明）

检验项	扰动/方法	阈值	观测值	状态
批发价 $\pm 10\%$ 利润稳定	$W \times 1.1$ 、 $W \times 0.9$	$\leq 30\%$	0.7%	PASS
弹性 $\pm 20\%$ 决策稳定	品类弹性 $\times 1.2$ 、 $\times 0.8$	$\leq 30\%$	0.5%	PASS
价格口径切换	固定权重 $\rightarrow$ 当期加权/中位价	$\leq 30\%$	72.8%	CONDITIONAL
报童优于确定性	最优分位数 y^* vs 均值	利润差 ≥ 0	+754.74 元	PASS
损耗率替换	固定份额 $\rightarrow$ 单品销量加权	补货变化 $\leq 20\%$	1.0%（补货量）	PASS
2022-07 同期回溯	样本截止 2022-06-30	MAPE $\leq 40\%$	66.4%	CONDITIONAL
品类相关联合模拟	残差相关 vs 独立（高斯 copula）	利润偏差 $\leq 1\%$	0.11%	PASS

- **四层递进框架完整刻画需求结构。**问题一从分布形态、季节规律、关联网络、需求聚类四个层次逐层深入，四部分结论分别对应后续三个问题的核心输入。
- **定价—补货联合优化理论自治、操作可行。**问题二证明顺序决策等价于联合优化，最优加价率仅依赖品类弹性与损耗率、与批发价无关，直接回应“凌晨 3:00–4:00 不知具体进货价”的时序约束；无跨品类约束、风险中性下逐品类独立优化与整体联合优化等价。
- **MIP 联合优化有效处理单品级耦合约束。**问题三以经验贝叶斯收缩估计单品弹性、以价格网格将非线性利润目标线性化，在选品数 27–33 与最小陈列量 2.5 kg 的双重约束下实现全局最优求解。
- **数据需求分析紧密咬合前序模型。**问题四每类数据建议均指向前序模型中的特定参数或公式，并给出采集后的具体升级路径，方案系统性强。

缺点：

- **单品需求独立假设与现实存在差距。**问题三 MIP 假设 $D_i(p_i)$ 不含其他单品价格，忽略替代效应与竞品分流，已在问题四中列为数据扩展方向。
- **单周期日清模型未覆盖日内动态定价。**实际商超对品相变差商品打折出售，改变了“未售罄残值为零”的报童假设基础，需小时级库存数据支持。
- **缺货日需求截断导致补货量系统性低估。**基准需求基于已售净销量，缺货日真实需求不可观测，需缺货登记数据修正；单品层弹性识别困难亦限制差异化定价精度。

8.4 模型的改进与推广

改进：

- **耦合需求：**引入货架距离 dist_{ij} 与竞品价，将独立需求扩展为 $D_i(p_i, \mathbf{p}_{-i}) = d_i(p_i/p_i^0)^{\beta_i} \prod_{j \neq i} (p_j/p_j^0)^{\epsilon_{ij}}$ 其中 $\epsilon_{ij} = \epsilon_{ij}^0 \exp(-\lambda \text{dist}_{ij})$ ；

- **日内动态定价：**引入分时段库存 $I_{i,t,\tau}$ 与品相等级 g_τ ，以贝尔曼方程 $V_\tau(I_\tau, g_\tau) = \max_{\delta_\tau} \{\mathbb{E}[p(\delta_\tau) \min(D_\tau(p), I_\tau) + V_{\tau+1}(\cdot)]\}$ 刻画分时段折扣决策；
- **缺货修正：**采集缺货登记 $m_{i,t}$ 后按 $d_i^* = \frac{1}{T_{\text{win}}} \sum_{t \in \text{win}} [Q_{i,t} + \alpha_i m_{i,t}]$ 修正基准需求；
- **滚动优化：**每日更新数据、重新预测与求解，增强模型对需求不确定性的适应能力。

推广：该框架可迁移至水果、肉禽、水产等保鲜期短生鲜品类及面包、熟食等短保食品（仅需调整损耗率与弹性参数），亦适用于服装、电子产品等在有限销售窗口内定价并清仓的行业；品类弹性回归与报童补货模型还可嵌入“中央预测—门店决策—区域调配”的三级供应链优化框架。

本章小结

三问结论在灵敏度与模型检验下量级稳定：问题一结构结论不依赖阈值选择、聚类 $\text{ARI}=0.4567$ ；问题二批发价与弹性扰动下利润变化均小于 1%，报童增益 +754.74 元，相关模拟证实“相关性只进入方差、不进期望”（偏差 +0.11%，方差比 2.47）；问题三全部情景 optimal 且约束通过，批发价为最敏感输入（约 $\pm 18\%$ ）、网格离散化影响约 $\pm 4.5\%$ 。七项检验 5 项 PASS、2 项 CONDITIONAL，在标注的两项边界条件下，模型结论可作为商超补货定价决策的依据；评价部分进一步指出替代效应、日内定价与需求截断方面的局限，并给出数据驱动的改进路径，整体框架具备可迁移性。

参考文献

- [1] YULE G U. Why do we sometimes get nonsense correlations between time-series? A study in sampling and the nature of time-series[J/OL]. Journal of the Royal Statistical Society, 1926, 89(1):1-63. DOI: 10.1111/j.2397-2335.1926.tb01829.x.
- [2] KÖK A G, FISHER M L, VAIDYANATHAN R. Assortment planning: review of literature and industry practice[M/OL]//AGRAWAL N, SMITH S A. Retail Supply Chain Management. 2nd ed. New York: Springer, 2009: 99-153. DOI: 10.1007/978-0-387-78902-6_5.
- [3] VAN RYZIN G, MAHAJAN S. On the relationship between inventory costs and variety benefits in retail assortments[J/OL]. Management Science, 1999, 45(11):1496-1509. DOI: 10.1287/mnsc.45.11.1496.
- [4] JAMES W, STEIN C. Estimation with quadratic loss[C]//Proceedings of the Fourth Berkeley Symposium on Mathematical Statistics and Probability: volume 1. Berkeley: University of California Press, 1961: 361-379.
- [5] WOLSEY L A. Integer programming[M]. New York: Wiley, 1998.
- [6] KHOUJA M. The single-period (news-vendor) problem: literature review and suggestions for future research[J/OL]. Omega, 1999, 27(5):537-553. DOI: 10.1016/S0305-0483(99)00017-1.
- [7] AKAIKE H. A new look at the statistical model identification[J/OL]. IEEE Transactions on Automatic Control, 1974, 19(6):716-723. DOI: 10.1109/TAC.1974.1100705.
- [8] SCHWARZ G. Estimating the dimension of a model[J/OL]. The Annals of Statistics, 1978, 6(2):461-464. DOI: 10.1214/aos/1176344136.
- [9] MULLAHY J. Specification and testing of some modified count data models[J/OL]. Journal of Econometrics, 1986, 33(3):341-365. DOI: 10.1016/0304-4076(86)90002-3.
- [10] CLEVELAND R B, CLEVELAND W S, MCRAE J E, et al. STL: A seasonal-trend decomposition procedure based on loess[J]. Journal of Official Statistics, 1990, 6(1):3-73.
- [11] HYNDMAN R J, ATHANASOPOULOS G. Forecasting: Principles and practice[M/OL]. 2nd ed. Melbourne: OTexts, 2018. <https://otexts.com/fpp2/>.

- [12] WANG X, SMITH K, HYNDMAN R. Characteristic-based clustering for time series data [J/OL]. *Data Mining and Knowledge Discovery*, 2006, 13(3):335-364. DOI: 10.1007/s10618-005-0039-x.
- [13] GRANGER C W J, NEWBOLD P. Spurious regressions in econometrics[J/OL]. *Journal of Econometrics*, 1974, 2(2):111-120. DOI: 10.1016/0304-4076(74)90034-7.
- [14] WOOLDRIDGE J M. *Econometric analysis of cross section and panel data*[M]. 2nd ed. Cambridge, MA: MIT Press, 2010.
- [15] SPEARMAN C. The proof and measurement of association between two things[J/OL]. *American Journal of Psychology*, 1904, 15(1):72-101. DOI: 10.2307/1412159.
- [16] BENJAMINI Y, HOCHBERG Y. Controlling the false discovery rate: A practical and powerful approach to multiple testing[J/OL]. *Journal of the Royal Statistical Society: Series B*, 1995, 57(1):289-300. DOI: 10.1111/j.2517-6161.1995.tb02031.x.
- [17] TUMMINELLO M, ASTE T, DI MATTEO T, et al. A tool for filtering information in complex systems[J/OL]. *Proceedings of the National Academy of Sciences*, 2005, 102(30):10421-10426. DOI: 10.1073/pnas.0500298102.
- [18] MANTEGNA R N. Hierarchical structure in financial markets[J/OL]. *The European Physical Journal B*, 1999, 11(1):193-197. DOI: 10.1007/s100510050929.
- [19] FREEMAN L C. A set of measures of centrality based on betweenness[J/OL]. *Sociometry*, 1977, 40(1):35-41. DOI: 10.2307/3033543.
- [20] WASSERMAN S, FAUST K. *Social network analysis: Methods and applications*[M]. Cambridge: Cambridge University Press, 1994.
- [21] WARD J H. Hierarchical grouping to optimize an objective function[J/OL]. *Journal of the American Statistical Association*, 1963, 58(301):236-244. DOI: 10.1080/01621459.1963.10500845.
- [22] MURTAGH F, LEGENDRE P. Ward's hierarchical agglomerative clustering method: Which algorithms implement Ward's criterion?[J/OL]. *Journal of Classification*, 2014, 31(3):274-295. DOI: 10.1007/s00357-014-9161-z.
- [23] KAUFMAN L, ROUSSEEUW P J. *Finding groups in data: An introduction to cluster analysis*[M]. New York: Wiley, 1990.

- [24] ROUSSEEUW P J. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis[J/OL]. *Journal of Computational and Applied Mathematics*, 1987, 20: 53-65. DOI: 10.1016/0377-0427(87)90125-7.
- [25] HUBERT L, ARABIE P. Comparing partitions[J/OL]. *Journal of Classification*, 1985, 2 (1):193-218. DOI: 10.1007/BF01908075.
- [26] D'AGOSTINO R B, STEPHENS M A. Goodness-of-fit techniques[M]. New York: Marcel Dekker, 1986.
- [27] DEATON A, MUELLBAUER J. Economics and consumer behavior[M]. Cambridge: Cambridge University Press, 1980.
- [28] MAS-COLELL A, WHINSTON M D, GREEN J R. Microeconomic theory[M]. New York: Oxford University Press, 1995.
- [29] TELLIS G J. The price elasticity of selective demand: a meta-analysis of econometric models of sales[J/OL]. *Journal of Marketing Research*, 1988, 25(4):331-341. DOI: 10.1177/002224378802500401.
- [30] HOCH S J, KIM B D, MONTGOMERY A L, et al. Determinants of store-level price elasticity[J/OL]. *Journal of Marketing Research*, 1995, 32(1):17-29. DOI: 10.1177/002224379503200104.
- [31] NAHMIAS S. Perishable inventory theory: a review[J/OL]. *Operations Research*, 1982, 30(4):680-708. DOI: 10.1287/opre.30.4.680.
- [32] BAKKER M, RIEZEBOS J, TEUNTER R H. Review of inventory systems with deterioration since 2001[J/OL]. *European Journal of Operational Research*, 2012, 221(2):275-284. DOI: 10.1016/j.ejor.2012.03.004.
- [33] BLACKBURN J, SCUDDER G. Supply chain strategies for perishable products: the case of fresh produce[J/OL]. *Production and Operations Management*, 2009, 18(2):129-137. DOI: 10.1111/j.1937-5956.2009.01016.x.
- [34] 唐跃武, 范体军, 刘莎. 考虑策略性消费者的生鲜农产品定价和库存决策[J/OL]. *中国管理科学*, 2018, 26(11). DOI: 10.16381/j.cnki.issn1003-207x.2018.11.011.
- [35] PETRUZZI N C, DADA M. Pricing and the newsvendor problem: a review with extensions[J/OL]. *Operations Research*, 1999, 47(2):183-194. DOI: 10.1287/opre.47.2.183.

- [36] WHITIN T M. Inventory control and price theory[J/OL]. *Management Science*, 1955, 2 (1):61-68. DOI: 10.1287/mnsc.2.1.61.
- [37] TAYLOR S J, LETHAM B. Forecasting at scale[J/OL]. *The American Statistician*, 2018, 72(1):37-45. DOI: 10.1080/00031305.2017.1380080.
- [38] SILVER E A, PYKE D F, THOMAS D J. Inventory and production management in supply chains[M]. 4th ed. Boca Raton: CRC Press, 2016.
- [39] ARROW K J, HARRIS T, MARSCHAK J. Optimal inventory policy[J/OL]. *Econometrica*, 1951, 19(3):250-298. DOI: 10.2307/1906813.
- [40] AGRAWAL N, SMITH S A. Estimating negative binomial demand for retail inventory management with unobservable lost sales[J/OL]. *Naval Research Logistics*, 1996, 43(6): 839-861. DOI: 10.1002/(SICI)1520-6750(199609)43:6<839::AID-NAV5>3.0.CO;2-V.
- [41] ERLEBACHER S J. Optimal and heuristic solutions for the multi-item newsvendor problem with a single capacity constraint[J/OL]. *Production and Operations Management*, 2000, 9(3):303-318. DOI: 10.1111/j.1937-5956.2000.tb00139.x.
- [42] MARKOWITZ H. Portfolio selection[J/OL]. *The Journal of Finance*, 1952, 7(1):77-91. DOI: 10.2307/2975974.
- [43] EECKHOUDT L, GOLLIER C, SCHLESINGER H. The risk-averse (and prudent) newsboy[J/OL]. *Management Science*, 1995, 41(5):786-794. DOI: 10.1287/mnsc.41.5.786.
- [44] CHEN Y F, XU M, ZHANG Z G. Technical note—a risk-averse newsvendor model under the CVaR criterion[J/OL]. *Operations Research*, 2009, 57(4):1040-1044. DOI: 10.1287/opre.1080.0603.
- [45] ABDEL-MALEK L, MONTANARI R, MORALES L C. Exact, approximate, and generic iterative models for the multi-product newsboy problem with budget constraint[J/OL]. *International Journal of Production Economics*, 2004, 91(2):189-198. DOI: 10.1016/j.ijpe.2003.09.004.
- [46] EFRON B, MORRIS C. Stein's paradox in statistics[J/OL]. *Scientific American*, 1977, 236(5):119-127. DOI: 10.1038/scientificamerican0577-119.
- [47] MONTGOMERY A L. Creating micro-marketing pricing strategies using supermarket scanner data[J/OL]. *Marketing Science*, 1997, 16(4):315-337. DOI: 10.1287/mksc.16.4.315.

- [48] HUANGFU Q, HALL J A J. Parallelizing the dual revised simplex method [J/OL]. *Mathematical Programming Computation*, 2018, 10(1):119-142. DOI: 10.1007/s12532-017-0130-5.
- [49] KÖK A G, FISHER M L. Demand estimation and assortment optimization under substitution: application to a supermarket chain[J/OL]. *Operations Research*, 2007, 55(6): 1007-1021. DOI: 10.1287/opre.1070.0409.
- [50] FEDERGRUEN A, HECHING A. Combined pricing and inventory control under uncertainty[J/OL]. *Operations Research*, 1999, 47(3):454-475. DOI: 10.1287/opre.47.3.454.
- [51] RUSMEVICHIENTONG P, TOPALOGLU H. Robust assortment optimization under the multinomial logit choice model[J/OL]. *Operations Research*, 2012, 60(4):865-882. DOI: 10.1287/opre.1120.1045.
- [52] ELMAGHRABY W, KESKINOCAK P. Dynamic pricing in the presence of inventory considerations: research overview, current practices, and future directions[J/OL]. *Management Science*, 2003, 49(10):1287-1309. DOI: 10.1287/mnsc.49.10.1287.17315.
- [53] DHAR S K, HOCH S J, KUMAR N. Effective category management depends on the role of the category[J/OL]. *Journal of Retailing*, 2001, 77(2):165-184. DOI: 10.1016/S0022-4359(01)00045-8.
- [54] SALTELLI A, RATTO M, ANDRES T, et al. *Global sensitivity analysis: The primer*[M]. Chichester: John Wiley & Sons, 2008.
- [55] EFRON B, TIBSHIRANI R J. *An introduction to the bootstrap*[M/OL]. New York: Chapman & Hall/CRC, 1993. DOI: 10.1007/978-1-4899-4541-9.
- [56] HAN J, HU M, SHEN G. Deep neural newsvendor[J/OL]. *Management Science*, 2026. DOI: 10.1287/mnsc.2023.03157.
- [57] NELSEN R B. *An introduction to copulas*[M/OL]. 2nd ed. ed. New York: Springer, 2006. DOI: 10.1007/0-387-28678-0.

附录 A 图表说明

1.1 问题一图表

图表编号	内容说明
表 A1	各品类最优分布拟合结果（AIC/BIC 选型，6 个品类）
图 A1	各品类最优分布 QQ 图（q1_dist_cat_qq.pdf）
图 A2	代表性单品分布拟合对比（q1_dist_item_sample.pdf）
表 A2	相关系数阈值敏感性（0.3/0.4/0.5）
图 A3	品类级 Spearman 相关热力图（q1_corr_heatmap_cat.pdf）
图 A4	单品协同变动网络（q1_corr_network.pdf）
表 A3	silhouette 系数与簇数（k=3 10，最优 k=5）
图 A5	层次聚类树状图（q1_dendrogram.pdf）
图 A6	各簇标准化特征均值曲线（q1_cluster_profile.pdf）
图 A7	silhouette 曲线（q1_silhouette.pdf）

表 15 问题一图表清单

表 16 各品类最优分布拟合结果

品类	最优分布	AIC
花叶类	gamma	12444.912
花菜类	gamma	9548.503
水生根茎类	gamma	9898.119
茄类	gamma	8059.611
辣椒类	lognorm	11037.887
食用菌	lognorm	10841.520

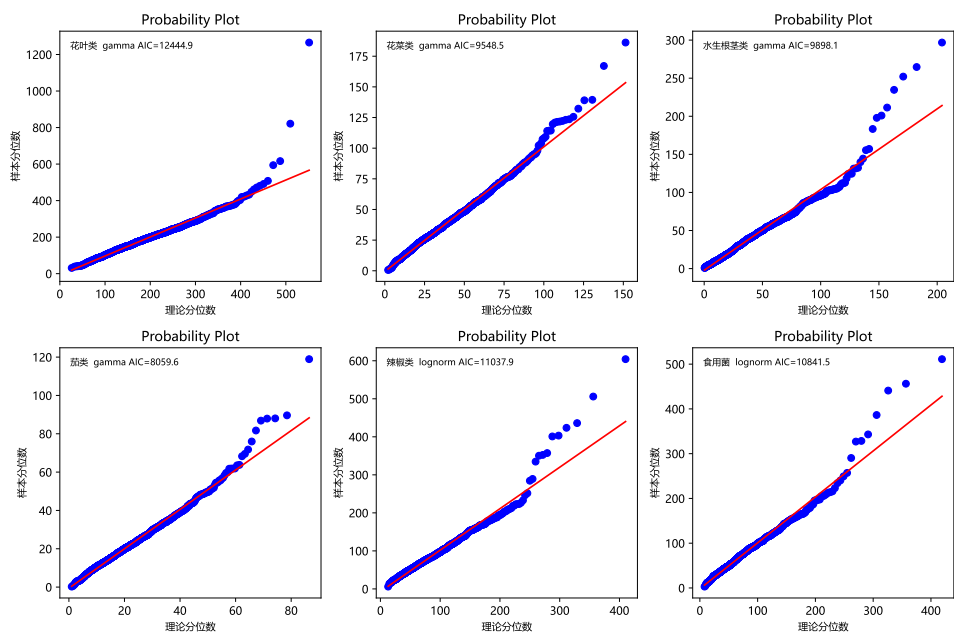


图 9 各品类最优分布 QQ 图

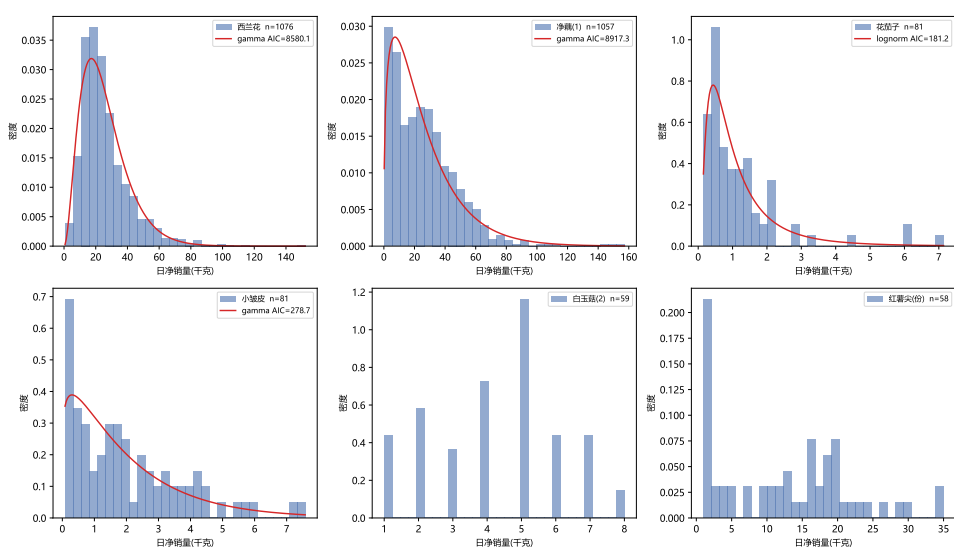


图 10 代表性单品分布拟合对比

表 17 相关系数阈值敏感性

阈值	主边数	正边数	负边数	稳健边数
0.3	1848	1289	559	249
0.4	1062	820	242	154
0.5	523	456	67	81

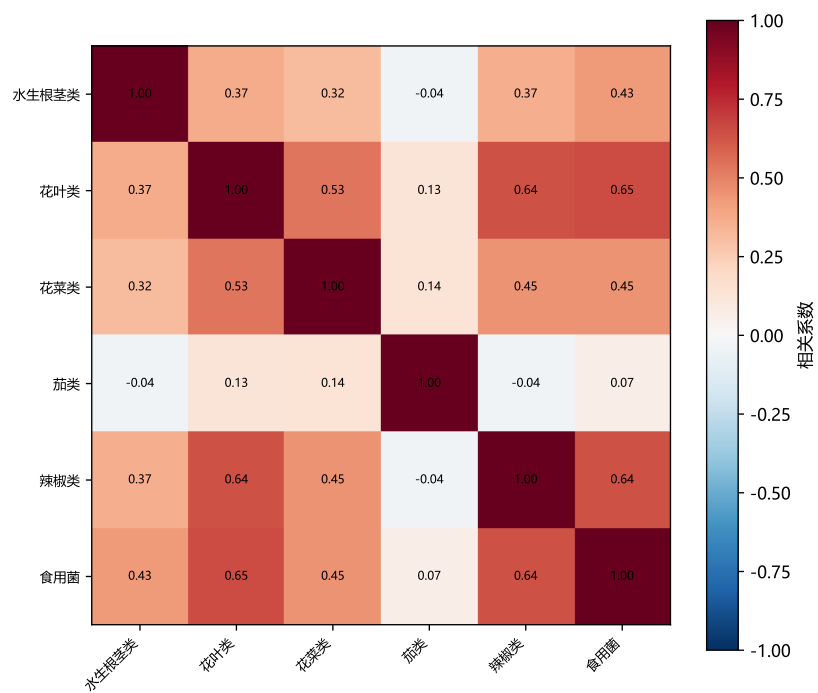


图 11 品类级 Spearman 相关热力图

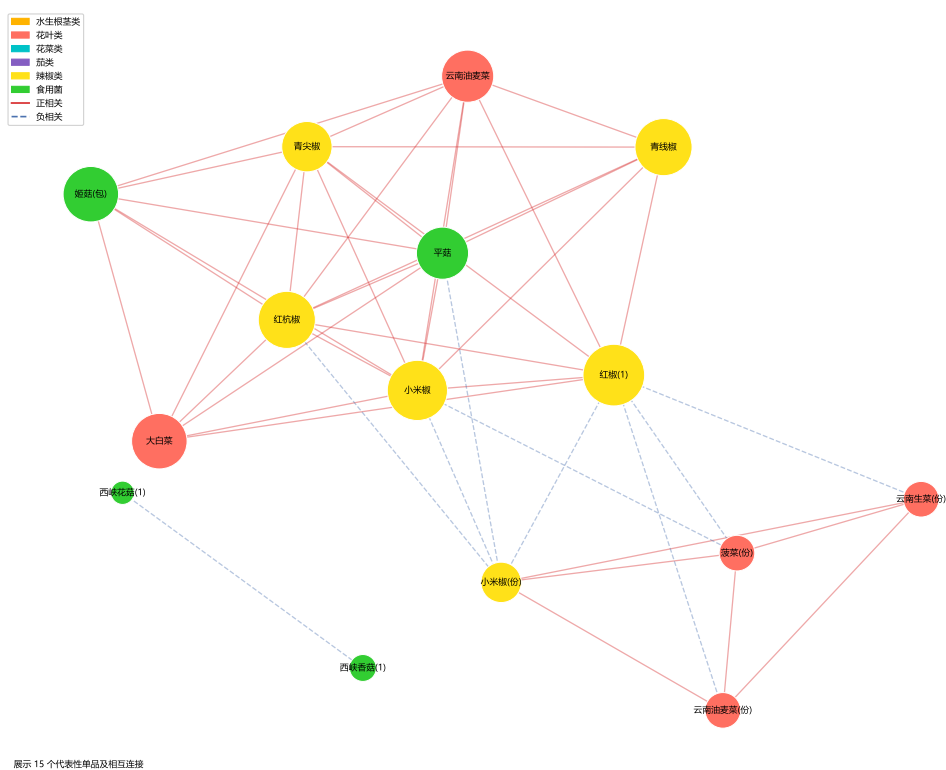


图 12 单品协同变动网络

表 18 silhouette 系数与簇数

候选 k	3	4	5	6	7	8	9	10
silhouette	0.2083	0.2429	0.2583	0.2579	0.2192	0.2257	0.2269	0.2058
稳定性 ARI	—	—	0.4567	—	—	—	—	—

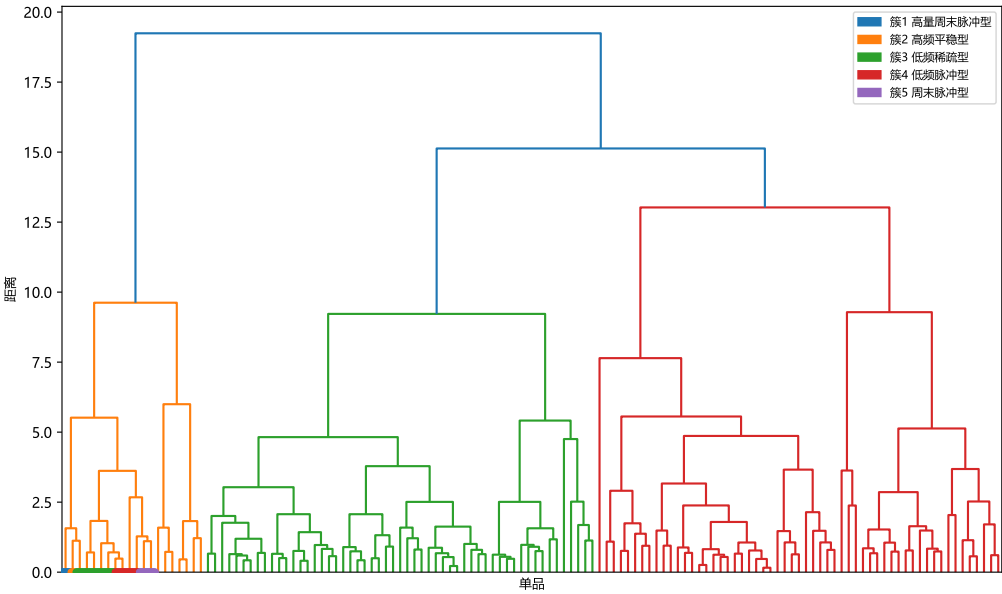


图 13 层次聚类树状图

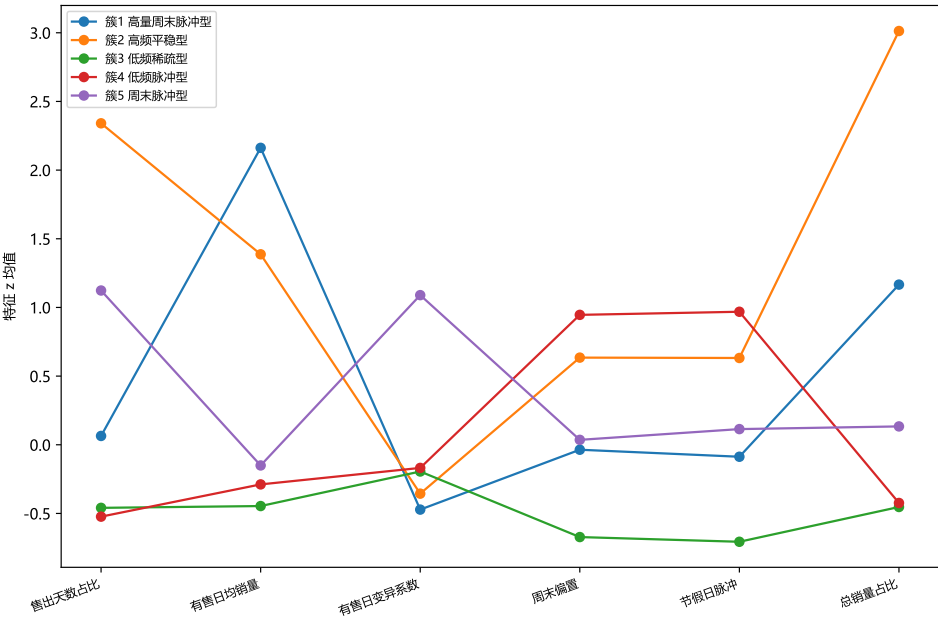


图 14 各簇标准化特征均值曲线

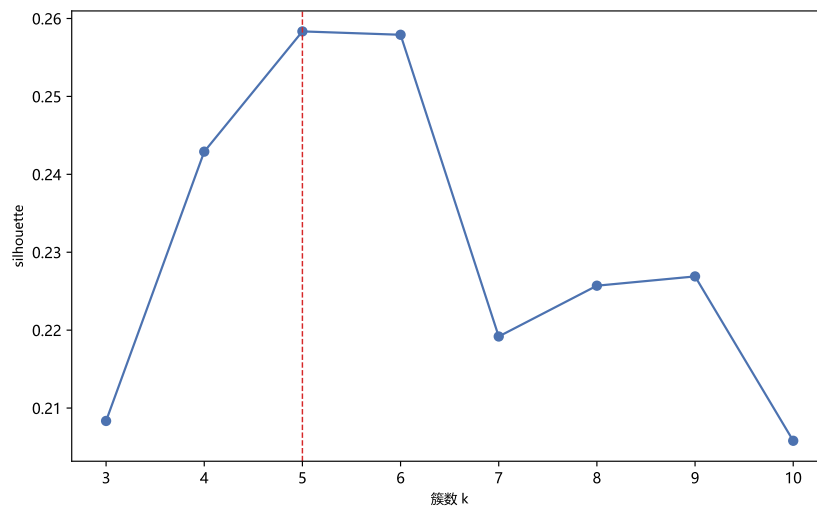


图 15 silhouette 曲线

1.2 问题二图表

图表编号	内容说明
表 A4	各品类弹性拟合结果
图 A8	各品类弹性拟合散点图 (q2_elasticity_fit.pdf)
图 A9	未来一周批发价格预测 (q2_forecast.pdf)
图 A10	价格-销量关系曲线 (q2_price_curve.pdf)
图 A11	各品类补货总量与定价策略 (q2_replenishment.pdf)
表 A5	灵敏度分析结果 (基准 2607.89kg/5305.19 元)
图 A12	灵敏度分析对比图 (q2_sensitivity.pdf)
表 A6	预测回测 MAPE (%)
表 A7	报童模型与确定性订货对比

表 19 问题二图表清单

表 20 各品类弹性拟合结果

品类	价格弹性	截距	R^2
花叶类	—	—	—
花菜类	—	—	—
水生根茎类	—	—	—
茄类	—	—	—
辣椒类	—	—	—
食用菌	—	—	—

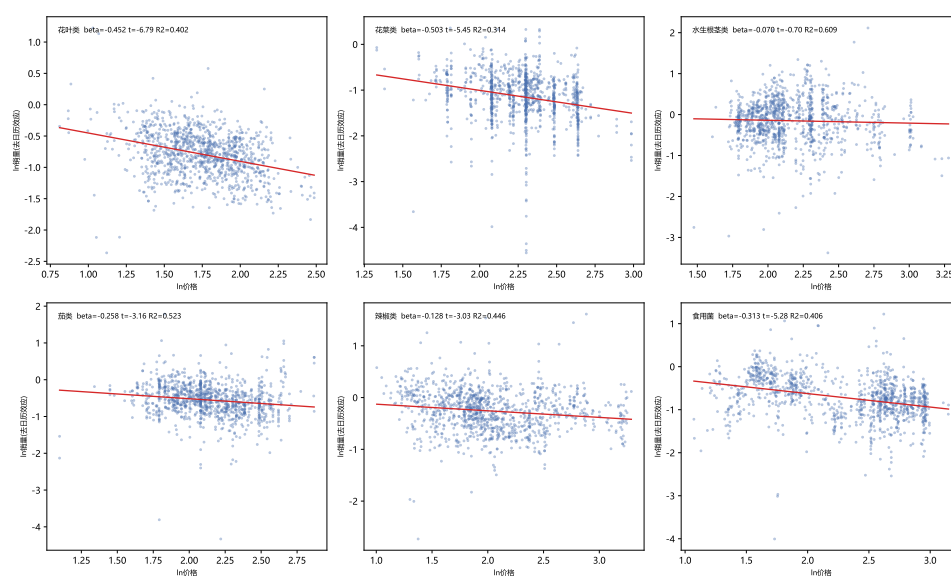


图 16 各品类弹性拟合散点图

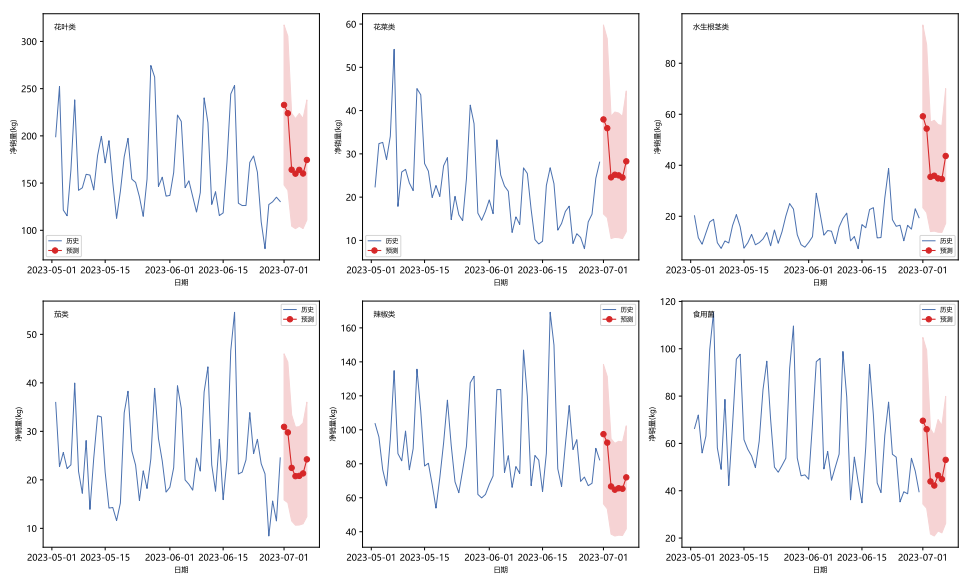


图 17 未来一周批发价格预测

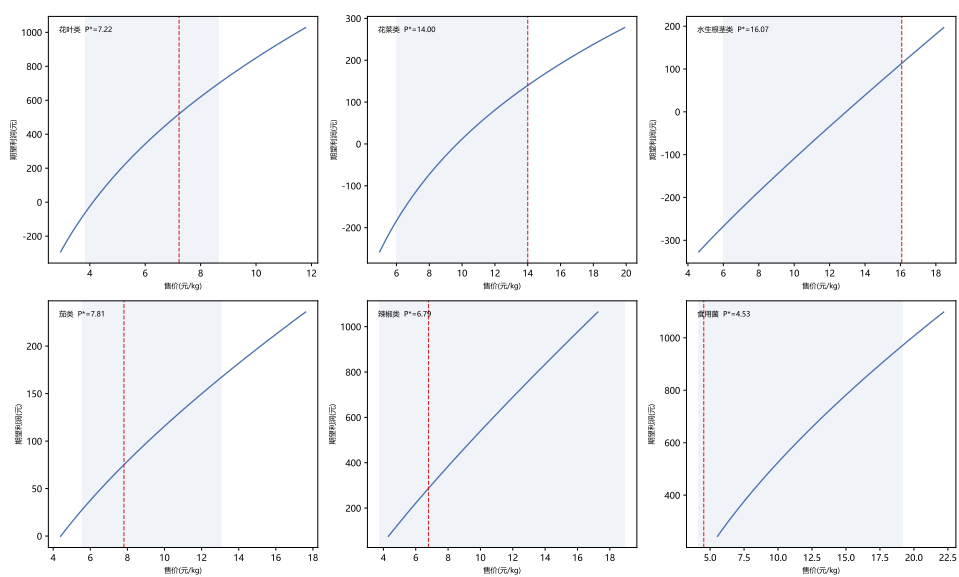


图 18 价格-销量关系曲线

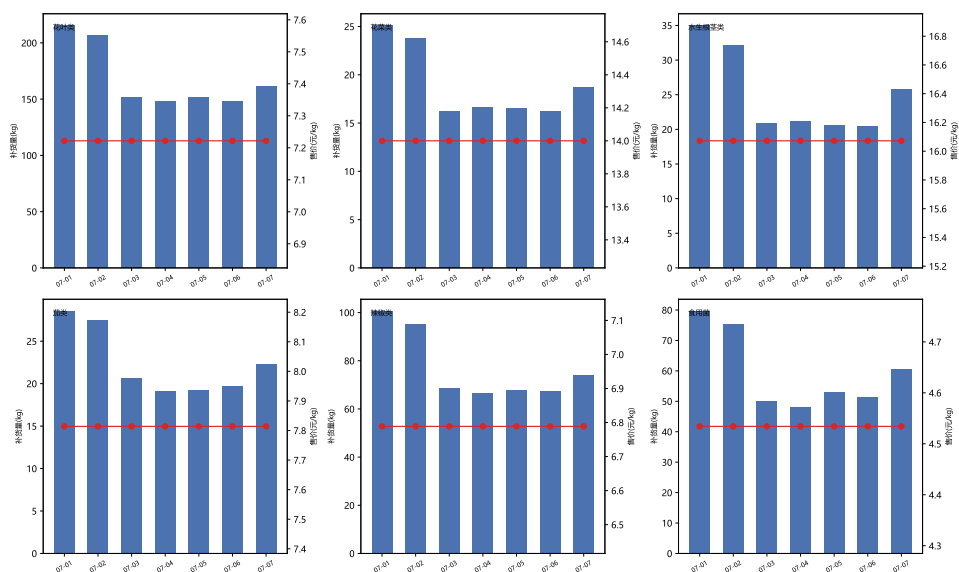


图 19 各品类补货总量与定价策略

表 21 问题二灵敏度分析

情景	7 天总补货量 (kg)	7 天总利润 (元)	利润变化
基准	2607.89	5305.19	—
批发价 +10%	2490.69	5322.36	+0.3%
批发价 -10%	2733.33	5270.62	-0.7%
弹性 +20%	2608.90	5282.80	-0.4%
弹性 -20%	2608.79	5330.49	+0.5%
损耗率 = 单品销量加权	2582.87	5705.22	+7.5%
价格口径 = 当期销售额加权	2536.22	6722.15	+26.7%
价格口径 = 中位价	2801.29	9165.93	+72.8%

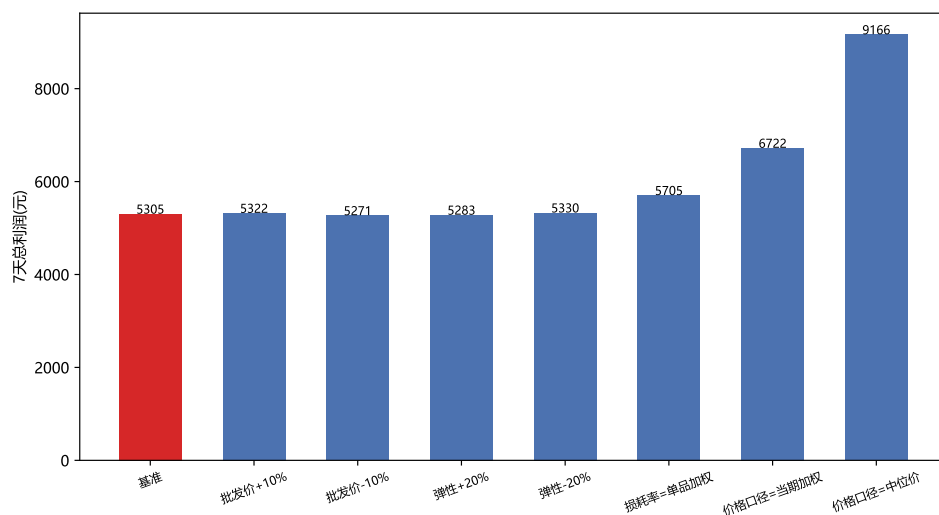


图 20 各灵敏度情景下总期望利润对比

表 22 预测回测 MAPE (%)

品类	后 28 天 MAPE	2022-07 同期 MAPE
花叶类	24.1	29.9
花菜类	37.2	46.5
水生根茎类	20.6	60.3
茄类	30.3	66.4
辣椒类	27.4	23.9
食用菌	23.5	43.1

表 23 报童与确定性订货对比（未来一周）

品类	报童补货 (kg)	确定性补货 (kg)	补货降幅	报童利润 (元)	确定性利润 (元)	日均缺货率 (报童/确定性)
花叶类	1182.61	1338.92	11.7%	2517.33	2447.20	0.57/0.43
花菜类	133.07	200.86	33.7%	355.93	230.06	0.69/0.39
水生根茎类	175.91	332.03	47.0%	360.99	-112.79	0.80/0.39
茄类	157.01	188.19	16.6%	365.76	347.50	0.56/0.41
辣椒类	540.64	595.51	9.2%	1209.04	1192.69	0.51/0.42
食用菌	418.66	535.04	21.8%	496.14	445.79	0.61/0.41
合计	2607.89	3190.54	18.3%	5305.19	4550.45	—

1.3 问题三图表

图表编号	内容说明
表 A8	灵敏度分析结果（基准 682.08 元/满足率 1.00/选中 33 个）
图 A13	收益与满足率权衡曲线（q3_fr_tradeoff.pdf）
图 A14	各单品补货量与定价策略（q3_price_qty.pdf）
图 A15	单品选择结果（q3_selection.pdf）

表 24 问题三图表清单

表 25 问题三灵敏度分析

情景	总利润（元）	满足率	选中数
$\rho = 0.5$	682.08	1.00	33
$\rho = 1.0$ （基准）	682.08	1.00	33
$\rho = 1.5$	682.08	1.00	33
选品数 =27	657.54	1.00	27
选品数 =30	673.60	1.00	30
选品数 =33	682.08	1.00	33
弹性 $\times 1.2$	670.92	1.00	33
弹性 $\times 0.8$	693.75	1.00	33
批发价 $\times 1.1$	561.98	1.00	32
批发价 $\times 0.9$	805.49	1.00	33
损耗率 = 分类级	657.44	1.00	33
网格 $K = 7$	656.78	1.00	33
网格 = 分位数取点	712.69	1.00	33
大 $M \times 1.5$	682.08	1.00	33

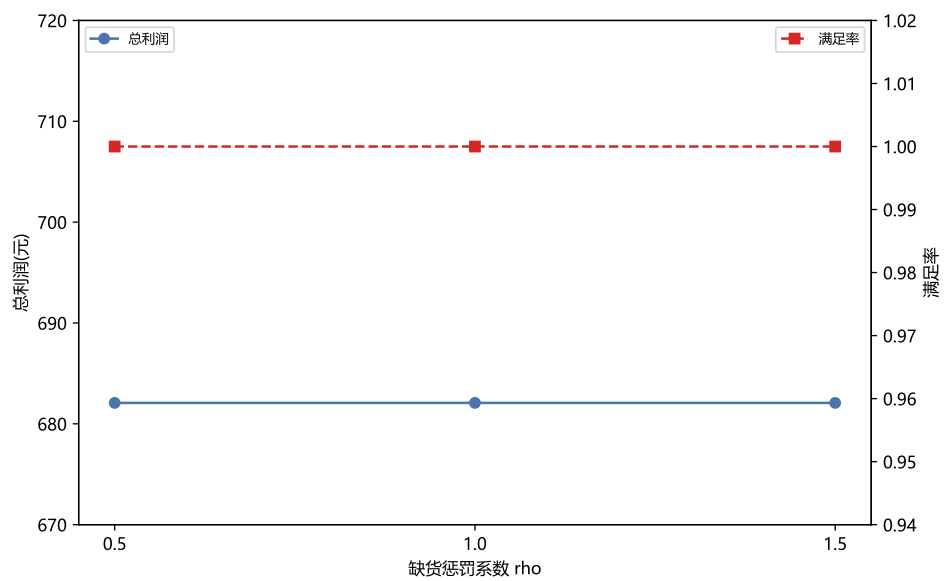


图 21 收益与满足率权衡曲线

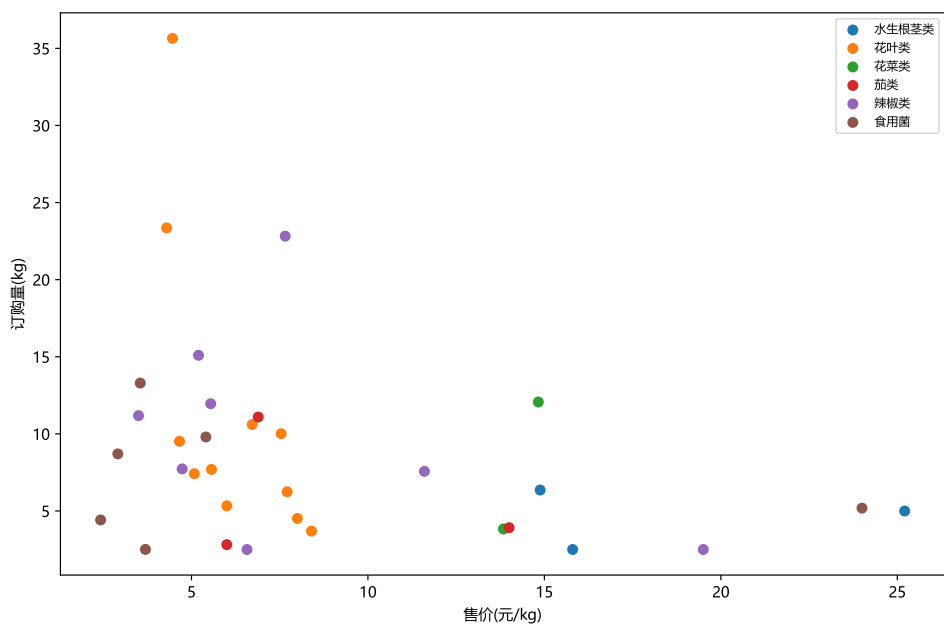


图 22 各单品补货量与定价策略

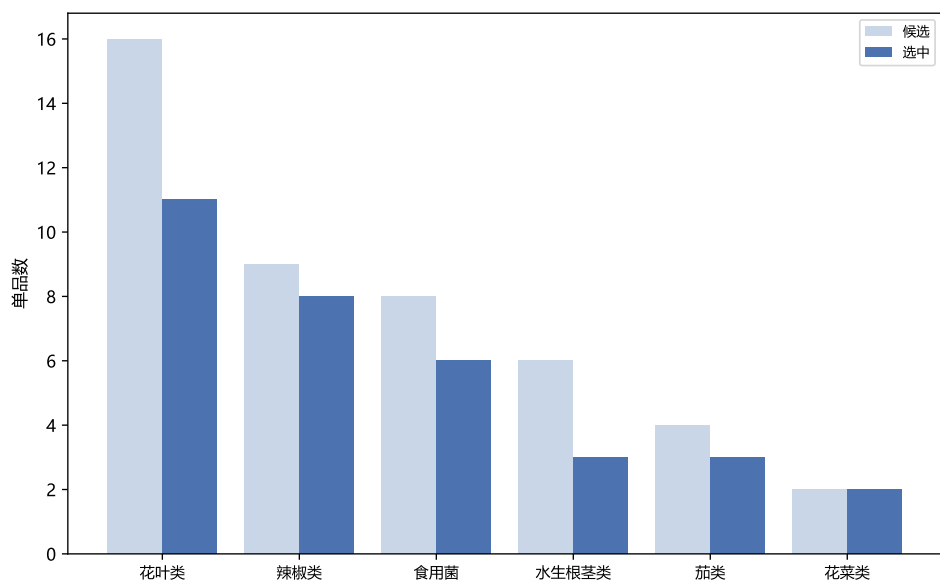


图 23 单品选择结果

1.4 模型检验与灵敏度分析补充图表

表 26 相关系数阈值敏感性（数据文件：q1_corr_threshold.csv）

阈值	主边数	正边数	负边数	稳健边数
0.3	1848	1289	559	249
0.4	1062	820	242	154
0.5	523	456	67	81

表 27 层次聚类候选 k 的 silhouette 系数（数据文件：q1_cluster_k.csv）

候选 k	3	4	5	6	7	8	9	10
silhouette	0.2083	0.2429	0.2583	0.2579	0.2192	0.2257	0.2269	0.2058
稳定性 ARI	—	—	0.4567	—	—	—	—	—

注： $k = 5$ 与 $k = 6$ 的 silhouette 接近（0.2583 与 0.2579），结合稳定性 ARI 与簇的可解释性取 $k = 5$ 。

表 28 问题二灵敏度分析结果（基准：补货 2607.89 kg、利润 5305.19 元）

情景	7 天总补货量（kg）	7 天总利润（元）	利润变化
基准	2607.89	5305.19	—
批发价 +10%	2490.69	5322.36	+0.3%
批发价 -10%	2733.33	5270.62	-0.7%
弹性 +20%	2608.90	5282.80	-0.4%
弹性 -20%	2608.79	5330.49	+0.5%
损耗率 = 单品销量加权	2582.87	5705.22	+7.5%
价格口径 = 当期销售额加权	2536.22	6722.15	+26.7%
价格口径 = 中位价	2801.29	9165.93	+72.8%

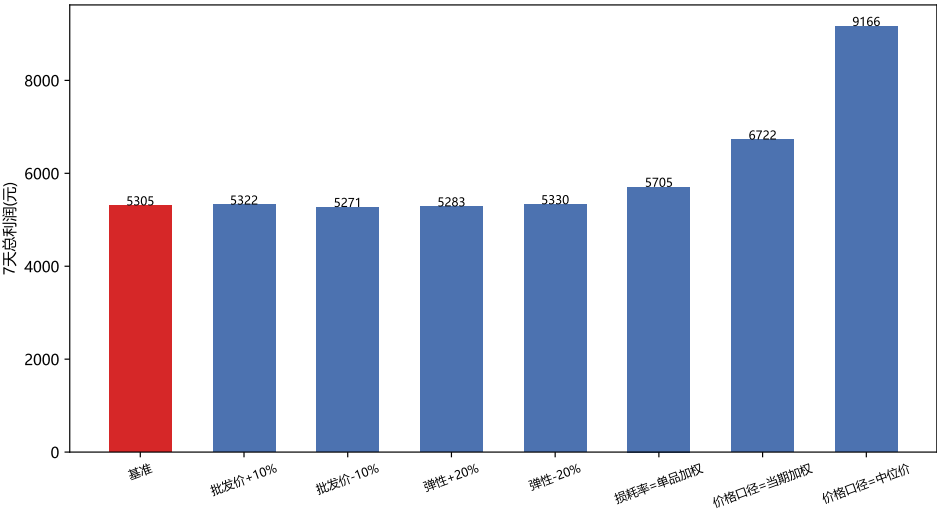


图 24 问题二各灵敏度情景下未来一周总期望利润对比（红色柱为基准情景）

表 29 问题三灵敏度分析结果（基准：利润 **682.08** 元、满足率 **1.00**、选中 **33** 个）

情景	总利润（元）	满足率	选中数
$\rho = 0.5$	682.08	1.00	33
$\rho = 1.0$ （基准）	682.08	1.00	33
$\rho = 1.5$	682.08	1.00	33
选品数 = 27	657.54	1.00	27
选品数 = 30	673.60	1.00	30
选品数 = 33	682.08	1.00	33
弹性 $\times 1.2$	670.92	1.00	33
弹性 $\times 0.8$	693.75	1.00	33
批发价 $\times 1.1$	561.98	1.00	32
批发价 $\times 0.9$	805.49	1.00	33
损耗率 = 分类级	657.44	1.00	33
网格 $K = 7$	656.78	1.00	33
网格 = 分位数取点	712.69	1.00	33
大 $M \times 1.5$	682.08	1.00	33

表 30 预测回测 **MAPE**（%）

品类	后 28 天 MAPE	2022-07 同期 MAPE
花叶类	24.1	29.9
花菜类	37.2	46.5
水生根茎类	20.6	60.3
茄类	30.3	66.4
辣椒类	27.4	23.9
食用菌	23.5	43.1

表 31 报童与确定性订货对比（未来一周，数据文件：q2_nv_det.csv）

品类	报童补货（kg）	确定性补货（kg）	补货降幅	报童利润（元）	确定性利润（元）	日均缺货率（报童/确定性）
花叶类	1182.61	1338.92	11.7%	2517.33	2447.20	0.57/0.69
花菜类	133.07	200.86	33.7%	355.93	230.06	0.69/0.80
水生根茎类	175.91	332.03	47.0%	360.99	-112.79	0.80/0.56
茄类	157.01	188.19	16.6%	365.76	347.50	0.56/0.51
辣椒类	540.64	595.51	9.2%	1209.04	1192.69	0.51/0.61
食用菌	418.66	535.04	21.8%	496.14	445.79	
合计	2607.89	3190.54	18.3%	5305.19	4550.45	

附录 B 程序代码

本附录给出全部建模代码，运行环境为 Python 3.13，依赖库包括 numpy、pandas、scipy、statsmodels、matplotlib、networkx、sklearn。

2.1 数据预处理

```

1 # clean_data.py
2 """清洗并聚合 2023 国赛 C 题原始数据。
3 输入：../Problem 下的附件 1-4，输出：results/ 下汇总表。
4 运行：python code/clean_data.py
5 """
6
7 import json
8 import sys
9 from pathlib import Path
10
11 import numpy as np
12 import pandas as pd
13
14
15 ROOT = Path(__file__).resolve().parents[1]
16 DATA = ROOT.parent / "Problem"
17 OUT = ROOT / "results"
18 OUT.mkdir(parents=True, exist_ok=True)
19
20
21 def save(df, name):
22     """保存为 utf-8-sig CSV."""
23     df.to_csv(OUT / name, index=False, encoding="utf-8-sig")

```

```

24     print(f"[save] {name}: {len(df)} 行")
25
26
27 def load_info():
28     """读取附件 1 商品信息."""
29     df = pd.read_excel(DATA / "附件1.xlsx")
30     return df[["单品编码", "单品名称", "分类编码", "分类名称"]]
31
32
33 def load_loss():
34     """读取附件 4 单品损耗率和分类平均损耗率."""
35     it = pd.read_excel(DATA / "附件4.xlsx", sheet_name="Sheet1")
36     it = it[["单品编码", "损耗率(%)"]].rename(columns={"损耗率(%)": "损耗率"})
37     ct = pd.read_excel(DATA / "附件4.xlsx",
38                         sheet_name="平均损耗率(%)_小分类编码_不同值")
39     ct = ct.rename(columns={"小分类编码": "分类编码",
40                             "平均损耗率(%)_小分类编码_不同值": "平均损耗率"})
41     return it, ct[["分类编码", "平均损耗率"]]
42
43
44 def load_sales():
45     """读取附件 2 销售流水，构造净销量和打折拆分."""
46     df = pd.read_excel(DATA / "附件2.xlsx")
47     df = df.rename(columns={"销量(千克)": "销量", "销售单价(元/千克)": "销售单价"})
48     df["销售日期"] = pd.to_datetime(df["销售日期"])
49     ok = df["销售类型"] == "销售"
50     df["净销量"] = np.where(ok, df["销量"], -df["销量"])
51     df["售出量"] = np.where(ok, df["销量"], 0.0)
52     df["销售额"] = np.where(ok, df["销量"] * df["销售单价"], 0.0)
53     df["打折销量"] = np.where(ok & (df["是否打折销售"] == "是"), df["销量"], 0.0)
54     df["正常销量"] = np.where(ok & (df["是否打折销售"] == "否"), df["销量"], 0.0)
55     return df
56
57
58 def daily_item(s):
59     """按单品-日聚合销量."""
60     agg = s.groupby(["销售日期", "单品编码"], as_index=False).agg(
61         净销量=("净销量", "sum"), 售出量=("售出量", "sum"), 销售额=("销售额", "sum"),
62         打折销量=("打折销量", "sum"), 正常销量=("正常销量", "sum"),
63         售出记录数=("售出量", "count"), 退货记录数=("净销量", lambda x: (x < 0).sum()))
64     agg["均价"] = np.where(agg["售出量"] > 0, agg["销售额"] / agg["售出量"], np.nan)
65     return agg
66
67
68 def daily_cat(s, info):
69     """按品类-日聚合销量."""
70     df = s.merge(info[["单品编码", "分类编码", "分类名称"]],

```

```

71         on="单品编码", how="left")
72     agg = df.groupby(["销售日期", "分类编码", "分类名称"], as_index=False).agg(
73         净销量=("净销量", "sum"), 售出量=("售出量", "sum"), 销售额=("销售额", "sum"),
74         打折销量=("打折销量", "sum"), 正常销量=("正常销量", "sum"),
75         售出记录数=("售出量", "count"), 退货记录数=("净销量", lambda x: (x < 0).sum()),
76         在售单品数=("单品编码", "nunique"))
77     agg["均价"] = np.where(agg["售出量"] > 0, agg["销售额"] / agg["售出量"], np.nan)
78     return agg
79
80
81 def daily_ws(w):
82     """补全单品-日批发价，先按单品前向填充再用单品均值补齐."""
83     mean = w.groupby("单品编码")["批发价"].mean()
84     codes = w["单品编码"].drop_duplicates()
85     dates = pd.date_range(w["销售日期"].min(), w["销售日期"].max(), freq="D")
86     grid = pd.DataFrame([(c, d) for c in codes for d in dates],
87                         columns=["单品编码", "销售日期"])
88     full = grid.merge(w, on=["单品编码", "销售日期"], how="left")
89     full = full.sort_values(["单品编码", "销售日期"])
90     full["批发价"] = full.groupby("单品编码")["批发价"].ffill()
91     full["价格来源"] = np.where(full["批发价"].isna(), "缺失", "前向")
92     bad = full["批发价"].isna()
93     full.loc[bad, "批发价"] = full.loc[bad, "单品编码"].map(mean)
94     full.loc[bad, "价格来源"] = "单品均值"
95     return full
96
97
98 def daily_cat_ws(ws, item, info):
99     """按品类-日聚合批发价，输出均值与销量加权."""
100     df = ws.merge(info[["单品编码", "分类编码", "分类名称"]], on="单品编码")
101     df = df.merge(item[["销售日期", "单品编码", "售出量"]],
102                 on=["销售日期", "单品编码"], how="left")
103     df["售出量"] = df["售出量"].fillna(0.0)
104     agg = df.groupby(["销售日期", "分类编码", "分类名称"], as_index=False).agg(
105         批发价均值=("批发价", "mean"), 覆盖单品数=("单品编码", "nunique"),
106         售出量=("售出量", "sum"))
107     df["加权值"] = df["批发价"] * df["售出量"]
108     w = df.groupby(["销售日期", "分类编码"])[["加权值", "售出量"]].sum()
109     w = w.reset_index()
110     w["批发价加权"] = np.where(w["售出量"] > 0, w["加权值"] / w["售出量"], np.nan)
111     agg = agg.merge(w[["销售日期", "分类编码", "批发价加权"]],
112                 on=["销售日期", "分类编码"])
113     agg["批发价加权"] = agg["批发价加权"].fillna(agg["批发价均值"])
114     return agg
115
116
117 def profile(info, s, w, item, cat, ws):

```

```

118 """输出数据质量与口径校验摘要."""
119 sold = s.loc[s["销售类型"] == "销售", "销量"].sum()
120 back = s.loc[s["销售类型"] == "退货", "销量"].sum()
121 return {
122     "商品数": int(len(info)),
123     "品类数": int(info["分类编码"].nunique()),
124     "流水行数": int(len(s)),
125     "退货行数": int((s["销售类型"] == "退货").sum()),
126     "售出总量_kg": round(float(sold), 3),
127     "退货总量_kg": round(float(back), 3),
128     "流水日期范围": [str(s["销售日期"].min().date()),
129                       str(s["销售日期"].max().date())],
130     "有销售商品数": int(s["单品编码"].nunique()),
131     "批发价行数": int(len(w)),
132     "批发价日期范围": [str(w["销售日期"].min().date()),
133                         str(w["销售日期"].max().date())],
134     "有价商品数": int(w["单品编码"].nunique()),
135     "损耗率商品数": int(info["损耗率"].notna().sum()),
136     "损耗率均值_%": round(float(info["损耗率"].mean()), 4),
137     "单品日表行数": int(len(item)),
138     "品类日表行数": int(len(cat)),
139     "批发价日表行数": int(len(ws)),
140     "均值补齐行数": int((ws["价格来源"] == "单品均值").sum()),
141     "均值补齐占比_%": round(100 * (ws["价格来源"] == "单品均值").mean(), 4),
142 }
143
144
145 def main():
146     """执行完整清洗流程."""
147     print("=" * 60)
148     print("任务 1: 数据清洗与聚合")
149     print("=" * 60)
150
151     info = load_info()
152     loss, loss_c = load_loss()
153     info = info.merge(loss, on="单品编码").merge(loss_c, on="分类编码")
154     save(info, "item_info.csv")
155
156     s = load_sales()
157     item = daily_item(s).merge(info, on="单品编码", how="left")
158     save(item, "item_daily_sales.csv")
159
160     cat = daily_cat(s, info).merge(loss_c, on="分类编码", how="left")
161     save(cat, "category_daily_sales.csv")
162
163     w = pd.read_excel(DATA / "附件3.xlsx").rename(columns={"日期": "销售日期",
164                                                            "批发价格(元/千克)": "批发价

```

```

    "})
165 w["销售日期"] = pd.to_datetime(w["销售日期"])
166 ws = daily_ws(w)
167 save(ws, "item_daily_wholesale.csv")
168
169 cat_ws = daily_cat_ws(ws, item, info)
170 save(cat_ws, "category_daily_wholesale.csv")
171
172 full = item.merge(ws[["销售日期", "单品编码", "批发价"]],
173                  on=["销售日期", "单品编码"], how="left")
174 full["加价率"] = full["均价"] / full["批发价"] - 1
175 save(full, "item_daily_full.csv")
176
177 cat_full = cat.merge(cat_ws[["销售日期", "分类编码", "批发价均值", "批发价加权"
178                               ]],
179                      on=["销售日期", "分类编码"], how="left")
180 cat_full["加价率"] = cat_full["均价"] / cat_full["批发价加权"] - 1
181 save(cat_full, "category_daily_full.csv")
182
183 pro = profile(info, s, w, item, cat, ws)
184 with (OUT / "data_profile.json").open("w", encoding="utf-8") as f:
185     json.dump(pro, f, ensure_ascii=False, indent=2)
186 print("\n[校验]")
187 print(json.dumps(pro, ensure_ascii=False, indent=2))
188 print("\n任务 1 完成：结果已写入 results/")
189
190 if __name__ == "__main__":
191     sys.exit(main())

```

2.2 问题一代码

```

1 # question1/A.py
2 # 任务A：品类/单品销量分布拟合
3 # 输入 results/ 日销量表，输出 results/ 拟合表与 figures/ 图
4
5 from datetime import datetime
6 from pathlib import Path
7 import time
8
9 import matplotlib.pyplot as plt
10 import numpy as np
11 import pandas as pd
12 from scipy import stats
13
14

```



```

15 ROOT = Path(__file__).resolve().parents[2]
16 OUT = ROOT / "results"
17 FIG = ROOT / "figures"
18 LOG_DIR = Path(__file__).resolve().parent / "outputs"
19 OUT.mkdir(parents=True, exist_ok=True)
20 FIG.mkdir(parents=True, exist_ok=True)
21 LOG_DIR.mkdir(parents=True, exist_ok=True)
22 t = 1095
23 n_day_min = 60
24 dists = {"lognorm": stats.lognorm, "gamma": stats.gamma}
25 k = {"lognorm": 2, "gamma": 2}
26 DEBUG = True
27
28 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei"]
29 plt.rcParams["axes.unicode_minus"] = False
30
31
32 def fit(x, name):
33     d = dists[name]
34     par = d.fit(x, floc=0)
35     llf = np.sum(d.logpdf(x, *par))
36     aic = 2 * k[name] - 2 * llf
37     bic = k[name] * np.log(len(x)) - 2 * llf
38     return par, llf, aic, bic
39
40
41 def par_str(name, par):
42     if name == "lognorm":
43         return f"s={par[0]:.4f}, loc={par[1]:.4f}, scale={par[2]:.4f}"
44     return f"a={par[0]:.4f}, loc={par[1]:.4f}, scale={par[2]:.4f}"
45
46
47 def best_fit(x):
48     best = None
49     for name in dists:
50         par, llf, aic, bic = fit(x, name)
51         if best is None or aic < best[2]:
52             best = (name, par, aic)
53     return best
54
55
56 def dist_cat():
57     df = pd.read_csv(OUT / "category_daily_sales.csv", encoding="utf-8-sig")
58     rows = []
59     for c, g in df.groupby("分类编码"):
60         x = g["净销量"].values
61         x = x[x > 0]

```

```

62     if DEBUG:
63         print("[debug] cat", c, "positive days:", len(x))
64     res = []
65     for name in dists:
66         par, llf, aic, bic = fit(x, name)
67         res.append((name, par, llf, aic, bic))
68     best = min(res, key=lambda r: r[3])
69     for name, par, llf, aic, bic in res:
70         rows.append({"分类编码": c, "分类名称": g["分类名称"].iloc[0],
71                     "分布": name, "参数": par_str(name, par),
72                     "负对数似然": round(-llf, 3), "AIC": round(aic, 3),
73                     "BIC": round(bic, 3),
74                     "是否最优": "是" if name == best[0] else "否"})
75 out = pd.DataFrame(rows)
76 out.to_csv(OUT / "q1_dist_cat.csv", index=False, encoding="utf-8-sig")
77
78
79 def dist_item():
80     df = pd.read_csv(OUT / "item_daily_sales.csv", encoding="utf-8-sig")
81     df = df[df["净销量"] > 0]
82     n = df.groupby("单品编码").agg(n=("净销量", "size"),
83                                     name=("单品名称", "first"),
84                                     cat=("分类名称", "first"))
85     if DEBUG:
86         print("[debug] items:", len(n))
87         print("[debug] sale-day min/med/max:", n["n"].min(), n["n"].median(),
88               n["n"].max())
89     rows, qs = [], []
90     for code, r in n.iterrows():
91         x = df.loc[df["单品编码"] == code, "净销量"].values
92         nd = int(r["n"])
93         q = np.percentile(x, [50, 75, 90, 95, 99])
94         qs.append({"单品编码": code, "单品名称": r["name"], "有效销售天数": nd,
95                  "P50": round(q[0], 3), "P75": round(q[1], 3),
96                  "P90": round(q[2], 3), "P95": round(q[3], 3),
97                  "P99": round(q[4], 3)})
98         if nd < n_day_min:
99             continue
100     name, par, aic = best_fit(x)
101     rows.append({"单品编码": code, "单品名称": r["name"], "分类名称": r["cat"],
102                 "有效销售天数": nd, "售出概率": round(nd / t, 4),
103                 "分布": name, "参数": par_str(name, par),
104                 "AIC": round(aic, 3), "是否最优": "是"})
105 pd.DataFrame(qs).to_csv(OUT / "q1_item_quantiles.csv", index=False,
106                        encoding="utf-8-sig")
107 pd.DataFrame(rows).to_csv(OUT / "q1_dist_item.csv", index=False,
108                          encoding="utf-8-sig")

```

```

109     if DEBUG:
110         print("[debug] fitted:", len(rows), "sparse:", len(qs) - len(rows))
111     return df
112
113
114 def sample_codes(df):
115     n = df.groupby("单品编码")["净销量"].size()
116     hi = n.nlargest(2).index.tolist()
117     mid = n.iloc[(n - n.median()).abs().argsort()[:2]].index.tolist()
118     sp = n[n < n_day_min].nlargest(2).index.tolist()
119     return hi + mid + sp
120
121
122 def qq_cat():
123     df = pd.read_csv(OUT / "category_daily_sales.csv", encoding="utf-8-sig")
124     fig, axs = plt.subplots(2, 3, figsize=(12, 8))
125     for ax, (c, g) in zip(axs.ravel(), df.groupby("分类编码")):
126         x = g["净销量"].values
127         x = x[x > 0]
128         name, par, aic = best_fit(x)
129         stats.probplot(x, dist=dists[name], sparams=par, plot=ax)
130         ax.set_xlabel("理论分位数", fontsize=9)
131         ax.set_ylabel("样本分位数", fontsize=9)
132         ax.text(0.04, 0.96, f"{g['分类名称'].iloc[0]} {name} AIC={aic:.1f}",
133               transform=ax.transAxes, va="top", fontsize=8)
134     fig.tight_layout()
135     fig.savefig(FIG / "q1_dist_cat_qq.pdf")
136     plt.close(fig)
137
138
139 def fig_item(df):
140     codes = sample_codes(df)
141     fig, axs = plt.subplots(2, 3, figsize=(14, 8))
142     for ax, code in zip(axs.ravel(), codes):
143         g = df[df["单品编码"] == code]
144         x = g["净销量"].values
145         nd = len(x)
146         ax.hist(x, bins=30, density=True, alpha=0.6, color="#4C72B0",
147               label=f"{g['单品名称'].iloc[0]} n={nd}")
148         if nd >= n_day_min:
149             name, par, aic = best_fit(x)
150             xs = np.linspace(x.min(), x.max(), 200)
151             ax.plot(xs, dists[name].pdf(xs, *par), color="C3", lw=1.5,
152                   label=f"{name} AIC={aic:.1f}")
153         ax.set_xlabel("日净销量(千克)", fontsize=9)
154         ax.set_ylabel("密度", fontsize=9)
155         ax.legend(fontsize=8)

```

```

156 fig.tight_layout()
157 fig.savefig(FIG / "q1_dist_item_sample.pdf")
158 plt.close(fig)
159
160
161 def main():
162     t0 = time.time()
163     dist_cat()
164     df = dist_item()
165     qq_cat()
166     fig_item(df)
167     c = pd.read_csv(OUT / "q1_dist_cat.csv", encoding="utf-8-sig")
168     it = pd.read_csv(OUT / "q1_dist_item.csv", encoding="utf-8-sig")
169     qt = pd.read_csv(OUT / "q1_item_quantiles.csv", encoding="utf-8-sig")
170     if DEBUG:
171         print("[debug] cat rows:", len(c))
172         print("[debug] winners:", c.loc[c["是否最优"] == "是", "分布"].tolist())
173         print("[debug] item fitted:", len(it), "sparse:",
174               len(qt) - len(it), "no-sale:", 251 - len(qt))
175         print("[debug] nd range:", it["有效销售天数"].min(),
176               it["有效销售天数"].max())
177         print("[debug] nan total:", int(c.isna().sum().sum()
178                                         + it.isna().sum().sum()
179                                         + qt.isna().sum().sum()))
180         print("[debug] elapsed: %.1fs" % (time.time() - t0))
181     lines = [
182         f"run_time={datetime.now().isoformat(timespec='seconds')}",
183         f"cat_winner={c.loc[c['是否最优']=='是', '分布'].tolist()}",
184         f"item_fitted={len(it)} "
185         f"gamma={len(it[it['分布']=='gamma'])} "
186         f"lognorm={len(it[it['分布']=='lognorm'])}",
187         f"item_quantiles={len(qt)}",
188     ]
189     (LOG_DIR / "A.log").write_text("\n".join(lines) + "\n", encoding="utf-8")
190     print("[save] q1_dist_cat.csv / q1_dist_item.csv / q1_item_quantiles.csv")
191     print("[save] q1_dist_cat_qq.pdf / q1_dist_item_sample.pdf")
192     print("[save] outputs/A.log")
193
194
195 if __name__ == "__main__":
196     main()
197
198 # question1/B.py
199 # 任务B: 品类日销量 STL 季节分解
200 # 输入 results/category_daily_sales.csv, 输出分解表与两张图
201
202 from datetime import datetime

```

```

203 from pathlib import Path
204 import time
205
206 import matplotlib.dates as mdates
207 import matplotlib.pyplot as plt
208 import numpy as np
209 import pandas as pd
210 from statsmodels.tsa.seasonal import STL
211
212
213 ROOT = Path(__file__).resolve().parents[2]
214 OUT = ROOT / "results"
215 FIG = ROOT / "figures"
216 LOG_DIR = Path(__file__).resolve().parent / "outputs"
217 OUT.mkdir(parents=True, exist_ok=True)
218 FIG.mkdir(parents=True, exist_ok=True)
219 LOG_DIR.mkdir(parents=True, exist_ok=True)
220 PERIOD = 7
221 DATE0 = "2020-07-01"
222 DATE1 = "2023-06-30"
223 PEAK = [4, 5, 6, 7, 8, 9, 10]
224 DEBUG = True
225
226 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei"]
227 plt.rcParams["axes.unicode_minus"] = False
228
229
230 def load():
231     df = pd.read_csv(OUT / "category_daily_sales.csv", encoding="utf-8-sig")
232     df["销售日期"] = pd.to_datetime(df["销售日期"])
233     return df
234
235
236 def full_daily(df):
237     idx = pd.date_range(DATE0, DATE1, freq="D")
238     out = []
239     for c, g in df.groupby("分类编码"):
240         s = g.set_index("销售日期")["净销量"].reindex(idx, fill_value=0.0)
241         d = pd.DataFrame({"销售日期": idx, "净销量": s.values})
242         d["分类编码"] = c
243         d["分类名称"] = g["分类名称"].iloc[0]
244         out.append(d)
245     return pd.concat(out, ignore_index=True)
246
247
248 def decomp(s):
249     res = STL(s, period=PERIOD, robust=True).fit()

```

```

250     return res.trend, res.seasonal, res.resid
251
252
253 def strength(t, s, r):
254     fs = max(0.0, 1 - np.var(r) / np.var(s + r))
255     ft = max(0.0, 1 - np.var(r) / np.var(t + r))
256     return fs, ft
257
258
259 def monthly(g):
260     m = g.assign(月份=g["销售日期"].dt.month)
261     return m.groupby("月份")["净销量"].mean()
262
263
264 def fig_stl(full, rows, dec):
265     fig, axs = plt.subplots(2, 3, figsize=(15, 8), sharex=True)
266     for ax, (c, g) in zip(axs.ravel(), full.groupby("分类编码")):
267         d = g["销售日期"]
268         t, se, r = dec[c]
269         ax.plot(d, t, color="C0", lw=1.2, label="趋势")
270         ax.plot(d, se, color="C1", lw=1.0, label="季节")
271         ax.plot(d, r, color="C2", lw=0.7, alpha=0.7, label="残差")
272         info = rows.loc[rows["分类编码"] == c].iloc[0]
273         ax.text(0.03, 0.96,
274                f"{info['分类名称']} F_s={info['季节强度']:.2f} "
275                f"F_t={info['趋势强度']:.2f}",
276                transform=ax.transAxes, va="top", fontsize=8)
277         # 日期刻度按半年间隔，避免月份挤在一起
278         ax.xaxis.set_major_locator(mdates.MonthLocator(interval=6))
279         ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))
280         ax.tick_params(axis="x", labelsize=7, rotation=20)
281         ax.set_xlabel("日期", fontsize=8)
282         ax.set_ylabel("销量(千克)", fontsize=8)
283     handles, labels = axs.ravel()[0].get_legend_handles_labels()
284     fig.legend(handles, labels, loc="lower center", ncol=3, fontsize=9)
285     fig.tight_layout(rect=[0, 0.04, 1, 1])
286     fig.savefig(FIG / "q1_stl.pdf")
287     plt.close(fig)
288
289
290 def fig_month(full):
291     fig, axs = plt.subplots(2, 3, figsize=(14, 8))
292     for ax, (c, g) in zip(axs.ravel(), full.groupby("分类编码")):
293         mm = monthly(g)
294         colors = ["#D62728" if i in PEAK else "#4C72B0" for i in mm.index]
295         ax.bar(mm.index, mm.values, color=colors, alpha=0.85)
296         ax.set_xticks(range(1, 13))

```

```

297     ax.set_xlabel("月份", fontsize=9)
298     ax.set_ylabel("平均日销量(千克)", fontsize=9)
299     ax.text(0.03, 0.96, g["分类名称"].iloc[0],
300           transform=ax.transAxes, va="top", fontsize=8)
301     fig.tight_layout()
302     fig.savefig(FIG / "q1_monthly_mean.pdf")
303     plt.close(fig)
304
305
306 def main():
307     t0 = time.time()
308     df = load()
309     full = full_daily(df)
310     missing = len(full) - len(df)
311     if DEBUG:
312         print("[debug] total missing days:", missing)
313     dec = {}
314     stl_rows, str_rows, m_rows = [], [], []
315     for c, g in full.groupby("分类编码"):
316         s = g["净销量"].reset_index(drop=True)
317         t, se, r = decomp(s)
318         dec[c] = (t, se, r)
319         fs, ft = strength(t, se, r)
320         miss_c = 1095 - len(df[df["分类编码"] == c])
321         d = g[["销售日期", "分类编码", "分类名称"]].reset_index(drop=True)
322         d["净销量"] = s.values
323         d["趋势"] = t.values
324         d["季节"] = se.values
325         d["残差"] = r.values
326         stl_rows.append(d)
327         mm = monthly(g)
328         peak = mm[mm.index.isin(PEAK)].mean()
329         off = mm[~mm.index.isin(PEAK)].mean()
330         ratio = peak / off
331         if DEBUG:
332             print("[debug]", g["分类名称"].iloc[0],
333                   "missing:", miss_c,
334                   "F_s=%.4f F_t=%.4f ratio=%.4f" % (fs, ft, ratio))
335         str_rows.append({"分类编码": c, "分类名称": g["分类名称"].iloc[0],
336                         "季节强度": round(fs, 4), "趋势强度": round(ft, 4),
337                         "旺季平均日销量": round(peak, 3),
338                         "淡季平均日销量": round(off, 3),
339                         "旺季比值": round(ratio, 4)})
340     for mo, v in mm.items():
341         m_rows.append({"分类编码": c, "分类名称": g["分类名称"].iloc[0],
342                       "月份": int(mo), "平均日销量": round(v, 3)})
343     stl = pd.concat(stl_rows, ignore_index=True)

```

```

344 stl[["净销量", "趋势", "季节", "残差"]] = \
345     stl[["净销量", "趋势", "季节", "残差"]].round(3)
346 strength_df = pd.DataFrame(str_rows)
347 month_df = pd.DataFrame(m_rows)
348 stl.to_csv(OUT / "q1_stl_fit.csv", index=False, encoding="utf-8-sig")
349 strength_df.to_csv(OUT / "q1_seasonal_strength.csv", index=False,
350                   encoding="utf-8-sig")
351 month_df.to_csv(OUT / "q1_monthly_mean.csv", index=False,
352               encoding="utf-8-sig")
353 fig_stl(full, strength_df, dec)
354 fig_month(full)
355 if DEBUG:
356     print("[debug] stl rows:", len(stl),
357           "nan:", int(stl.isna().sum().sum()))
358     print("[debug] F_s range:", strength_df["季节强度"].min(),
359           strength_df["季节强度"].max())
360     print("[debug] F_t range:", strength_df["趋势强度"].min(),
361           strength_df["趋势强度"].max())
362     print("[debug] monthly rows:", len(month_df))
363     print("[debug] elapsed: %.1fs" % (time.time() - t0))
364 lines = [
365     f"run_time={datetime.now().isoformat(timespec='seconds')}",
366     f"missing_days={missing}",
367     f"seasonal_strength={strength_df['季节强度'].tolist()}",
368     f"trend_strength={strength_df['趋势强度'].tolist()}",
369     f"peak_ratio={strength_df['旺季比值'].tolist()}",
370     f"monthly_rows={len(month_df)}",
371 ]
372 (LOG_DIR / "B.log").write_text("\n".join(lines) + "\n", encoding="utf-8")
373 print("[save] q1_stl_fit.csv / q1_seasonal_strength.csv / "
374       "q1_monthly_mean.csv")
375 print("[save] q1_stl.pdf / q1_monthly_mean.pdf")
376 print("[save] outputs/B.log")
377
378
379 if __name__ == "__main__":
380     main()
381
382 # question1/C.py
383 # 任务C: 去季节相关性与协同变动网络
384 # 输入 results/item_daily_sales.csv, 输出相关表/网络表与两张图
385
386 import time
387 from datetime import datetime
388 from pathlib import Path
389
390 import matplotlib.pyplot as plt

```



```

391 import networkx as nx
392 import numpy as np
393 import pandas as pd
394 from matplotlib.lines import Line2D
395 from matplotlib.patches import Patch
396 from scipy import stats
397
398
399 ROOT = Path(__file__).resolve().parents[2]
400 OUT = ROOT / "results"
401 FIG = ROOT / "figures"
402 LOG_DIR = Path(__file__).resolve().parent / "outputs"
403 OUT.mkdir(parents=True, exist_ok=True)
404 FIG.mkdir(parents=True, exist_ok=True)
405 LOG_DIR.mkdir(parents=True, exist_ok=True)
406 DATE0 = "2020-07-01"
407 DATE1 = "2023-06-30"
408 n_day_min = 60
409 r_th = 0.4
410 fdr_q = 0.05
411 ths = [0.3, 0.4, 0.5]
412 DEBUG = True
413
414 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei"]
415 plt.rcParams["axes.unicode_minus"] = False
416
417
418 def load():
419     df = pd.read_csv(OUT / "item_daily_sales.csv", encoding="utf-8-sig")
420     df["销售日期"] = pd.to_datetime(df["销售日期"])
421     return df
422
423
424 def wide(df):
425     idx = pd.date_range(DATE0, DATE1, freq="D")
426     q = df.pivot(index="销售日期", columns="单品编码",
427                 values="净销量").reindex(idx).fillna(0.0)
428     info = df.drop_duplicates("单品编码").set_index("单品编码")[
429         ["单品名称", "分类编码", "分类名称"]]
430     return q, info
431
432
433 def design(d):
434     wd = d["销售日期"].dt.weekday
435     mo = d["销售日期"].dt.month
436     x = pd.get_dummies(wd, prefix="w", drop_first=True).astype(float)
437     x = pd.concat([x, pd.get_dummies(mo, prefix="m",

```

```

438                                     drop_first=True).astype(float)], axis=1)
439 x["c"] = 1.0
440 return x.values
441
442
443 def resid(q, x):
444     r = np.empty_like(q, dtype=float)
445     for i in range(q.shape[1]):
446         b, _, _, _ = np.linalg.lstsq(x, q[:, i], rcond=None)
447         r[:, i] = q[:, i] - x @ b
448     return r
449
450
451 def sp(r):
452     rank = np.apply_along_axis(stats.rankdata, 0, r)
453     with np.errstate(invalid="ignore", divide="ignore"):
454         rho = np.corrcoef(rank, rowvar=False)
455     n = r.shape[0]
456     with np.errstate(divide="ignore", invalid="ignore"):
457         t = rho * np.sqrt((n - 2) / (1 - rho ** 2))
458     p = 2 * stats.t.sf(np.abs(t), n - 2)
459     np.fill_diagonal(p, 1.0)
460     return rho, p
461
462
463 def bh(p):
464     m = len(p)
465     order = np.argsort(p)
466     q = np.empty(m)
467     q[order] = p[order] * m / np.arange(1, m + 1)
468     q = np.minimum.accumulate(q[::-1])[::-1]
469     return q
470
471
472 def cat_heat():
473     df = pd.read_csv(OUT / "category_daily_sales.csv", encoding="utf-8-sig")
474     df["销售日期"] = pd.to_datetime(df["销售日期"])
475     q = df.pivot(index="销售日期", columns="分类名称",
476                  values="净销量").reindex(pd.date_range(DATE0, DATE1),
477                                           ).fillna(0.0)
478     d = pd.DataFrame({"销售日期": q.index})
479     rho, _ = sp(resid(q.values, design(d)))
480     fig, ax = plt.subplots(figsize=(7, 6))
481     im = ax.imshow(rho, cmap="RdBu_r", vmin=-1, vmax=1)
482     ax.set_xticks(range(6))
483     ax.set_yticks(range(6))
484     ax.set_xticklabels(q.columns, fontsize=8, rotation=45, ha="right")

```

```

485 ax.set_yticklabels(q.columns, fontsize=8)
486 for i in range(6):
487     for j in range(6):
488         ax.text(j, i, f"{rho[i, j]:.2f}", ha="center", va="center", fontsize=7)
489 fig.colorbar(im, ax=ax, label="相关系数")
490 fig.tight_layout()
491 fig.savefig(FIG / "q1_corr_heatmap_cat.pdf")
492 plt.close(fig)
493 return rho
494
495
496 def fig_net(sub, info):
497     g = nx.Graph()
498     for _, r in sub.iterrows():
499         g.add_edge(int(r["单品A编码"]), int(r["单品B编码"]),
500                   sign=np.sign(r["相关系数"]))
501     deg = dict(g.degree())
502     negd = {}
503     for u, v, d in g.edges(data=True):
504         if d["sign"] < 0:
505             negd[u] = negd.get(u, 0) + 1
506             negd[v] = negd.get(v, 0) + 1
507     # 按度中心度取 15 个代表单品, 不足时用负相关节点替换, 保证正负边都可见
508     topn = sorted(deg, key=deg.get, reverse=True)[:15]
509     for n in sorted(negd, key=negd.get, reverse=True):
510         if n in topn:
511             continue
512         cand = [x for x in topn if negd.get(x, 0) == 0]
513         if not cand:
514             break
515         topn.remove(cand[-1])
516         topn.append(n)
517     g = g.subgraph(topn)
518     if DEBUG:
519         print("[debug] network nodes shown:", len(g))
520         print("[debug] negative edges shown:",
521               sum(1 for _, _, d in g.edges(data=True) if d["sign"] < 0))
522     fig, ax = plt.subplots(figsize=(13, 10))
523     # kamada-kawai 布局比力导向更舒展, 降低边交叉
524     pos = nx.kamada_kawai_layout(g)
525     pe = [(u, v) for u, v, d in g.edges(data=True) if d["sign"] > 0]
526     ne = [(u, v) for u, v, d in g.edges(data=True) if d["sign"] < 0]
527     nx.draw_networkx_edges(g, pos, edgelist=pe, edge_color="#D62728",
528                           alpha=0.4, width=1.2, ax=ax)
529     nx.draw_networkx_edges(g, pos, edgelist=ne, edge_color="#4C72B0",
530                           alpha=0.4, width=1.2, style="dashed", ax=ax)
531     cats = sorted(info["分类名称"].unique())

```

```

532 bright = ["#FFB300", "#FF6F61", "#00C2C7", "#845EC2", "#FFE119", "#32CD32"]
533 ccolor = {c: bright[i % len(bright)] for i, c in enumerate(cats)}
534 # 按品类用亮色着色, 图例给出品类与正负边含义
535 for c in cats:
536     nodes = [n for n in g.nodes() if info.loc[n, "分类名称"] == c]
537     sizes = [300 + 150 * deg[n] for n in nodes]
538     nx.draw_networkx_nodes(g, pos, nodelist=nodes, node_size=sizes,
539                             node_color=[ccolor[c]] * len(nodes),
540                             edgecolors="white", linewidths=0.5, ax=ax)
541 labels = {n: info.loc[n, "单品名称"] for n in g.nodes() if n in info.index}
542 nx.draw_networkx_labels(g, pos, labels, font_size=8, ax=ax)
543 handles = [Patch(color=ccolor[c], label=c) for c in cats]
544 handles += [Line2D([0], [0], color="#D62728", lw=1.5, label="正相关"),
545             Line2D([0], [0], color="#4C72B0", lw=1.5, ls="--",
546                   label="负相关")]
547 ax.legend(handles=handles, loc="upper left", fontsize=8, framealpha=0.9)
548 ax.text(0.01, 0.01, "展示 15 个代表性单品及相互连接",
549         transform=ax.transAxes, fontsize=8, va="bottom")
550 ax.axis("off")
551 fig.tight_layout()
552 fig.savefig(FIG / "q1_corr_network.pdf")
553 plt.close(fig)
554
555
556 def main():
557     t0 = time.time()
558     df = load()
559     q, info = wide(df)
560     n_day = (q > 0).sum()
561     cols = n_day[n_day >= n_day_min].index
562     q = q[cols]
563     info = info.loc[cols]
564     d = pd.DataFrame({"销售日期": q.index})
565     x = design(d)
566     rho, p = sp(resid(q.values, x))
567     iu = np.triu_indices(rho.shape[0], 1)
568     rf, pf = rho[iu], p[iu]
569     qf = bh(pf)
570     h1 = q.index <= "2021-12-31"
571     h2 = ~h1
572     rho1, p1 = sp(resid(q[h1].values, design(d.loc[h1])))
573     rho2, p2 = sp(resid(q[h2].values, design(d.loc[h2])))
574     r1f, r2f = rho1[iu], rho2[iu]
575     p1f, p2f = p1[iu], p2[iu]
576     stable = ((np.sign(rf) == np.sign(r1f)) & (np.sign(rf) == np.sign(r2f))
577              & (p1f < 0.05) & (p2f < 0.05))
578     main = (np.abs(rf) >= r_th) & (qf < fdr_q)

```

```

579 edges = pd.DataFrame({
580     "单品A编码": q.columns[iu[0][main]],
581     "单品A名称": info.loc[q.columns[iu[0][main]], "单品名称"].values,
582     "单品B编码": q.columns[iu[1][main]],
583     "单品B名称": info.loc[q.columns[iu[1][main]], "单品名称"].values,
584     "相关系数": np.round(rf[main], 4),
585     "p值": np.round(pf[main], 6),
586     "FDR_q值": np.round(qf[main], 6),
587     "符号": np.where(rf[main] > 0, "正", "负"),
588     "是否稳健边": np.where(stable[main], "是", "否"),
589 })
590 edges.to_csv(OUT / "q1_corr_edges.csv", index=False, encoding="utf-8-sig")
591 thr_rows = []
592 for th in ths:
593     m1 = np.abs(rf) >= th
594     thr_rows.append({"阈值": th, "边数": int(m1.sum()),
595                     "正边数": int((m1 & (rf > 0)).sum()),
596                     "负边数": int((m1 & (rf < 0)).sum()),
597                     "稳健边数": int((m1 & stable).sum())})
598 thr = pd.DataFrame(thr_rows)
599 thr.to_csv(OUT / "q1_corr_threshold.csv", index=False,
600           encoding="utf-8-sig")
601 sub = edges[edges["是否稳健边"] == "是"]
602 if len(sub):
603     g = nx.Graph()
604     for _, r in sub.iterrows():
605         g.add_edge(int(r["单品A编码"]), int(r["单品B编码"]),
606                   sign=np.sign(r["相关系数"]))
607     deg = nx.degree_centrality(g)
608     bc = nx.betweenness_centrality(g)
609     pos_cnt = {}
610     neg_cnt = {}
611     for _, r in sub.iterrows():
612         for node in (int(r["单品A编码"]), int(r["单品B编码"])):
613             if r["符号"] == "正":
614                 pos_cnt[node] = pos_cnt.get(node, 0) + 1
615             else:
616                 neg_cnt[node] = neg_cnt.get(node, 0) + 1
617     top = pd.DataFrame({
618         "单品编码": list(deg.keys()),
619         "单品名称": [info.loc[n, "单品名称"] for n in deg],
620         "度中心度": np.round(list(deg.values()), 4),
621         "介数中心度": np.round([bc[n] for n in deg], 6),
622         "正边数": [pos_cnt.get(n, 0) for n in deg],
623         "负边数": [neg_cnt.get(n, 0) for n in deg],
624     }).sort_values("度中心度", ascending=False)
625     top.to_csv(OUT / "q1_network_top.csv", index=False, encoding="utf-8-sig")

```

```

626     fig_net(sub, info)
627 else:
628     pd.DataFrame(columns=["单品编码", "单品名称", "度中心度", "介数中心度",
629                          "正边数", "负边数"]).to_csv(
630         OUT / "q1_network_top.csv", index=False, encoding="utf-8-sig")
631 rho_cat = cat_heat()
632 if DEBUG:
633     print("[debug] items kept:", len(cols))
634     print("[debug] pairs tested:", len(rf))
635     print("[debug] main edges:", len(edges),
636           "stable:", int(stable.sum()))
637     print("[debug] threshold table:\n", thr.to_string(index=False))
638     print("[debug] cat corr:\n",
639           np.round(rho_cat, 3))
640     print("[debug] nan total:", int(edges.isna().sum().sum()
641                                     + thr.isna().sum().sum()))
642     print("[debug] elapsed: %.1fs" % (time.time() - t0))
643 lines = [
644     f"run_time={datetime.now().isoformat(timespec='seconds')}",
645     f"items={len(cols)} pairs={len(rf)}",
646     f"main_edges={len(edges)} stable={int(stable.sum())}",
647     f"edges_by_threshold={thr['边数'].tolist()}",
648     f"stable_by_threshold={thr['稳健边数'].tolist()}",
649 ]
650 (LOG_DIR / "C.log").write_text("\n".join(lines) + "\n", encoding="utf-8")
651 print("[save] q1_corr_edges.csv / q1_network_top.csv / "
652       "q1_corr_threshold.csv")
653 print("[save] q1_corr_heatmap_cat.pdf / q1_corr_network.pdf")
654 print("[save] outputs/C.log")
655
656
657 if __name__ == "__main__":
658     main()
659
660 # question1/D.py
661 # 任务D: 单品时序特征层次聚类
662 # 输入 results/item_daily_sales.csv, 输出特征/簇表与三张图
663
664 import time
665 from datetime import datetime
666 from pathlib import Path
667
668 import chinese_calendar as cc
669 import matplotlib.pyplot as plt
670 import numpy as np
671 import pandas as pd
672 from scipy.cluster import hierarchy

```

```

673 from scipy.stats import zscore
674 from sklearn.cluster import AgglomerativeClustering
675 from sklearn.metrics import adjusted_rand_score, silhouette_score
676 from matplotlib.patches import Patch
677
678
679 ROOT = Path(__file__).resolve().parents[2]
680 OUT = ROOT / "results"
681 FIG = ROOT / "figures"
682 LOG_DIR = Path(__file__).resolve().parent / "outputs"
683 OUT.mkdir(parents=True, exist_ok=True)
684 FIG.mkdir(parents=True, exist_ok=True)
685 LOG_DIR.mkdir(parents=True, exist_ok=True)
686 DATE0 = "2020-07-01"
687 DATE1 = "2023-06-30"
688 n_day_min = 60
689 k_lo, k_hi = 3, 10
690 sil_min = 0.15
691 n_boot = 50
692 boot_frac = 0.8
693 DEBUG = True
694
695 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei"]
696 plt.rcParams["axes.unicode_minus"] = False
697
698 FEATS = ["售出天数占比", "有售日均销量", "有售日变异系数", "周末偏置",
699         "节假日脉冲", "总销量占比"]
700
701
702 def load():
703     df = pd.read_csv(OUT / "item_daily_sales.csv", encoding="utf-8-sig")
704     df["销售日期"] = pd.to_datetime(df["销售日期"])
705     return df
706
707
708 def feats(df):
709     idx = pd.date_range(DATE0, DATE1, freq="D")
710     q = df.pivot(index="销售日期", columns="单品编码",
711                  values="净销量").reindex(idx).fillna(0.0)
712     info = df.drop_duplicates("单品编码").set_index("单品编码")[
713         ["单品名称", "分类编码", "分类名称"]]
714     n_day = (q > 0).sum()
715     cols = n_day[n_day >= n_day_min].index
716     q = q[cols]
717     n = q.shape[1]
718     mu = np.zeros(n)
719     cv = np.zeros(n)

```

```

720 for i, c in enumerate(cols):
721     x = q[c].values
722     xp = x[x > 0]
723     mu[i] = xp.mean()
724     cv[i] = xp.std() / mu[i] if mu[i] > 0 else 0.0
725 wd = (q.index.weekday >= 5)
726 hd = np.array([cc.is_holiday(d.date()) for d in q.index])
727 wk = (q[wd].mean() / q[~wd].mean() - 1).values
728 hl = (q[hd].mean() / q[~hd].mean() - 1).values
729 sh = q.sum().values / q.sum().sum()
730 f = pd.DataFrame({
731     "售出天数占比": (n_day[cols] / len(idx)).values,
732     "有售日均销量": mu,
733     "有售日变异系数": cv,
734     "周末偏置": wk,
735     "节假日脉冲": hl,
736     "总销量占比": sh,
737 }, index=cols)
738 return f, info.loc[cols]
739
740
741 def pick_k(z):
742     lnk = hierarchy.linkage(z, method="ward")
743     rows = [(k, silhouette_score(z, hierarchy.fcluster(
744         lnk, t=k, criterion="maxclust")) for k in range(k_lo, k_hi + 1)]
745     best = max(rows, key=lambda r: r[1])[0]
746     return lnk, rows, best
747
748
749 def stability(z, best, seed=7):
750     rng = np.random.default_rng(seed)
751     full = hierarchy.fcluster(hierarchy.linkage(z, method="ward"),
752                             t=best, criterion="maxclust")
753     ari = []
754     for _ in range(n_boot):
755         idx = rng.choice(len(z), size=int(len(z) * boot_frac), replace=False)
756         sub = AgglomerativeClustering(n_clusters=best,
757                                     linkage="ward").fit_predict(z[idx])
758         ari.append(adjusted_rand_score(full[idx], sub))
759     return np.mean(ari)
760
761
762 def name_cluster(fm):
763     p, cv, wk, mu = (fm["售出天数占比"], fm["有售日变异系数"],
764                     fm["周末偏置"], fm["有售日均销量"])
765     if p >= 0.6 and cv <= 0.8:
766         return "高频平稳型"

```



```

767     if p < 0.3:
768         if wk >= 0.35:
769             return "低频脉冲型"
770             return "低频稀疏型"
771     if cv >= 1.2:
772         return "高波动型"
773     if wk >= 0.15:
774         if mu >= 15:
775             return "高量周末脉冲型"
776             return "周末脉冲型"
777     return "中频均衡型"
778
779
780 def fig_dend(lnk, lab, names):
781     fig, ax = plt.subplots(figsize=(10, 6))
782     d = hierarchy.dendrogram(lnk, ax=ax, no_labels=True)
783     # 叶子按簇着色, 图例给出簇标签与命名
784     leaves = np.array(d["leaves"])
785     ks = np.unique(lab)
786     cmap = plt.get_cmap("tab10")
787     colors = {k: cmap(i % 10) for i, k in enumerate(ks)}
788     ax.scatter(np.arange(len(leaves)), np.zeros(len(leaves)), s=18,
789               c=[colors[lab[i]] for i in leaves], zorder=3)
790     handles = [Patch(color=colors[k], label=f"簇{k} {names[k]}") for k in ks]
791     ax.legend(handles=handles, fontsize=8, loc="upper right")
792     ax.set_xlabel("单品", fontsize=9)
793     ax.set_ylabel("距离", fontsize=9)
794     fig.tight_layout()
795     fig.savefig(FIG / "q1_dendrogram.pdf")
796     plt.close(fig)
797
798
799 def fig_sil(rows, best):
800     ks = [r[0] for r in rows]
801     ss = [r[1] for r in rows]
802     fig, ax = plt.subplots(figsize=(8, 5))
803     ax.plot(ks, ss, "o-", color="#4C72B0")
804     ax.axvline(best, color="#D62728", ls="--", lw=1)
805     ax.set_xlabel("簇数 k", fontsize=9)
806     ax.set_ylabel("silhouette", fontsize=9)
807     fig.tight_layout()
808     fig.savefig(FIG / "q1_silhouette.pdf")
809     plt.close(fig)
810
811
812 def fig_profile(z, lab, names):
813     fig, ax = plt.subplots(figsize=(9, 6))

```

```

814 x = np.arange(len(FEATS))
815 for k in np.unique(lab):
816     m = z[lab == k].mean(axis=0)
817     ax.plot(x, m, "o-", label=f"簇{k} {names[k]}")
818 ax.set_xticks(x)
819 ax.set_xticklabels(FEATS, fontsize=8, rotation=20, ha="right")
820 ax.set_ylabel("特征 z 均值", fontsize=9)
821 ax.legend(fontsize=8)
822 fig.tight_layout()
823 fig.savefig(FIG / "q1_cluster_profile.pdf")
824 plt.close(fig)
825
826
827 def main():
828     t0 = time.time()
829     df = load()
830     f, info = feats(df)
831     z = zscore(f.values)
832     lnk, rows, best = pick_k(z)
833     lab = hierarchy.fcluster(lnk, t=best, criterion="maxclust")
834     ari = stability(z, best)
835     out = f.reset_index().rename(columns={"index": "单品编码"})
836     out["单品名称"] = info["单品名称"].values
837     out["分类编码"] = info["分类编码"].values
838     out["分类名称"] = info["分类名称"].values
839     out["簇标签"] = lab
840     out = out[["单品编码", "单品名称", "分类编码", "分类名称"] + FEATS
841             + ["簇标签"]]
842     out.to_csv(OUT / "q1_item_cluster.csv", index=False, encoding="utf-8-sig")
843     prof = []
844     for k in np.unique(lab):
845         sub = out[out["簇标签"] == k]
846         fm = sub[FEATS].mean()
847         top = sub.sort_values("总销量占比", ascending=False).head(3)[
848             "单品名称"].tolist()
849         prof.append({"簇标签": int(k), "单品数": len(sub),
850                     **{c: round(v, 4) for c, v in fm.items()},
851                     "典型单品名称": "/".join(top),
852                     "建议命名": name_cluster(fm)})
853     prof_df = pd.DataFrame(prof)
854     prof_df.to_csv(OUT / "q1_cluster_profile.csv", index=False,
855                   encoding="utf-8-sig")
856     k_df = pd.DataFrame(rows, columns=["候选k", "silhouette值"])
857     k_df["稳定性ARI"] = np.nan
858     k_df.loc[k_df["候选k"] == best, "稳定性ARI"] = round(ari, 4)
859     k_df.to_csv(OUT / "q1_cluster_k.csv", index=False, encoding="utf-8-sig")
860     names = dict(zip(prof_df["簇标签"], prof_df["建议命名"]))

```

```

861 fig_dend(lnk, lab, names)
862 fig_sil(rows, best)
863 fig_profile(z, lab, names)
864 if DEBUG:
865     print("[debug] items:", len(f))
866     print("[debug] chosen k:", best, "silhouette:",
867           round([s for k, s in rows if k == best][0], 4), "ARI:", round(ari, 4))
868     print("[debug] cluster sizes:", np.bincount(lab)[1:].tolist())
869     print("[debug] nan total:", int(out.isna().sum().sum()
870                                     + prof_df.isna().sum().sum()))
871     print("[debug] elapsed: %.1fs" % (time.time() - t0))
872 lines = [
873     f"run_time={datetime.now().isoformat(timespec='seconds')}",
874     f"items={len(f)} k={best}",
875     f"silhouette={round([s for k, s in rows if k == best][0], 4)}",
876     f"ari={round(ari, 4)}",
877     f"cluster_sizes={np.bincount(lab)[1:].tolist()}",
878 ]
879 (LOG_DIR / "D.log").write_text("\n".join(lines) + "\n", encoding="utf-8")
880 print("[save] q1_item_cluster.csv / q1_cluster_profile.csv / "
881       "q1_cluster_k.csv")
882 print("[save] q1_dendrogram.pdf / q1_silhouette.pdf / "
883       "q1_cluster_profile.pdf")
884 print("[save] outputs/D.log")
885
886
887 if __name__ == "__main__":
888     main()

```

2.3 问题二代码

```

1 # question2/q2.py
2 # 问题二主流程：数据 -> 弹性 -> 定价 -> 预测 -> 报童
3
4 import ctypes
5 from pathlib import Path
6
7 import matplotlib.pyplot as plt
8 import numpy as np
9 import pandas as pd
10 import statsmodels.api as sm
11 from scipy.optimize import differential_evolution
12
13
14 ROOT = Path(__file__).resolve().parents[2]
15 OUT = ROOT / "results"

```

```

16 FIG = ROOT / "figures"
17 FIG.mkdir(parents=True, exist_ok=True)
18 DATE0 = "2020-07-01"
19 DATE1 = "2023-06-30"
20 DEBUG = True
21
22 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei"]
23 plt.rcParams["axes.unicode_minus"] = False
24
25 _lib = ctypes.CDLL(str(Path(__file__).resolve().parent / "nv.dll"))
26 _f64 = np.ctypeslib.ndpointer(dtype=np.float64, ndim=1, flags="C_CONTIGUOUS")
27 _lib.newsvendor_batch.restype = None
28 _lib.newsvendor_batch.argtypes = [_f64] * 5 + [ctypes.c_int] + [_f64] * 5
29
30
31 def nv(mu, sg, p, c, l):
32     # 批量报童补货, 返回 y/kappa/R/R0/期望利润
33     mu = np.ascontiguousarray(mu, dtype=np.float64)
34     sg = np.ascontiguousarray(sg, dtype=np.float64)
35     p = np.ascontiguousarray(p, dtype=np.float64)
36     c = np.ascontiguousarray(c, dtype=np.float64)
37     l = np.ascontiguousarray(l, dtype=np.float64)
38     n = len(mu)
39     y = np.empty(n)
40     kap = np.empty(n)
41     r = np.empty(n)
42     r0 = np.empty(n)
43     ep = np.empty(n)
44     _lib.newsvendor_batch(mu, sg, p, c, l, n, y, kap, r, r0, ep)
45     return y, kap, r, r0, ep
46
47
48 def d2i(s):
49     return (pd.Timestamp(s) - pd.Timestamp(DATE0)).days
50
51
52 def load():
53     df = pd.read_csv(OUT / "item_daily_full.csv", encoding="utf-8-sig")
54     df["日期"] = pd.to_datetime(df["销售日期"])
55     df["d"] = (df["日期"] - pd.Timestamp(DATE0)).dt.days
56     names = df.drop_duplicates("分类编码").set_index("分类编码")["分类名称"].to_dict()
57     return df, names
58
59
60 def agg(df):
61     win = df[(df["d"] >= d2i("2022-07-01")) & (df["d"] <= d2i("2023-06-30"))]

```

```

62 iq = win.groupby("单品编码")["净销量"].sum()
63 cq = win.groupby("分类编码")["净销量"].sum()
64 cat = df.drop_duplicates("单品编码").set_index("单品编码")["分类编码"]
65 w = (iq / cat.map(cq)).fillna(0)
66
67 def one(g):
68     sw = g["w"].sum()
69     return pd.Series({
70         "Q": g["净销量"].sum(),
71         "P": (g["均价"] * g["w"]).sum() / sw if sw > 0 else np.nan,
72         "W": (g["批发价"] * g["w"]).sum() / sw if sw > 0 else np.nan,
73         "L": g["平均损耗率"].mean() / 100,
74     })
75
76 tmp = df[df["均价"] > 0].copy()
77 tmp["w"] = tmp["单品编码"].map(w)
78 tab = tmp.groupby(["d", "分类编码"]).apply(one).unstack()
79 idx = pd.date_range(DATE0, DATE1)
80 d = (idx - pd.Timestamp(DATE0)).days
81 q = df.groupby(["d", "分类编码"])["净销量"].sum().unstack().reindex(d).fillna(0)
82 p = tab["P"].reindex(d).fillna(tab["P"].mean())
83 w = tab["W"].reindex(d).fillna(tab["W"].mean())
84 l = tab["L"].mean()
85 return list(q.columns), idx, q, p, w, l
86
87
88 def stat(q, p, w, l, names):
89     # 每品类均值/分位数/成本与最近一周批发价
90     rows = []
91     for c in q.columns:
92         qv = q[c].values
93         pos = qv > 0
94         qq, pp = qv[pos], p[c].values[pos]
95         wv = w[c].values[pos]
96         m = qq.mean()
97         wb = w[c].iloc[d2i("2023-06-24")].mean()
98         wb = wb if not np.isnan(wb) else w[c].mean()
99         rows.append({
100             "c": c, "name": names[c], "mu": m, "sd": qq.std(ddof=0),
101             "pbar": pp.mean(), "p5": np.percentile(pp, 5),
102             "p95": np.percentile(pp, 95), "m5": np.percentile(pp / wv - 1, 5),
103             "m95": np.percentile(pp / wv - 1, 95), "W": wb, "L": l[c],
104         })
105     return pd.DataFrame(rows).set_index("c")
106
107
108 def ols(q, p, cats, idx):

```

```

109 # 品类 Log-Log 弹性回归
110 wd = idx.weekday.to_numpy()
111 mo = idx.month.to_numpy()
112 B = np.zeros((len(cats), len(cats)))
113 rows = []
114 sc = []
115 for i, c in enumerate(cats):
116     qv = q[c].values
117     pos = qv > 0
118     d = np.where(pos)[0]
119     X = np.column_stack([
120         np.log(p.values[d]), d,
121         (wd[d][:, None] == np.arange(1, 7)).astype(float),
122         (mo[d][:, None] == np.arange(2, 13)).astype(float),
123         np.ones(len(d)),
124     ])
125     y = np.log(qv[pos])
126     fit = sm.OLS(y, X).fit()
127     res = y - X @ fit.params
128     B[i] = fit.params[:len(cats)]
129     rows.append({"c": c, "beta": fit.params, "b0": fit.params[i],
130                 "t": fit.tvalues[i], "r2": fit.rsquared,
131                 "mse": fit.mse_resid, "res": res})
132     sc.append(pd.DataFrame({"ln价格": np.log(p[c].values[pos]), "ln销量": y}))
133 sc = pd.concat(sc, keys=cats)
134 sc.index.names = ["分类编码", "i"]
135 return B, pd.DataFrame(rows).set_index("c"), sc.reset_index().drop(columns=["i"])
136
137
138 def mk(st, B):
139     # 最优加价率：题面成本加成口径，网格按夹逼后价格评估
140     cp = st["W"] / (1 - st["L"])
141     mo = []
142     an = []
143     for i, c in enumerate(st.index):
144         E = abs(B[i, i])
145         if E > 1:
146             p = E / (E - 1) * cp.loc[c]
147             a = 1
148         else:
149             g = np.linspace(st.loc[c, "m5"], st.loc[c, "m95"], 201)
150             best = g[0]
151             bv = -1e100
152             for mg in g:
153                 pg = np.clip((1 + mg) * st.loc[c, "W"],
154                             st.loc[c, "p5"], st.loc[c, "p95"])
155                 v = (pg - cp.loc[c]) * st.loc[c, "mu"] * \

```

```

156         (pg / st.loc[c, "pbar"]) ** B[i, i]
157     if v > bv:
158         bv = v
159         best = mg
160     p = np.clip((1 + best) * st.loc[c, "W"],
161                 st.loc[c, "p5"], st.loc[c, "p95"])
162     a = 0
163     p = np.clip(p, st.loc[c, "p5"], st.loc[c, "p95"])
164     mo.append(p / st.loc[c, "W"] - 1)
165     an.append(a)
166     return np.array(mo), np.array(an)
167
168
169 def jp(st, B):
170     # 联合定价：交叉弹性 + 差分进化，独立基准用同一价格可行域
171     cp = st["W"].values / (1 - st["L"].values)
172     pbar = st["pbar"].values
173     mu = st["mu"].values
174     n = len(mu)
175     lo = st["p5"].values
176     hi = st["p95"].values
177
178     def prof(p):
179         s = 0.0
180         for i in range(n):
181             s += (p[i] - cp[i]) * mu[i] * np.prod((p / pbar) ** B[i])
182         return -s
183
184     res = differential_evolution(prof, list(zip(lo, hi)), seed=7, tol=1e-8)
185     pj = res.x
186     pib = np.zeros(n)
187     pi_i = np.zeros(n)
188     for i in range(n):
189         g = np.linspace(lo[i], hi[i], 201)
190         v = (g - cp[i]) * mu[i] * (g / pbar[i]) ** B[i, i]
191         pib[i] = g[np.argmax(v)]
192         pi_i[i] = v.max()
193     pi_j = np.array([(pj[i] - cp[i]) * mu[i] * np.prod((pj / pbar) ** B[i])
194                      for i in range(n)])
195     return pj, pib, pi_i, pi_j
196
197
198 def fc(el, st, cats):
199     # 参考价处外推未来 7 天基准需求
200     FQ = []
201     FS = []
202     for c in cats:

```

```

203     qb = []
204     sg = []
205     for k in range(7):
206         d = 1095 + k
207         row = np.r_[np.log(st["pbar"].values), d,
208                     ((d + 2) % 7 == np.arange(1, 7)).astype(float),
209                     (np.arange(2, 13) == 7).astype(float), 1.0]
210         lp = row @ el.loc[c, "beta"]
211         q = np.exp(lp + el.loc[c, "mse"] / 2)
212         qb.append(q)
213         sg.append(q * np.sqrt(np.exp(el.loc[c, "mse"]) - 1))
214     FQ.append(qb)
215     FS.append(sg)
216     return np.array(FQ), np.array(FS)
217
218
219 def bt(q, p, cats, idx):
220     # 后 28 天滚动回测, 返回每品类 MAPE
221     wd = idx.weekday.to_numpy()
222     mo = idx.month.to_numpy()
223     rows = []
224     for c in cats:
225         qv = q[c].values
226         pos = qv > 0
227         d = np.where(pos)[0]
228         X = np.column_stack([
229             np.log(p.values[d]), d,
230             (wd[d][:, None] == np.arange(1, 7)).astype(float),
231             (mo[d][:, None] == np.arange(2, 13)).astype(float),
232             np.ones(len(d)),
233         ])
234         fit = sm.OLS(np.log(qv[pos]), X).fit()
235         te = d >= d[-1] - 27
236         pred = np.exp(X[te] @ fit.params)
237         mape = np.mean(np.abs(pred - qv[d[te]]) / qv[d[te]]) * 100
238         rows.append({"c": c, "mape": mape, "n": int(te.sum())})
239     return pd.DataFrame(rows).set_index("c")
240
241
242 def fig(st, el, B, sc, q, idx, FQ, FS, pst, R, ep, names, dts):
243     # 输出 4 张论文图
244     cats = list(st.index)
245     for ax, (c, r) in zip(
246         plt.subplots(2, 3, figsize=(15, 9))[1].ravel(), el.iterrows()):
247         g = sc[sc["分类编码"] == c]
248         x = g["ln价格"].values
249         yp = r["res"] + r["b0"] * x

```



```

250     ax.scatter(x, yp, s=4, alpha=0.4, color="#4C72B0")
251     xs = np.linspace(x.min(), x.max(), 50)
252     ax.plot(xs, r["b0"] * xs, color="#D62728", lw=1.5)
253     ax.text(0.04, 0.96, f"{names[c]}  beta={r['b0']:.3f} "
254             f"t={r['t']:.2f} R2={r['r2']:.3f}",
255             transform=ax.transAxes, va="top", fontsize=8)
256     ax.set_xlabel("ln 价格", fontsize=8)
257     ax.set_ylabel("ln 销量(去日历效应)", fontsize=8)
258     plt.tight_layout()
259     plt.savefig(FIG / "q2_elasticity_fit.pdf")
260     plt.close()
261     fig, axs = plt.subplots(2, 3, figsize=(15, 9))
262     for ax, (i, c) in zip(axs.ravel(), enumerate(cats)):
263         h = q[c].iloc[-60:]
264         ax.plot(idx[-60:], h.values, color="#4C72B0", lw=1.0, label="历史")
265         d = pd.to_datetime(dts)
266         ax.plot(d, FQ[i], "o-", color="#D62728", lw=1.2, label="预测")
267         ax.fill_between(d, FQ[i] - FS[i], FQ[i] + FS[i], color="#D62728", alpha=0.2)
268         ax.text(0.04, 0.96, names[c], transform=ax.transAxes, va="top", fontsize=8)
269         ax.set_xlabel("日期", fontsize=8)
270         ax.set_ylabel("净销量(kg)", fontsize=8)
271         ax.legend(fontsize=7)
272     plt.tight_layout()
273     plt.savefig(FIG / "q2_forecast.pdf")
274     plt.close()
275     fig, axs = plt.subplots(2, 3, figsize=(15, 9))
276     for ax, (i, c) in zip(axs.ravel(), enumerate(cats)):
277         p = np.linspace(st.loc[c, "pbar"] * 0.5, st.loc[c, "pbar"] * 2.0, 80)
278         prof = (p - st.loc[c, "w"] / (1 - st.loc[c, "L"])) * st.loc[c, "mu"] * \
279             (p / st.loc[c, "pbar"]) ** B[i, i]
280         ax.axvspan(st.loc[c, "p5"], st.loc[c, "p95"], color="#4C72B0", alpha=0.08)
281         ax.plot(p, prof, color="#4C72B0", lw=1.5)
282         ax.axvline(pst[i], color="#D62728", ls="--", lw=1.2)
283         ax.text(0.04, 0.96, f"{names[c]}  P*={pst[i]:.2f}",
284             transform=ax.transAxes, va="top", fontsize=8)
285         ax.set_xlabel("售价(元/kg)", fontsize=8)
286         ax.set_ylabel("期望利润(元)", fontsize=8)
287     plt.tight_layout()
288     plt.savefig(FIG / "q2_price_curve.pdf")
289     plt.close()
290     fig, axs = plt.subplots(2, 3, figsize=(15, 9))
291     for ax, (i, c) in zip(axs.ravel(), enumerate(cats)):
292         ax.bar(range(7), R[i], 0.6, color="#4C72B0", label="补货量")
293         ax2 = ax.twinx()
294         ax2.plot(range(7), pst[i] * np.ones(7), "o-", color="#D62728", lw=1.3,
295             label="最优售价")
296         ax2.set_ylabel("售价(元/kg)", fontsize=8)

```

```

297     ax.set_xticks(range(7))
298     ax.set_xticklabels([d[5:] for d in dts], fontsize=7, rotation=30)
299     ax.set_ylabel("补货量(kg)", fontsize=8)
300     ax.text(0.04, 0.96, names[c], transform=ax.transAxes, va="top", fontsize=8)
301 plt.tight_layout()
302 plt.savefig(FIG / "q2_replenishment.pdf")
303 plt.close()
304
305
306 def main():
307     df, names = load()
308     cats, idx, q, p, w, l = agg(df)
309     st = stat(q, p, w, l, names)
310     B, el, sc = ols(q, p, cats, idx)
311     back = bt(q, p, cats, idx)
312     mo, an = mk(st, B)
313     FQ, FS = fc(el, st, cats)
314     cp = st["W"].values / (1 - st["L"].values)
315     pst = (1 + mo) * st["W"].values
316     pj, pib, pi_i, pi_j = jp(st, B)
317     sca = (pst / st["pbar"].values) ** np.diag(B)
318     mu = FQ * sca[:, None]
319     sg = FS * sca[:, None]
320     y, kap, R, R0, ep = nv(
321         mu.ravel(), sg.ravel(),
322         np.broadcast_to(pst[:, None], mu.shape).ravel(),
323         np.broadcast_to(cp[:, None], mu.shape).ravel(),
324         np.broadcast_to(st["L"].values[:, None], mu.shape).ravel())
325     y = y.reshape(mu.shape)
326     kap = kap.reshape(mu.shape)
327     R = R.reshape(mu.shape)
328     R0 = R0.reshape(mu.shape)
329     ep = ep.reshape(mu.shape)
330     dts = pd.date_range("2023-07-01", periods=7).strftime("%Y-%m-%d")
331     e1, e2, e3 = [], [], []
332     for i, c in enumerate(cats):
333         e1.append({
334             "分类编码": c, "分类名称": names[c], "价格弹性": B[i, i],
335             "t值": el.loc[c, "t"], "R2": el.loc[c, "r2"],
336             "截距": el.loc[c, "beta"][-1], "参考价": st.loc[c, "pbar"],
337             "参考需求": st.loc[c, "mu"], "有效成本": cp[i],
338             "加价率P5": st.loc[c, "m5"], "加价率P95": st.loc[c, "m95"],
339             "最优加价率": mo[i], "是否解析解": an[i],
340         })
341     for k in range(7):
342         e2.append({"分类编码": c, "分类名称": names[c], "日期": dts[k],
343             "基准需求": FQ[i, k], "需求标准差": FS[i, k],

```

```

344         "参考价": st.loc[c, "pbar"]})
345     e3.append({"分类编码": c, "分类名称": names[c], "日期": dts[k],
346             "批发价": st.loc[c, "W"], "最优售价": pst[i],
347             "最优加价率": mo[i], "期望需求": mu[i, k],
348             "临界比": kap[i, k], "补货量": R[i, k],
349             "确定性补货量": R0[i, k], "期望利润": ep[i, k]})
350 pd.DataFrame(e1).to_csv(OUT / "q2_elasticity.csv", index=False,
351                       encoding="utf-8-sig")
352 pd.DataFrame(e2).to_csv(OUT / "q2_forecast.csv", index=False,
353                       encoding="utf-8-sig")
354 pd.DataFrame(e3).to_csv(OUT / "q2_replenishment.csv", index=False,
355                       encoding="utf-8-sig")
356 pd.DataFrame({"分类编码": cats, "分类名称": [names[c] for c in cats],
357             "7天总补货量": R.sum(axis=1), "7天总期望利润": ep.sum(axis=1)}
358             ).to_csv(OUT / "q2_summary.csv", index=False,
359                     encoding="utf-8-sig")
360 pd.DataFrame({"分类编码": cats, "分类名称": [names[c] for c in cats],
361             "独立最优价": np.round(pib, 3),
362             "联合最优价": np.round(pj, 3),
363             "独立利润": np.round(pi_i, 2),
364             "联合利润": np.round(pi_j, 2)}
365             ).to_csv(OUT / "q2_joint_pricing.csv", index=False,
366                     encoding="utf-8-sig")
367 sc.to_csv(OUT / "q2_scatter.csv", index=False, encoding="utf-8-sig")
368 back.reset_index().rename(columns={"c": "分类编码"}).to_csv(
369     OUT / "q2_backtest.csv", index=False, encoding="utf-8-sig")
370 fig(st, el, B, sc, q, idx, FQ, FS, pst, R, ep, names, dts)
371 if DEBUG:
372     lp = np.log(p.values)
373     lp = lp[~np.isnan(lp).any(axis=1)]
374     print("[debug] cond(lnP): %.1f" % np.linalg.cond(np.corrcoef(lp.T)))
375     print("[debug] 弹性:", np.round(np.diag(B), 4))
376     print("[debug] t值:", el["t"].round(2).to_dict())
377     print("[debug] MAPE:", back["mape"].round(1).to_dict())
378     print("[debug] 独立定价总利润: %.2f 联合定价总利润: %.2f 提升: %.1f%%"
379           % (pi_i.sum(), pi_j.sum(), (pi_j.sum() / pi_i.sum() - 1) * 100))
380     print("[debug] 总补货: %.2f 总利润: %.2f" % (R.sum(), ep.sum()))
381 print("[save] q2_elasticity.csv / q2_forecast.csv / "
382       "q2_replenishment.csv / q2_summary.csv / q2_scatter.csv / "
383       "q2_backtest.csv / q2_joint_pricing.csv")
384 print("[save] q2_elasticity_fit.pdf / q2_forecast.pdf / "
385       "q2_price_curve.pdf / q2_replenishment.pdf")
386
387
388 if __name__ == "__main__":
389     main()
390

```

```

391 # question2/q2_sensitivity.py
392 # 问题二灵敏度与模型检验：批发价/弹性/口径/损耗率/报童基线/同期回测/相关模拟
393
394 import json
395 import sys
396 from pathlib import Path
397
398 import matplotlib.pyplot as plt
399 import numpy as np
400 import pandas as pd
401 from scipy.stats import lognorm, norm
402
403 sys.path.insert(0, str(Path(__file__).resolve().parent))
404 import q2
405
406 OUT = q2.OUT
407 FIG = q2.FIG
408 N = 20000
409 SEED = 7
410
411
412 def d2i(s):
413     return q2.d2i(s)
414
415
416 def design(d, wd, mo, lp):
417     # 与 q2.ols 一致的回归设计矩阵
418     return np.column_stack([lp, d,
419                             (wd[d][:, None] == np.arange(1, 7)).astype(float),
420                             (mo[d][:, None] == np.arange(2, 13)).astype(float),
421                             np.ones(len(d))])
422
423
424 def mk2(st, B, cp, ws):
425     # 最优加价率(可换批发价/成本口径), 与 q2.mk 同逻辑
426     mo = []
427     an = []
428     for i, c in enumerate(st.index):
429         E = abs(B[i, i])
430         if E > 1:
431             p = E / (E - 1) * cp.loc[c]
432             a = 1
433         else:
434             g = np.linspace(st.loc[c, "m5"], st.loc[c, "m95"], 201)
435             best = g[0]
436             bv = -1e100
437             for mg in g:

```

```

438         pg = np.clip((1 + mg) * ws.loc[c], st.loc[c, "p5"], st.loc[c, "p95"])
439         v = (pg - cp.loc[c]) * st.loc[c, "mu"] * \
440             (pg / st.loc[c, "pbar"]) ** B[i, i]
441         if v > bv:
442             bv = v
443             best = mg
444         p = np.clip((1 + best) * ws.loc[c], st.loc[c, "p5"], st.loc[c, "p95"])
445         a = 0
446         p = np.clip(p, st.loc[c, "p5"], st.loc[c, "p95"])
447         mo.append(p / ws.loc[c] - 1)
448         an.append(a)
449     return np.array(mo), np.array(an)
450
451
452 def run(st, B, FQ, FS, cp, lv, ws):
453     # 给定成本/损耗/批发价口径跑 定价+预测+报童
454     mo, an = mk2(st, B, cp, ws)
455     pst = (1 + mo) * ws.values
456     sca = (pst / st["pbar"].values) ** np.diag(B)
457     mu = FQ * sca[:, None]
458     sg = FS * sca[:, None]
459     y, kap, R, R0, ep = q2.nv(
460         mu.ravel(), sg.ravel(),
461         np.broadcast_to(pst[:, None], mu.shape).ravel(),
462         np.broadcast_to(cp.values[:, None], mu.shape).ravel(),
463         np.broadcast_to(lv[:, None], mu.shape).ravel())
464     return (y.reshape(mu.shape), kap.reshape(mu.shape), R.reshape(mu.shape),
465            R0.reshape(mu.shape), ep.reshape(mu.shape), mo, pst, mu, sg)
466
467
468 def emin(mu, sg, y):
469     # lognormal E[min(D,y)]
470     s2 = np.log(1 + (sg / mu) ** 2)
471     m1 = np.log(mu) - s2 / 2
472     s = np.sqrt(s2)
473     a = mu * norm.cdf((np.log(y) - m1 - s * s) / s)
474     b = y * (1 - norm.cdf((np.log(y) - m1) / s))
475     return a + b
476
477
478 def altP(df, cats, idx, mode):
479     # 替代价格口径：当期销售额加权 / 中位价
480     tmp = df[df["均价"] > 0]
481     if mode == "cur":
482         g = tmp.assign(amt=tmp["均价"] * tmp["售出量"]).groupby(
483             ["d", "分类编码"]).apply(
484             lambda x: x["amt"].sum() / max(x["售出量"].sum(), 1e-9))

```

```

485 else:
486     g = tmp.groupby(["d", "分类编码"])["均价"].median()
487     g = g.unstack()
488     d = (idx - pd.Timestamp(q2.DATE0)).days
489     g = g.reindex(d)
490     for c in cats:
491         g[c] = g[c].fillna(g[c].mean())
492     return g
493
494
495 def bt22(q, p, cats, idx):
496     # 2022-07 同期回测：用 2022-06-30 前样本拟合，预测 7/1-7/7
497     wd = idx.weekday.to_numpy()
498     mo = idx.month.to_numpy()
499     cut = d2i("2022-07-01")
500     rows = []
501     for c in cats:
502         qv = q[c].values
503         pos = qv > 0
504         d = np.where(pos)[0]
505         tr = d < cut
506         X = design(d, wd, mo, np.log(p[c].values[d]))
507         fit = np.linalg.lstsq(X[tr], np.log(qv[pos][tr]), rcond=None)[0]
508         d7 = np.arange(cut, cut + 7)
509         lp = np.log(p[c].values[d7])
510         if not np.isfinite(lp).all():
511             lp = np.where(np.isfinite(lp), lp, np.log(p[c].mean()))
512         X7 = design(d7, wd, mo, lp)
513         pred = np.exp(X7 @ fit)
514         act = qv[d7]
515         m = act > 0
516         mape = np.mean(np.abs(pred[m] - act[m]) / act[m]) * 100
517         rows.append({"c": c, "mape22": mape, "n": int(m.sum())})
518     return pd.DataFrame(rows).set_index("c")
519
520
521 def sim(cats, idx, q, mu, sg, pst, R, cp, lv):
522     # 品类相关联合模拟：残差相关矩阵 + 高斯 copula
523     wd = idx.weekday.to_numpy()
524     mo = idx.month.to_numpy()
525     Xc = np.column_stack([
526         (wd[:, None] == np.arange(1, 7)).astype(float),
527         (mo[:, None] == np.arange(2, 13)).astype(float), np.ones(len(idx))]
528     E = np.column_stack([q[c].values for c in cats])
529     res = E - Xc @ np.linalg.lstsq(Xc, E, rcond=None)[0]
530     C = np.corrcoef(res.T)
531     rng = np.random.default_rng(SEED)

```

```

532 y = R * (1 - lv[:, None])
533 s2 = np.log(1 + (sg / mu) ** 2)
534 m1 = np.log(mu) - s2 / 2
535 s = np.sqrt(s2)
536 ej = 0.0
537 vj = 0.0
538 vi = 0.0
539 for k in range(7):
540     z = rng.multivariate_normal(np.zeros(len(cats)), C, size=N)
541     D = lognorm.ppf(norm.cdf(z), s=s[:, k], scale=np.exp(m1[:, k]))
542     prof = (pst[None, :] * np.minimum(D, y[:, k]) -
543             cp[None, :] * y[:, k]).sum(axis=1)
544     ej += prof.mean()
545     vj += prof.var()
546     zi = rng.multivariate_normal(
547         np.zeros(len(cats)), np.eye(len(cats)), size=N)
548     Di = lognorm.ppf(norm.cdf(zi), s=s[:, k], scale=np.exp(m1[:, k]))
549     profi = (pst[None, :] * np.minimum(Di, y[:, k]) -
550             cp[None, :] * y[:, k]).sum(axis=1)
551     vi += profi.var()
552 return ej, C, vj, vi
553
554
555 def fig(rows):
556     # 各情景 7 天总利润对比
557     names = [r[0] for r in rows]
558     vals = [r[1] for r in rows]
559     fig, ax = plt.subplots(figsize=(9, 5))
560     x = np.arange(len(names))
561     col = ["#4C72B0"] * len(names)
562     col[0] = "#D62728"
563     ax.bar(x, vals, 0.6, color=col)
564     for i, v in enumerate(vals):
565         ax.text(i, v + 5, "%.0f" % v, ha="center", fontsize=8)
566     ax.set_xticks(x)
567     ax.set_xticklabels(names, fontsize=8, rotation=20)
568     ax.set_ylabel("7天总利润(元)")
569     fig.tight_layout()
570     fig.savefig(FIG / "q2_sensitivity.pdf")
571     plt.close(fig)
572
573
574 def main():
575     df, names = q2.load()
576     cats, idx, q, p, w, l = q2.agg(df)
577     st = q2.stat(q, p, w, l, names)
578     B, el, sc = q2.ols(q, p, cats, idx)

```

```

579 back = q2.bt(q, p, cats, idx)
580 FQ, FS = q2.fc(el, st, cats)
581 cp0 = st["W"] / (1 - st["L"])
582 base = run(st, B, FQ, FS, cp0, st["L"].values, st["W"])
583 y0, kap0, R0, R00, ep0, mo0, pst0, mu0, sg0 = base
584 mo1, an1 = q2.mk(st, B)
585 assert np.allclose(mo1, mo0), "mk2 与 q2.mk 不一致"
586 rows = []
587 det = []
588
589 def add(nm, B_, mo_, pst_, R_, ep_):
590     for i, c in enumerate(cats):
591         rows.append({"情景": nm, "分类编码": c, "分类名称": names[c],
592                     "价格弹性": B_[i, i], "最优加价率": mo_[i],
593                     "最优售价": pst_[i],
594                     "7天补货量": R_[i].sum(), "7天利润": ep_[i].sum()})
595
596 add("基准", B, mo0, pst0, R0, ep0)
597 tot = pd.DataFrame(rows).groupby("情景")[["7天补货量", "7天利润"].sum()
598 agg = [{"情景": "基准", "7天总补货量": tot.loc["基准", "7天补货量"],
599         "7天总利润": tot.loc["基准", "7天利润"], "利润变化%": 0.0}]
600
601 def scn(nm, st_, B_, FQ_, FS_, cp_, lv_, ws_):
602     r = run(st_, B_, FQ_, FS_, cp_, lv_, ws_)
603     add(nm, B_, r[5], r[6], r[2], r[4])
604     b = tot.loc["基准", "7天利润"]
605     v = r[4].sum()
606     agg.append({"情景": nm, "7天总补货量": r[2].sum(),
607               "7天总利润": v, "利润变化%": (v / b - 1) * 100})
608     return r
609
610 for f, nm in [(1.1, "批发价+10%"), (0.9, "批发价-10%")]:
611     ws = st["W"] * f
612     cpf = ws / (1 - st["L"])
613     scn(nm, st, B, FQ, FS, cpf, st["L"].values, ws)
614 for f, nm in [(1.2, "弹性+20%"), (0.8, "弹性-20%")]:
615     Bm = B.copy()
616     np.fill_diagonal(Bm, np.diag(B) * f)
617     scn(nm, st, Bm, FQ, FS, cp0, st["L"].values, st["W"])
618 win = df[(df["d"] >= d2i("2022-07-01")) & (df["d"] <= d2i("2023-06-30"))]
619 iq = win.groupby("单品编码")["净销量"].sum()
620 cq = win.groupby("分类编码")["净销量"].sum()
621 cat = df.drop_duplicates("单品编码").set_index("单品编码")["分类编码"]
622 wgt = (iq / cat.map(cq)).fillna(0)
623 li = df.drop_duplicates("单品编码").set_index("单品编码")["损耗率"] / 100
624 Li = (li * wgt).groupby(cat).sum().reindex(st.index)
625 cpl = st["W"] / (1 - Li)

```



```

626 scn("损耗率=单品加权", st, B, FQ, FS, cpL, Li.values, st["W"])
627 for mode, nm in [("cur", "价格口径=当期加权"), ("med", "价格口径=中位价")]:
628     pa = altP(df, cats, idx, mode)
629     sta = q2.stat(q, pa, w, l, names)
630     Ba, ela, sca = q2.ols(q, pa, cats, idx)
631     FQa, FSa = q2.fc(ela, sta, cats)
632     cpa = sta["W"] / (1 - sta["L"])
633     scn(nm, sta, Ba, FQa, FSa, cpa, sta["L"].values, sta["W"])
634
635 # 报童 vs 确定性
636 ydet = R00 * (1 - st["L"].values[:, None])
637 e0 = (pst0[:, None] * emin(mu0, sg0, ydet) -
638       cp0.values[:, None] * ydet)
639 sd2 = np.log(1 + (sg0 / mu0) ** 2)
640 sd1 = np.sqrt(sd2)
641 mdl = np.log(mu0) - sd2 / 2
642 sr_det = 1 - lognorm.cdf(ydet, s=sd1, scale=np.exp(mdl))
643 for i, c in enumerate(cats):
644     det.append({
645         "分类编码": c, "分类名称": names[c],
646         "报童7天补货量": R0[i].sum(), "确定性7天补货量": R00[i].sum(),
647         "安全库存增量%": (R0[i].sum() / R00[i].sum() - 1) * 100,
648         "报童7天期望利润": ep0[i].sum(),
649         "确定性订货7天期望利润": e0[i].sum(),
650         "报童日均缺货率": (1 - kap0[i]).mean(),
651         "确定性日均缺货率": sr_det[i].mean(),
652     })
653
654 # 2022-07 同期回测
655 b22 = bt22(q, p, cats, idx)
656 bcmp = b22.join(back["mape"]).rename(
657     columns={"mape": "mape28"})
658
659 # 品类相关联合模拟
660 ej, C, vj, vi = sim(cats, idx, q, mu0, sg0, pst0, R0, cp0.values, st["L"].values)
661 ei = ep0.sum()
662 srow = {"独立7天期望利润": ei, "相关模拟7天期望利润": ej,
663         "期望利润差": ej - ei, "期望利润差%": (ej / ei - 1) * 100,
664         "独立方差(模拟)": vi, "相关方差(模拟)": vj,
665         "方差比": vj / vi, "样本量": N,
666         "残差相关最大非对角": np.abs(C - np.eye(len(C))).max(),
667         "残差相关最小非对角": C[np.triu_indices(len(C), 1)].min()}
668
669 pd.DataFrame(rows).to_csv(OUT / "q2_sens_detail.csv", index=False,
670                          encoding="utf-8-sig")
671 pd.DataFrame(agg).to_csv(OUT / "q2_sensitivity.csv", index=False,
672                          encoding="utf-8-sig")

```

```

673 pd.DataFrame(det).to_csv(OUT / "q2_nv_det.csv", index=False,
674                        encoding="utf-8-sig")
675 bcmp.reset_index().rename(columns={"c": "分类编码"}).to_csv(
676     OUT / "q2_backtest_2022.csv", index=False, encoding="utf-8-sig")
677 pd.DataFrame([srow]).to_csv(OUT / "q2_joint_sim.csv", index=False,
678                        encoding="utf-8-sig")
679
680 # 汇总 JSON 契约
681 b = tot.loc["基准", "7天利润"]
682 rw = {r["情景"]: r["7天总利润"] for r in agg}
683 rR = {r["情景"]: r["7天总补货量"] for r in agg}
684 summary = [
685     {"claim": "批发价±10% 内利润不剧烈恶化",
686      "sources": ["q2_sensitivity.csv", "q2_replenishment.csv"],
687      "perturbation": "W x 1.1 / x 0.9",
688      "metric": "7天总利润相对变化绝对值",
689      "threshold": 30.0,
690      "observed": max(abs((rw["批发价+10%"] / b - 1) * 100),
691                      abs((rw["批发价-10%"] / b - 1) * 100)),
692      "status": "PASS" if max(abs((rw["批发价+10%"] / b - 1) * 100),
693                              abs((rw["批发价-10%"] / b - 1) * 100)) <= 30
694                      else "CONDITIONAL",
695      "limitation": "补货量用近一周均价预测，未建模日内批发价随机性"},
696     {"claim": "弹性±20% 内最优价与利润稳定",
697      "sources": ["q2_sensitivity.csv"],
698      "perturbation": "品类弹性 x 1.2 / x 0.8",
699      "metric": "7天总利润相对变化绝对值",
700      "threshold": 30.0,
701      "observed": max(abs((rw["弹性+20%"] / b - 1) * 100),
702                      abs((rw["弹性-20%"] / b - 1) * 100)),
703      "status": "PASS" if max(abs((rw["弹性+20%"] / b - 1) * 100),
704                              abs((rw["弹性-20%"] / b - 1) * 100)) <= 30
705                      else "CONDITIONAL",
706      "limitation": "仅扰动对角弹性，交叉弹性不变"},
707     {"claim": "价格口径切换不改变弹性符号与利润量级",
708      "sources": ["q2_sensitivity.csv", "q2_sens_detail.csv"],
709      "perturbation": "固定权重 -> 当期销售额加权 / 中位价",
710      "metric": "7天总利润相对固定权重变化绝对值",
711      "threshold": 30.0,
712      "observed": max(abs((rw["价格口径=当期加权"] / b - 1) * 100),
713                      abs((rw["价格口径=中位价"] / b - 1) * 100)),
714      "status": "PASS" if max(abs((rw["价格口径=当期加权"] / b - 1) * 100),
715                              abs((rw["价格口径=中位价"] / b - 1) * 100)) <= 30
716                      else "CONDITIONAL",
717      "limitation": "未对口径切换重估报童分位数分布参数"},
718     {"claim": "报童订货优于确定性订货",
719      "sources": ["q2_nv_det.csv"],

```

```

720     "perturbation": "最优分位数  $y^*$  vs 均值  $y_0$ ",
721     "metric": "报童7天期望利润  $\geq$  确定性订货期望利润",
722     "threshold": 0.0,
723     "observed": (ep0.sum() - e0.sum()),
724     "status": "PASS" if ep0.sum()  $\geq$  e0.sum() - 1e-6 else "CONDITIONAL",
725     "limitation": "确定性口径按 lognormal 期望利润评估, 缺货率口径为  $P(D > y)$ "},
726 {"claim": "损耗率替换后补货量稳定",
727     "sources": ["q2_sensitivity.csv"],
728     "perturbation": "分类级损耗率  $\rightarrow$  单品销量加权损耗率",
729     "metric": "7天总补货量相对变化绝对值",
730     "threshold": 20.0,
731     "observed": abs(rR["损耗率=单品加权"] / rR["基准"] - 1) * 100,
732     "status": "PASS" if abs(rR["损耗率=单品加权"] / rR["基准"] - 1) * 100  $\leq$  20
733         else "CONDITIONAL",
734     "limitation": "损耗率口径差异来自附件4两类表, 取加权口径作对照"},
735 {"claim": "2022-07 同期回测误差与后28天同量级",
736     "sources": ["q2_backtest_2022.csv", "q2_backtest.csv"],
737     "perturbation": "样本截止 2022-06-30",
738     "metric": "2022-07 预测 MAPE  $\leq$  40%",
739     "threshold": 40.0,
740     "observed": float(b22["mape22"].max()),
741     "status": "PASS" if b22["mape22"].max()  $\leq$  40 else "CONDITIONAL",
742     "limitation": "目标周仅 7 天, 实际销量个别日为 0 时剔除"},
743 {"claim": "品类相关性与独立优化期望收益一致",
744     "sources": ["q2_joint_sim.csv"],
745     "perturbation": "残差相关矩阵 vs 单位阵(高斯 copula + lognormal 边际)",
746     "metric": "相关模拟期望利润相对独立期望利润偏差  $\leq$  1%",
747     "threshold": 1.0,
748     "observed": abs((ej / ei - 1) * 100),
749     "status": "PASS" if abs((ej / ei - 1) * 100)  $\leq$  1 else "CONDITIONAL",
750     "limitation": "copula 相关为残差 Pearson 相关, 与 Spearman 口径略有差异"},
751 ]
752 with open(OUT / "q2_robustness_summary.json", "w", encoding="utf-8") as f:
753     json.dump(summary, f, ensure_ascii=False, indent=2)
754
755 frows = [(r["情景"], r["7天总利润"]) for r in agg]
756 fig(frows)
757 if q2.DEBUG:
758     print("[debug] 基准总利润: %.2f 总补货: %.2f" % (b, rR["基准"]))
759     for r in agg:
760         print("[debug] %s: 利润 %.2f (%+.1f%) 补货 %.2f" %
761               (r["情景"], r["7天总利润"], r["利润变化%"], r["7天总补货量"]))
762     print("[debug] 报童-确定性利润差: %.2f" % (ep0.sum() - e0.sum()))
763     print("[debug] 2022-07 MAPE:", b22["mape22"].round(1).to_dict())
764     print("[debug] 相关模拟: 独立 %.2f vs 相关 %.2f, 方差比 %.3f" %
765           (ei, ej, vj / vi))
766     print("[debug] 残差相关矩阵:\n", np.round(C, 3))

```

```

767     print("[save] q2_sensitivity.csv / q2_sens_detail.csv / q2_nv_det.csv / "
768           "q2_backtest_2022.csv / q2_joint_sim.csv / "
769           "q2_robustness_summary.json / q2_sensitivity.pdf")
770
771
772 if __name__ == "__main__":
773     main()

```

2.4 问题三代码

```

1  # question3/q3.py
2  # 问题三主流程：候选集 -> 弹性 -> 网格 -> MIP -> 灵敏度
3
4  from pathlib import Path
5
6  import matplotlib.pyplot as plt
7  import numpy as np
8  import pandas as pd
9  import statsmodels.api as sm
10 from scipy.optimize import Bounds, LinearConstraint, milp
11 from scipy.sparse import coo_matrix
12
13
14 ROOT = Path(__file__).resolve().parents[2]
15 OUT = ROOT / "results"
16 FIG = ROOT / "figures"
17 FIG.mkdir(parents=True, exist_ok=True)
18 DATE0 = "2020-07-01"
19 DATE1 = "2023-06-30"
20 K = 11
21 K7 = 7
22 NMIN = 14
23 M0 = 30
24 LO = 27
25 HI = 33
26 MINQ = 2.5
27 RHOS = [0.5, 1.0, 1.5]
28 EXACT = [27, 30, 33]
29 DEBUG = True
30
31 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei"]
32 plt.rcParams["axes.unicode_minus"] = False
33
34
35 def d2i(s):
36     return (pd.Timestamp(s) - pd.Timestamp(DATE0)).days

```

```

37
38
39 def load():
40     df = pd.read_csv(OUT / "item_daily_full.csv", encoding="utf-8-sig")
41     df["日期"] = pd.to_datetime(df["销售日期"])
42     df["d"] = (df["日期"] - pd.Timestamp(DATE0)).dt.days
43     names = df.drop_duplicates("单品编码").set_index("单品编码")["单品名称"].to_dict
44         ()
45     e1 = pd.read_csv(OUT / "q2_elasticity.csv", encoding="utf-8-sig")
46     Bcat = dict(zip(e1["分类编码"], e1["价格弹性"]))
47     return df, names, Bcat
48
49 def cand(df):
50     # 候选集与基准需求/参考价/批发价/损耗率
51     w = df[(df["d"] >= d2i("2023-06-24")) & (df["d"] <= d2i("2023-06-30"))]
52     g = w.groupby("单品编码").agg(
53         名称=("单品名称", "first"), 品类=("分类编码", "first"),
54         品类名=("分类名称", "first"), 销量=("净销量", "sum"),
55         天数=("净销量", "count"), 批发=("批发价", "mean"),
56         损耗=("损耗率", "mean"), 损耗c=("平均损耗率", "mean"))
57     g = g[g["销量"] / 7 >= 0.5]
58     g["d"] = g["销量"] / 7
59     g["W"] = g["批发"]
60     g["L"] = g["损耗"] / 100
61     g["Lc"] = g["损耗c"] / 100
62     g["low"] = g["天数"] < 3
63     amt = w.assign(amt=w["净销量"] * w["均价"])
64     p0 = amt.groupby("单品编码").apply(
65         lambda x: x["amt"].sum() / max(x["净销量"].sum(), 1e-9))
66     g["p0"] = p0
67     g["mx"] = df.groupby("单品编码")["净销量"].max()
68     w4 = df[(df["d"] >= d2i("2023-06-03")) & (df["d"] <= d2i("2023-06-30"))]
69     d4 = w4.groupby("单品编码")["净销量"].mean()
70     tmp = w4[w4["均价"] > 0]
71     pp = tmp.groupby("单品编码")["均价"].apply(
72         lambda x: pd.Series([np.percentile(x, 5), np.percentile(x, 95)]).unstack())
73     pp.columns = ["p5", "p95"]
74     g = g.join(pp)
75     g["d"] = g["d"].fillna(d4)
76     g["p0"] = g["p0"].fillna(
77         w4.assign(amt=w4["净销量"] * w4["均价"]).groupby("单品编码").apply(
78             lambda x: x["amt"].sum() / max(x["净销量"].sum(), 1e-9))
79     g["W"] = g["W"].fillna(df.groupby("单品编码")["批发价"].mean())
80     g["p5"] = g["p5"].fillna(g["p0"])
81     g["p95"] = g["p95"].fillna(g["p0"])
82     for code in g.index[g["low"]]:

```

```

83     if code in d4.index:
84         g.loc[code, "d"] = d4[code]
85     return g
86
87
88 def beta(df, g, Bcat):
89     # 单品弹性回归 + Shrinkage 收缩 + 兜底
90     rows = []
91     for code, r in g.iterrows():
92         sub = df[(df["单品编码"] == code) & (df["净销量"] > 0) &
93                 (df["均价"] > 0)].sort_values("d")
94         y = np.log(sub["净销量"].values)
95         p = sub["均价"].values
96         d = sub["d"].values
97         wd = sub["日期"].dt.weekday.to_numpy()
98         mo = sub["日期"].dt.month.to_numpy()
99         X = np.column_stack([
100             np.log(p), d,
101             (wd[:, None] == np.arange(1, 7)).astype(float),
102             (mo[:, None] == np.arange(2, 13)).astype(float),
103             np.ones(len(d)),
104         ])
105         try:
106             fit = sm.OLS(y, X).fit()
107             raw = float(fit.params[0])
108         except Exception:
109             raw = np.nan
110         N = int((np.diff(p) != 0).sum())
111         cat = Bcat[r["品类"]]
112         if N < NMIN or not np.isfinite(raw):
113             b, u = cat, 1
114         else:
115             w = N / (N + M0)
116             b = w * raw + (1 - w) * cat
117             u = 0
118             if b > 0:
119                 b, u = cat, 1
120         rows.append((code, raw, b, N, u, cat))
121     return pd.DataFrame(rows, columns=["单品编码", "原始弹性", "收缩弹性",
122                                       "有效样本量", "是否借用品类弹性",
123                                       "品类弹性"]).set_index("单品编码")
124
125
126 def grid(g, K=K):
127     ps = {}
128     for code, r in g.iterrows():
129         lo = min(r["p5"], r["p0"])

```

```

130     hi = max(r["p95"], r["p0"])
131     arr = np.unique(np.append(np.linspace(lo, hi, K), r["p0"]))
132     if len(arr) < K:
133         arr = np.pad(arr, (0, K - len(arr)), mode="edge")
134     else:
135         arr = arr[:K]
136     ps[code] = arr
137     return ps
138
139
140 def grid_q(g, K, w4):
141     # 分位数网格: 近 4 周日售价等概率分位 + 参考价
142     ps = {}
143     for code, r in g.iterrows():
144         v = w4.loc[w4["单品编码"] == code, "均价"].values
145         v = v[v > 0]
146         if len(v) == 0:
147             arr = np.full(K, r["p0"])
148         else:
149             arr = np.unique(
150                 np.append(np.quantile(v, np.linspace(0.05, 0.95, K)), r["p0"]))
151         if len(arr) < K:
152             arr = np.pad(arr, (0, K - len(arr)), mode="edge")
153         else:
154             arr = arr[:K]
155         ps[code] = arr
156     return ps
157
158
159 def demand(d, p0, beta, ps, bs=1.0):
160     return d[:, None] * (ps / p0[:, None]) ** (beta[:, None] * bs)
161
162
163 def mip(g, el, ps, rho, cov=True, exact_n=None, bs=1.0, ws=1.0, ms=1.0,
164         lc=False):
165     codes = list(g.index)
166     n = len(codes)
167     K = len(ps[codes[0]])
168     V = 2 + 3 * K
169     P = np.array([ps[c] for c in codes])
170     D = demand(g["d"].values, g["p0"].values, el["收缩弹性"].values, P, bs)
171     Lv = g["Lc"].values if lc else g["L"].values
172     M = np.maximum(3 * g["mx"].values, MINQ)
173     M = np.maximum(M, D[:, 0] * 1.05)
174     if ms != 1.0:
175         M = M * ms
176     cp = g["W"].values * ws / (1 - Lv)

```

```

177 gam = np.maximum(0, g["p0"].values - cp) * rho
178 nv = n * V
179 c = np.zeros(nv)
180 lb = np.zeros(nv)
181 ub = np.ones(nv)
182 integ = np.zeros(nv)
183 for i in range(n):
184     base = i * V
185     integ[base] = 1
186     integ[base + 1:base + 1 + K] = 1
187     ub[base + 1 + K] = M[i]
188     ub[base + 2 + K:base + 2 + 2 * K] = D[i].max()
189     ub[base + 2 + 2 * K:base + 1 + 3 * K] = D[i].max()
190     c[base + 1 + K] = g["W"].values[i] * ws
191     c[base + 2 + K:base + 2 + 2 * K] = -P[i]
192     c[base + 2 + 2 * K:base + 1 + 3 * K] = gam[i]
193 rows, cols, data, clb, cub = [], [], [], [], []
194 row = 0
195
196 def add(cols_, vals, lb_, ub_):
197     nonlocal row
198     for col_, val_ in zip(cols_, vals):
199         rows.append(row)
200         cols.append(col_)
201         data.append(val_)
202         clb.append(lb_)
203         cub.append(ub_)
204         row += 1
205
206 for i in range(n):
207     base = i * V
208     y, q = base, base + 1 + K
209     add([y, q], [MINQ, -1.0], [-np.inf, 0])
210     add([q, y], [1.0, -M[i]], [-np.inf, 0])
211     add(list(range(base + 1, base + 1 + K)) + [y], [1.0] * K + [-1.0], 0, 0)
212     zs = list(range(base + 2 + K, base + 2 + 2 * K))
213     add(zs + [q], [1.0] * K + [-(1 - Lv[i])], [-np.inf, 0])
214     for j in range(K):
215         xj = base + 1 + j
216         zj = base + 2 + K + j
217         uj = base + 2 + 2 * K + j
218         add([zj, uj, xj], [1.0, 1.0, -D[i, j]], 0, 0)
219 ycols = [i * V for i in range(n)]
220 add(ycols, [-1.0] * n, [-np.inf, -L0])
221 add(ycols, [1.0] * n, [-np.inf, HI])
222 if exact_n is not None:
223     add(ycols, [1.0] * n, exact_n, exact_n)

```



```

224 if cov:
225     for cname in g["品类"].unique():
226         idx = [i for i, c in enumerate(codes) if g.loc[c, "品类"] == cname]
227         add([i * V for i in idx], [-1.0] * len(idx), -np.inf, -1)
228 A = coo_matrix((data, (rows, cols)), shape=(len(clb), nv)).tocsr()
229 res = milp(c, integrality=integ, bounds=Bounds(lb, ub),
230           constraints=LinearConstraint(A, np.array(clb), np.array(cub)),
231           options={"time_limit": 60})
232 x = res.x
233 y = np.array([x[i * V] for i in range(n)])
234 sel = np.where(y > 0.5)[0]
235 q = np.array([x[i * V + 1 + K] for i in range(n)])
236 sold = np.zeros(n)
237 short = np.zeros(n)
238 dem = np.zeros(n)
239 rev = np.zeros(n)
240 price = np.zeros(n)
241 for i in range(n):
242     j = int(np.argmax(x[i * V + 1:i * V + 1 + K]))
243     price[i] = P[i, j]
244     dem[i] = D[i, j] if y[i] > 0.5 else 0
245     sold[i] = x[i * V + 2 + K + j]
246     short[i] = x[i * V + 2 + 2 * K + j]
247     rev[i] = price[i] * sold[i]
248 cost = q * g["W"].values * ws
249 pen = short * gam
250 net = rev - cost - pen
251 fr = sold.sum() / max(dem.sum(), 1e-9)
252 return {"codes": codes, "sel": sel, "y": y, "q": q, "price": price,
253        "sold": sold, "short": short, "dem": dem, "rev": rev,
254        "cost": cost, "pen": pen, "net": net, "fr": fr, "M": M,
255        "D1": D[:, 0], "Lv": Lv,
256        "status": res.status, "fun": res.fun,
257        "gap": getattr(res, "mip_gap", None),
258        "nodes": getattr(res, "mip_node_count", None)}
259
260
261 def solve(g, el, rho=1.0, K=K, qg=False, w4=None, cov=True, exact_n=None,
262          bs=1.0, ws=1.0, ms=1.0, lc=False):
263     # 基准 K=11; 非最优按 K=7 -> 放宽品类覆盖 顺序降级
264     ps = grid_q(g, K, w4) if qg else grid(g, K)
265     out = mip(g, el, ps, rho, cov, exact_n, bs, ws, ms, lc)
266     lvl = 0
267     if out["status"] != 0:
268         lvl = 1
269         if K != K7:
270             ps = grid(g, K7)

```

```

271     out = mip(g, el, ps, rho, cov, exact_n, bs, ws, ms, lc)
272     if out["status"] != 0:
273         lvl = 2
274         ps = grid(g, K7)
275         out = mip(g, el, ps, rho, False, exact_n, bs, ws, ms, lc)
276         if out["status"] != 0:
277             lvl = 3
278     out["lvl"] = lvl
279     out["ps"] = ps
280     return out
281
282
283 def fig1(g, out):
284     # 品类选中构成
285     sel = out["sel"]
286     sub = g.iloc[sel]
287     cnt = sub["品类名"].value_counts()
288     allc = g["品类名"].value_counts()
289     fig, ax = plt.subplots(figsize=(8, 5))
290     x = np.arange(len(allc))
291     ax.bar(x - 0.2, allc.values, 0.4, color="#C9D6E8", label="候选")
292     ax.bar(x + 0.2, cnt.reindex(allc.index).fillna(0).values, 0.4,
293           color="#4C72B0", label="选中")
294     ax.set_xticks(x)
295     ax.set_xticklabels(allc.index, fontsize=9)
296     ax.set_ylabel("单品数")
297     ax.legend(fontsize=8)
298     fig.tight_layout()
299     fig.savefig(FIG / "q3_selection.pdf")
300     plt.close(fig)
301
302
303 def fig2(rows):
304     # rho-利润/满足率权衡
305     fig, ax = plt.subplots(figsize=(8, 5))
306     ax.plot([r["rho"] for r in rows], [r["net"] for r in rows], "o-",
307           color="#4C72B0", label="总利润")
308     ax.set_xlabel("缺货惩罚系数 rho")
309     ax.set_ylabel("总利润(元)")
310     ax.set_xticks([r["rho"] for r in rows])
311     ax.set_ylim(670, 720)
312     ax2 = ax.twinx()
313     ax2.plot([r["rho"] for r in rows], [r["fr"] for r in rows], "s--",
314           color="#D62728", label="满足率")
315     ax2.set_ylabel("满足率")
316     ax2.set_ylim(0.94, 1.02)
317     ax.legend(fontsize=8, loc="upper left")

```

```

318     ax2.legend(fontsize=8, loc="upper right")
319     fig.tight_layout()
320     fig.savefig(FIG / "q3_fr_tradeoff.pdf")
321     plt.close(fig)
322
323
324 def fig3(g, out):
325     # 选中单品价格-订购量散点
326     sel = out["sel"]
327     sub = g.iloc[sel]
328     fig, ax = plt.subplots(figsize=(9, 6))
329     cats = sorted(sub["品类名"].unique())
330     cmap = plt.get_cmap("tab10")
331     for i, c in enumerate(cats):
332         m = sub["品类名"] == c
333         ax.scatter(out["price"][sel][m.values], out["q"][sel][m.values],
334                   s=40, color=cmap(i % 10), label=c)
335     ax.set_xlabel("售价(元/kg)")
336     ax.set_ylabel("订购量(kg)")
337     ax.legend(fontsize=8)
338     fig.tight_layout()
339     fig.savefig(FIG / "q3_price_qty.pdf")
340     plt.close(fig)
341
342
343 def main():
344     df, names, Bcat = load()
345     g = cand(df)
346     el = beta(df, g, Bcat)
347     w4 = df[(df["d"] >= d2i("2023-06-03")) & (df["d"] <= d2i("2023-06-30"))]
348     out = solve(g, el)
349     rows = []
350     rows_rho = []
351     for rho in RHOS:
352         r = solve(g, el, rho=rho)
353         rows.append({"情景": "rho=%.1f" % rho, "总利润": r["net"].sum(),
354                    "满足率": r["fr"], "选中数": len(r["sel"]),
355                    "求解状态": r["status"]})
356         rows_rho.append({"rho": rho, "net": r["net"].sum(), "fr": r["fr"]})
357     for k in EXACT:
358         r = solve(g, el, exact_n=k)
359         rows.append({"情景": "选品数=%d" % k, "总利润": r["net"].sum(),
360                    "满足率": r["fr"], "选中数": len(r["sel"]),
361                    "求解状态": r["status"]})
362     for bs in [1.2, 0.8]:
363         r = solve(g, el, bs=bs)
364         rows.append({"情景": "弹性x%.1f" % bs, "总利润": r["net"].sum(),

```

```

365         "满足率": r["fr"], "选中数": len(r["sel"]),
366         "求解状态": r["status"]})
367 for ws in [1.1, 0.9]:
368     r = solve(g, el, ws=ws)
369     rows.append({"情景": "批发价x%.1f" % ws, "总利润": r["net"].sum(),
370                "满足率": r["fr"], "选中数": len(r["sel"]),
371                "求解状态": r["status"]})
372 for name, kw in [("损耗率=分类级", {"lc": True}), ("网格K=7", {"K": K7}),
373                ("网格分位数", {"qg": True, "w4": w4}),
374                ("大M放大50%", {"ms": 1.5})]:
375     r = solve(g, el, **kw)
376     rows.append({"情景": name, "总利润": r["net"].sum(),
377                "满足率": r["fr"], "选中数": len(r["sel"]),
378                "求解状态": r["status"]})
379 sel = out["sel"]
380 dec = pd.DataFrame({
381     "单品编码": [g.index[i] for i in sel],
382     "单品名称": [names[g.index[i]] for i in sel],
383     "分类名称": [g.iloc[i]["品类名"] for i in sel],
384     "售价": np.round(out["price"][sel], 3),
385     "订购量": np.round(out["q"][sel], 3),
386     "预计销量": np.round(out["sold"][sel], 3),
387     "缺货量": np.round(out["short"][sel], 3),
388     "需求": np.round(out["dem"][sel], 3),
389     "批发价": np.round(g["W"].values[sel], 3),
390     "损耗率": np.round(g["L"].values[sel], 4),
391     "弹性": np.round(el["收缩弹性"].values[sel], 4),
392     "参考价": np.round(g["p0"].values[sel], 3),
393     "基准需求": np.round(g["d"].values[sel], 3),
394     "低样本": [bool(g["low"].iloc[i]) for i in sel],
395     "单品毛利": np.round(out["net"][sel], 2),
396 })
397 frc = {}
398 for c in g["品类名"].unique():
399     idx = [i for i in sel if g.iloc[i]["品类名"] == c]
400     if idx:
401         s = sum(out["sold"][i] for i in idx)
402         d = sum(out["dem"][i] for i in idx)
403         frc[c] = s / d if d > 0 else 1.0
404 dec.insert(dec.columns.get_loc("单品毛利"), "品类满足率",
405           [round(frc[g.iloc[i]["品类名"]], 4) for i in sel])
406 dec.to_csv(OUT / "q3_decision.csv", index=False, encoding="utf-8-sig")
407 el.reset_index().to_csv(OUT / "q3_elasticity.csv", index=False,
408                        encoding="utf-8-sig")
409 catsel = g.iloc[sel]["品类名"].value_counts()
410 allcats = g["品类名"].value_counts()
411 over = np.maximum(0, out["q"][sel] - out["sold"][sel] /

```

```

412         (1 - g["L"].values[sel]))
413 ck = {
414     "选品数达标": bool(LO <= len(sel) <= HI),
415     "最小陈列达标": bool((out["q"][sel] >= MINQ - 1e-6).all()),
416     "可售上限达标": bool((out["sold"][sel] <=
417         out["q"][sel] * (1 - g["L"].values[sel]) + 1e-6).all()),
418     "价格网格内": bool(all(any(np.isclose(out["price"][i], p)
419         for p in out["ps"][g.index[i]]) for i in sel)),
420     "品类覆盖达标": bool(len(set(g.iloc[sel]["品类"])) == len(g["品类"].unique())),
421 }
422 diag = pd.DataFrame([
423     "候选数": len(g), "选中数": len(sel), "求解状态": out["status"],
424     "最优性gap": out["gap"], "节点数": out["nodes"], "降级层级": out["lvl"],
425     "总需求": round(out["dem"].sum(), 2), "总销量": round(out["sold"].sum(), 2),
426     "总缺货": round(out["short"].sum(), 2), "总收入": round(out["rev"].sum(), 2),
427     "总成本": round(out["cost"].sum(), 2), "缺货惩罚": round(out["pen"].sum(), 2),
428     "总利润": round(out["net"].sum(), 2), "满足率": round(out["fr"], 4),
429     "超订量": round(float(over.sum()), 3),
430     "超订成本": round(float((over * g["W"].values[sel]).sum()), 2),
431     **ck, **{f"选中_{c}": int(catsel.get(c, 0)) for c in allcats.index},
432 ])
433 diag.to_csv(OUT / "q3_diagnostics.csv", index=False, encoding="utf-8-sig")
434 pd.DataFrame(rows).to_csv(OUT / "q3_sensitivity.csv", index=False,
435     encoding="utf-8-sig")
436 mb = pd.DataFrame([
437     "单品编码": g.index,
438     "大M": np.round(out["M"], 3),
439     "最低档需求": np.round(out["D1"], 3),
440     "可售上限需求": np.round(out["D1"] / (1 - g["L"].values), 3),
441     "余量": np.round(out["M"] - out["D1"] / (1 - g["L"].values), 3),
442 ])
443 mb.to_csv(OUT / "q3_mbound.csv", index=False, encoding="utf-8-sig")
444 q2 = pd.read_csv(OUT / "q2_replenishment.csv", encoding="utf-8-sig")
445 q2d = q2[q2["日期"] == "2023-07-01"].set_index("分类编码")
446 q3v = g.iloc[sel][["品类", "品类名"].copy()
447 q3v["订购"] = out["q"][sel]
448 q3v["可售"] = out["q"][sel] * (1 - g["L"].values[sel])
449 gq = q3v.groupby("品类").agg(名称=("品类名", "first"),
450     订购合计=("订购", "sum"),
451     可售合计=("可售", "sum"))
452 vs = gq.join(q2d[["补货量", "期望需求"]], how="left")
453 vs.columns = ["分类名称", "Q3订购量合计", "Q3可售量合计", "Q2订购量", "Q2可售量"]
454 vs["可售量比值"] = vs["Q3可售量合计"] / vs["Q2可售量"]
455 vs.reset_index().rename(columns={"品类": "分类编码"}).to_csv(
456     OUT / "q3_vs_q2.csv", index=False, encoding="utf-8-sig")
457 fig1(g, out)
458 fig2(rows_rho)

```

```

459 fig3(g, out)
460 if DEBUG:
461     print("[debug] 候选数:", len(g), "选中数:", len(sel))
462     print("[debug] 求解状态:", out["status"], "gap:", out["gap"],
463           "节点数:", out["nodes"], "降级层级:", out["lvl"])
464     print("[debug] 满足率: %.4f 总利润: %.2f" % (out["fr"], out["net"].sum()))
465     print("[debug] 约束检查:", ck)
466     print("[debug] 品类选中:", catsel.to_dict())
467     print("[debug] 灵敏度行数:", len(rows))
468 print("[save] q3_decision.csv / q3_elasticity.csv / q3_diagnostics.csv / "
469       "q3_sensitivity.csv / q3_mbound.csv / q3_vs_q2.csv")
470 print("[save] q3_selection.pdf / q3_fr_tradeoff.pdf / q3_price_qty.pdf")
471
472
473 if __name__ == "__main__":
474     main()

```

附录 C 支撑材料文件清单

表 32 支撑材料文件清单

目录	文件说明
code/	全部 Python 源代码
code/question1/	问题一代码 (A.py, B.py, C.py, D.py)
code/question2/	问题二代码 (q2.py, q2_sensitivity.py)
code/question3/	问题三代码 (q3.py)
figures/	全部图表 (PDF 格式)
results/	全部输出数据 (CSV 格式)

C